



GeoLibre Tutorials

Hands-on, end-to-end workflows

Contents

Overview	5
Before you start	5
The tutorials	5
Video tutorials	5
Video Tutorials	7
GeoLibre 1.0: A Free, Open-Source Cloud-Native GIS That Runs Anywhere	7
Geoprocessing in the Browser: 700+ Free GIS Tools in GeoLibre, Zero Install	8
Access Free High-Resolution Disaster Satellite Imagery in Your Browser	9
Regularize Building Footprints in the Browser with GeoLibre	10
GeoLibre + GeoLens: A Modern GIS Stack for Self-Hosting Geospatial Data	12
Create Reusable GIS Workflows with GeoLibre Model Builder and AI Assistant	14
Mapping the 2026 Nepal Floods with Free High-Resolution Satellite Imagery	15
Building Cloud-Native GIS Workflows with GeoLibre	16
Image Georeferencing Using GeoLibre in the Browser	17
100 Interactive Maps from Open Data: Explore, Fork, and Build Your Own with GeoLibre	18
More videos	19
Your First Map	20
1. Open GeoLibre	20
2. Add a layer	20
3. Style the layer	21
4. Inspect the data	21
5. Save or share	22
Next steps	22
Cloud-Native Data	23
Load a remote GeoParquet	23
Load a FlatGeobuf	23
Load a COG	23
Convert local data to cloud-native	24
Next steps	24
Vector Analysis	25
1. Load input data	25
2. Buffer a layer	25
3. Overlay two layers	26
4. Inspect and refine	26
5. Export the result	27
Next steps	27
Terrain Analysis	28
1. Load a DEM	28

2. Hillshade	28
3. Slope and aspect	28
4. Contours	29
5. Clip to an area of interest	29
Next steps	29
Spatial SQL	30
1. Query a loaded layer	30
2. Query a remote file directly	31
3. Use spatial functions	31
4. Add the result to the map	31
5. Export	31
Next steps	32
Sharing & Embedding	33
1. Connect your account	33
2. Share the project	33
3. Open the shared map	33
4. Embed it in a page	33
5. Drive the map from your page	34
Next steps	35
Format Reference	36
Preface	36
1. Runtime Configurations & Platform Overview	36
2. Vector Data Format Support	37
3. Raster & Imagery	39
4. Point Clouds & LiDAR	41
5. 3D Models, Tiles & Gaussian Splats	41
6. Tiles & OGC Services	42
7. Cloud-Native Formats & Data Catalogs	43
8. Project & Style Interoperability	46
9. Export & Format Conversion: Essentially a Free GDAL Frontend	47
10. Platform Differences: Points of Caution	48
11. Architecture: How It Manages to Support So Many Formats	49
12. Performance: Some Real Numbers	50
13. Its Boundaries	52
In Closing	52
Format Reference (中文)	54
前言	54
一、运行形态与平台说明	54
二、矢量数据格式支持	55
三、栅格与影像	57
四、点云与 LiDAR	58
五、三维模型、瓦片与高斯泼溅	58

六、瓦片与 OGC 服务	59
七、云原生格式与数据目录	61
八、工程与样式互通	63
九、导出与格式转换：相当于一个免费的 GDAL 前端	64
十、平台差异：注意事项	65
十一、架构：它凭什么能吃下这么多格式	65
十二、性能：几个真实的数字	67
十三、它的边界	68
写在最后	69
Source Code Deep Dive	70
Preface	70
1. The Toolbox: Open-Source Libraries Worth Examining Individually	70
2. DuckDB-WASM Spatial: One Library as Both Format Driver and Compute Engine	74
3. Web-Side Performance: Five Optimizations, Plus One Honest Negative Result	80
4. State Management: Three Directly Reusable Patterns	85
5. Offline Capability: Workbox Three-Tier Caching Strategy	87
6. Cloud-Native Formats: What Gets Replaced Is Not the Frontend Library, but the Entire Distribution Architecture	89
In Closing: An Adoption Checklist Ordered by Investment	94
Source Code Deep Dive (中文)	96
前言	96
一、它的工具箱：值得单独拿出来看的开源库	96
二、DuckDB-WASM Spatial：一个库同时当格式驱动和计算引擎	99
三、Web 端性能：五条优化，加一个负面结果	104
四、状态管理：三个可直接复用的小模式	109
五、离线能力：Workbox 三层缓存策略	111
六、云原生格式：它换掉的不是前端库，是整个分发架构	112
写在最后：按投入排序的采纳清单	116

<https://geolibre.app/tutorials/>

Tutorials

These tutorials walk through common GeoLibre workflows end to end. Each one is short, builds on the [User Guide](#), and links back to the reference pages for the features it uses.

Before you start

- You can follow most tutorials in the **live viewer** at web.geolibre.app, which is the browser build of GeoLibre. No installation required.
- A few steps need the **desktop app**: reading local MBTiles and rasters, and the Python sidecar tools (raster tools without a browser engine, such as Contour). These are called out where they appear. See [Downloads](#) to install. The Whitebox geoprocessing toolbox is not among them — its 1,000+ tools run in the browser on WebAssembly.
- The sample dataset used in several tutorials is a public GeoParquet file of world countries:
`https://data.source.coop/giswqs/opengeos/countries.parquet`

The tutorials

Tutorial	You will learn to
Your First Map	Add a layer, style it, inspect attributes, and save.
Cloud-Native Data	Load remote GeoParquet, FlatGeobuf, and COG, and convert local data.
Vector Analysis	Buffer and overlay layers, then export the result.
Terrain Analysis	Derive hillshade, slope, and contours from a DEM.
Spatial SQL	Query data with DuckDB Spatial and map the results.
Sharing & Embedding	Share a project and embed it in a web page.

Work through them in order for a guided tour, or jump to the one that matches your task.

Video tutorials

- [GeoLibre 1.0: A Free, Open-Source Cloud-Native GIS That Runs Anywhere \(Browser, Desktop & Jupyter\)](#)
- [Geoprocessing in the Browser: 700+ Free GIS Tools in GeoLibre, Zero Install](#)
- [Access Free High-Resolution Disaster Satellite Imagery in Your Browser](#)
- [Regularize Building Footprints in the Browser with GeoLibre](#)
- [GeoLibre + GeoLens: A Modern GIS Stack for Self-Hosting Geospatial Data](#)
- [Create Reusable GIS Workflows with GeoLibre Model Builder and AI Assistant](#)
- [Mapping the 2026 Nepal Floods with Free High-Resolution Satellite Imagery](#)

- [Building Cloud-Native GIS Workflows with GeoLibre](#)
- [Image Georeferencing Using GeoLibre in the Browser](#)
- [100 Interactive Maps from Open Data: Explore, Fork, and Build Your Own with GeoLibre](#)

All of them, with chapters and summaries, are on [Video Tutorials](#).

<https://geolibre.app/tutorials/videos/>

Video Tutorials

Recorded walkthroughs of GeoLibre, oldest first — start at the top for a guided tour, or jump to the one that matches your task. Every video is part of the [GeoLibre playlist](#) on the Open Geospatial Solutions channel — subscribe there to catch new ones.

For written, step-by-step versions of the same workflows, see the [Tutorials overview](#); for a still-image tour of the app, see [Demos](#).

[Watch the full playlist](#)

GeoLibre 1.0: A Free, Open-Source Cloud-Native GIS That Runs Anywhere



► [GeoLibre 1.0: A Free, Open-Source Cloud-Native GIS That Runs Anywhere \(Browser, Desktop and Jupyter\)](#)

40:18 · June 2026 · [Watch on YouTube](#)

The full introduction, and the best place to start. Installs and runs all three builds — browser, desktop, and Jupyter — then works through projects, vector and raster data, attribute and geometry editing, streaming a 180 GB dataset without downloading it, PMTiles, 3D tiles, LiDAR, DuckDB, the map controls, and sharing.

Related: [Getting Started](#) · [Interface Overview](#) · [Your First Map](#)

Topics covered

The chapter timestamps in the video description run past the recording's 40:18 length, so they are listed here without times.

- What GeoLibre is, and the three ways to run it (browser, desktop, Jupyter)
- The responsive web layout and light/dark mode
- Installing the desktop app; running GeoLibre in a Jupyter notebook
- Saving and loading projects; the website tour and the tech stack
- 3D tiles and navigating the map canvas; the Layers and Style panels
- Creating a project and choosing a basemap
- Drag-and-drop vector data and the identify tool
- Attribute table: select, edit, and export; editing geometry
- Drag-and-drop raster data (DEM); adding a GeoParquet layer
- Streaming a 180 GB vector dataset without downloading it
- Vector styling: zoom levels, labels, 3D extrusion
- Raster data: COGs, local files, color maps, pixel inspection
- CSV/lat-long points, GPS (GPX) tracks, and XYZ/WFS/WMS services
- PMTiles (Overture buildings) in 2D and 3D; LiDAR point clouds and 3D tiles
- Connecting to DuckDB
- Map controls: scale, terrain, search, color bar, legend, measurement, bookmarks, minimap
- Plugins, sharing projects, tokens, and map-only embed links

Geoprocessing in the Browser: 700+ Free GIS Tools in GeoLibre, Zero Install



► Geoprocessing in the Browser: 700+ Free GIS Tools in GeoLibre, Zero Install

21:52 · June 2026 · [Watch on YouTube](#)

The Whitebox Next Gen toolbox running entirely on WebAssembly — terrain, hydrology, LiDAR, remote sensing, raster, and vector tools, with nothing installed. Walks the category-browsable Processing menu, runs terrain and hydrology tools on a DEM, and covers plugins, large local files, and the JupyterLite panel.

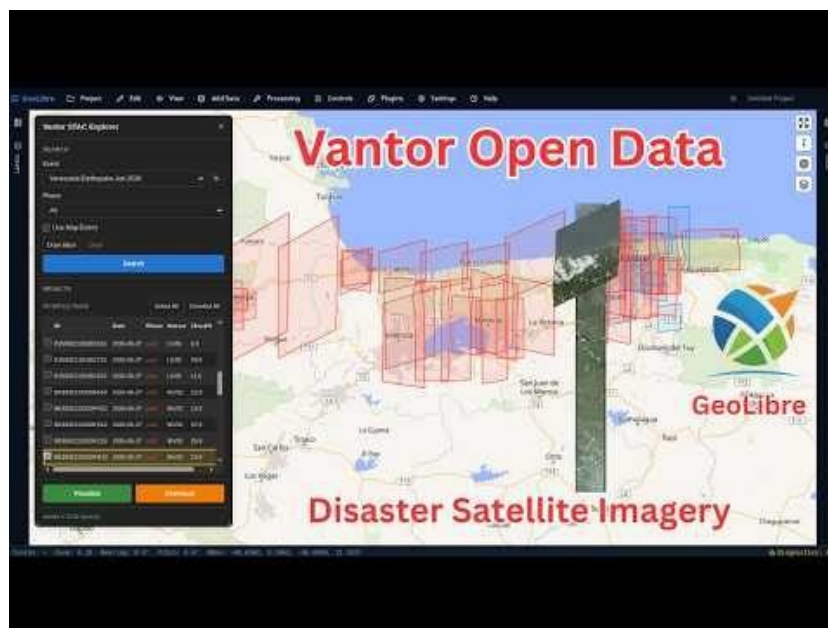
The toolbox has since grown past 1,000 tools; see [Processing Tools](#) for the current list.

Related: [Terrain Analysis](#) · [Vector Analysis](#) · [Notebook Panel](#)

Chapters

- 00:00 Introduction and GeoLibre 1.7 overview
- 03:10 Opening the web app and creating a project
- 04:03 Adding data and styling raster layers
- 06:26 Web-based geoprocessing with Whitebox tools
- 09:18 Running terrain and hydrology analysis
- 12:03 Vector data, labels, and centroid processing
- 14:38 Plugins and extending GeoLibre
- 17:39 Loading large local files
- 20:25 JupyterLite and field data collection
- 21:31 Wrap-up and feedback

Access Free High-Resolution Disaster Satellite Imagery in Your Browser



► [Access Free High-Resolution Disaster Satellite Imagery in Your Browser](#)

8:26 · July 2026 · [Watch on YouTube](#)

Installs the [Vantor Open Data plugin](#), searches its STAC catalog for a disaster event, and compares pre- and post-event high-resolution imagery on the map — then downloads the original Cloud Optimized GeoTIFFs and shares the result as a link.

Related: [Plugins & Marketplace](#) · [Adding Data](#)

Chapters

- 00:00 Introduction
- 00:53 What is GeoLibre and Vantor Open Data?
- 01:27 Installing the Vantor Open Data plugin
- 02:30 Browsing disaster events
- 03:00 Searching pre- and post-event imagery
- 03:55 Visualizing high-resolution satellite imagery
- 04:21 Comparing before and after imagery
- 05:06 Identifying earthquake damage
- 05:54 Downloading original imagery
- 06:18 Loading multiple image scenes
- 07:05 Accessing source URLs and sharing maps
- 07:53 Automatic STAC catalog updates
- 08:20 Wrap-up

Regularize Building Footprints in the Browser with GeoLibre



► [Regularize Building Footprints in the Browser with GeoLibre](#)

16:25 · July 2026 · [Watch on YouTube](#)

Cleans up irregular building polygons — the kind an AI segmentation model or hand digitizing produces — with the Rust-based regularization tool, entirely in the browser. Covers loading the data, projected coordinate systems, tuning the parameters, comparing before and after, and exporting the result.

Try it yourself with the [live sample project](#).

Related: [Processing Tools](#) · [AI Segmentation](#)

Chapters

- 00:00 Introduction
- 00:30 Why building footprint regularization matters
- 01:00 The traditional Whitebox workflow
- 02:20 GeoLibre overview
- 03:05 Launching GeoLibre in the browser
- 04:05 Loading sample building data
- 05:00 Importing your own building footprints
- 06:10 Coordinate systems and data preparation
- 06:45 Opening the building regularization tool
- 07:20 Configuring tool parameters
- 08:05 Running the tool and comparing results
- 09:05 Understanding regularization options
- 10:10 Working with different building shapes
- 11:10 Exploring additional Whitebox tools
- 12:20 Example: extracting urban areas
- 13:20 GeoLibre's Whitebox integration
- 14:30 More geoprocessing tools in GeoLibre
- 15:30 GeoLibre desktop and mobile apps
- 16:10 Downloading GeoLibre
- 16:17 Conclusion

GeoLibre + GeoLens: A Modern GIS Stack for Self-Hosting Geospatial Data



► [GeoLibre + GeoLens: A Modern GIS Stack for Self-Hosting Geospatial Data](#)

25:41 • August 2026 • [Watch on YouTube](#)

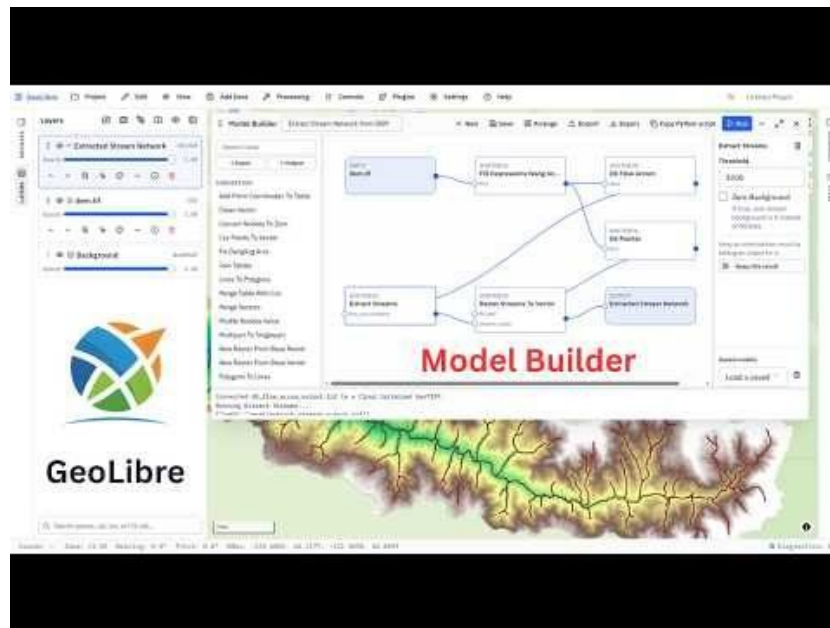
Pairs GeoLibre with [GeoLens](#) for a fully self-hosted stack: deploy GeoLens, browse its catalog from GeoLibre, build a 3D NYC map with extrusions and expression-driven styling, animate building growth over time, then edit footprints and push the edits back to GeoLens.

Related: [Data Integrations](#) • [Styling Layers](#) • [Self-Hosting](#)

Chapters

- 00:00 Introduction: a modern open-source GIS stack
- 01:15 Introduction to GeoLens
- 02:19 Exploring the GeoLens demo and data catalog
- 03:51 Introduction to GeoLibre
- 04:52 Self-hosting and managing data with GeoLens
- 06:31 Connecting GeoLibre to GeoLens
- 08:16 Exploring the NYC 3D sample project
- 08:58 Animating NYC building growth over time
- 10:21 Building a 3D NYC map from scratch
- 11:34 Creating 3D building extrusions
- 12:27 Styling buildings with expressions
- 14:41 Adding and styling NYC subway data
- 16:50 Creating a time animation
- 17:26 Sharing a GeoLibre project
- 17:52 Editing GeoLens data from GeoLibre
- 19:12 Loading editable GeoJSON data
- 20:02 Creating and editing building footprints
- 21:30 Saving changes back to GeoLens
- 22:27 Editing large vector datasets by map extent
- 23:03 Visualizing raster and Sentinel-2 data
- 24:23 GeoLens + GeoLibre as a modern GIS stack
- 24:29 Other GeoLibre data connectors and analysis tools
- 25:14 Conclusion

Create Reusable GIS Workflows with GeoLibre Model Builder and AI Assistant



► Create Reusable GIS Workflows with GeoLibre Model Builder and AI Assistant

18:55 · August 2026 · [Watch on YouTube](#)

Build graphical, reusable geoprocessing workflows in the browser: a simple vector chain first, then a full hydrology model that extracts a stream network from a DEM. The second half hands the same job to the [AI Assistant](#), which assembles and wires the model from a natural-language prompt, and shows how to edit what it produced.

Related: [Processing Tools](#) · [Terrain Analysis](#)

Chapters

- 00:00 Introduction
- 00:49 Model Builder overview
- 02:23 Building a simple workflow
- 04:19 Running the model
- 04:54 Importing and exporting models
- 05:23 Building a hydrology workflow
- 11:08 Running the hydrology model
- 13:01 Building models with AI
- 14:46 Generating a workflow with a prompt
- 17:32 Customizing AI-generated models
- 18:25 Conclusion

Mapping the 2026 Nepal Floods with Free High-Resolution Satellite Imagery



► Mapping the 2026 Nepal Floods with Free High-Resolution Satellite Imagery

23:14 · August 2026 · [Watch on YouTube](#)

A full disaster-response workflow on the Nepal–China border flood: find, view, compare, and download open imagery from Vantor Open Data, Planet Crisis Response, and OpenAerialMap. Builds a before-and-after swipe map, explores the valley in 3D terrain, pulls building footprints from Overture Maps, and exports the result for damage assessment.

Try it yourself with the [live sample project](#).

Related: [Plugins & Marketplace](#) · [Data Integrations](#) · [Map Controls](#)

Chapters

As with the 1.0 tour, the last few chapter timestamps in the video description run past the recording's 23:14 length.

- 00:00 Nepal–China border flood seen from space
- 02:02 Building the before-and-after map
- 03:05 Finding the affected location
- 06:09 Searching Vantor Open Data
- 08:40 Exploring the terrain in 3D
- 12:08 Adding post-event satellite imagery
- 14:53 Comparing damage with Layer Swipe
- 18:39 Sharing the interactive GeoLibre map
- 20:06 Exploring Planet Crisis Response imagery
- 22:52 Finding imagery with OpenAerialMap
- 25:14 Extracting buildings from Overture Maps
- 27:20 Exporting data for damage assessment
- 28:30 Conclusion

Building Cloud-Native GIS Workflows with GeoLibre



► [Building Cloud-Native GIS Workflows with GeoLibre](#)

1:07:28 · September 2026 · [Watch on YouTube](#)

A recorded webinar, and the longest single overview of the project. Covers how GeoLibre uses COG, GeoParquet, PMTiles, DuckDB, and WebAssembly to work with large vector, raster, and LiDAR datasets directly in the browser, then walks the plugin ecosystem, the cross-platform builds (browser, desktop, mobile, Jupyter, R), and the emerging GeoAI integration.

Slides: geolibre-slides-en.html

Related: [Cloud-Native Data](#) · [Architecture](#) · [Python API](#) · [R Package](#)

Image Georeferencing Using GeoLibre in the Browser



► [Image Georeferencing Using GeoLibre in the Browser](#)

15:35 · September 2026 · [Watch on YouTube](#)

Georeferences a scanned University of Tennessee campus map with no desktop GIS: place ground control points, pick a transformation, read the per-GCP and RMS residuals, and align the image to a basemap. Then exports a GeoTIFF, reloads it, handles NoData transparency, compares it against the basemap with the swipe tool, and shares the finished map.

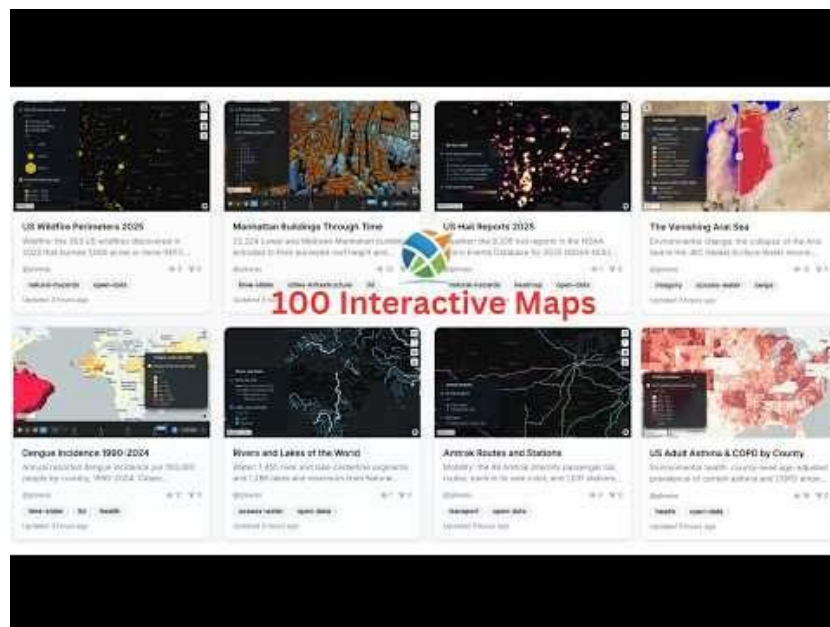
Sample data: [UTK campus map](#) · [control points](#)

Related: [Processing Tools](#) · [Adding Data](#) · [Sharing & Embedding](#)

Chapters

- 00:00 Introduction to image georeferencing
- 01:44 Launching GeoLibre and choosing a basemap
- 02:44 Opening the Georeferencing tool
- 03:20 Loading an image
- 03:50 Choosing a transformation method
- 04:55 Selecting ground control points
- 08:34 Checking coordinates and residuals
- 11:17 Evaluating georeferencing accuracy
- 11:30 Exporting control points and a GeoTIFF
- 12:26 Adding the georeferenced image to the map
- 12:59 Comparing layers with the swipe tool
- 13:35 Loading the exported GeoTIFF
- 14:07 Handling NoData and transparency
- 14:42 Sharing the georeferenced map
- 15:29 Conclusion

100 Interactive Maps from Open Data: Explore, Fork, and Build Your Own with GeoLibre



► [100 Interactive Maps from Open Data: Explore, Fork, and Build Your Own with GeoLibre](#)

24:37 · September 2026 · [Watch on YouTube](#)

Tours the open data gallery: 100 interactive maps built entirely from free public data and open live in the browser. Shows how to explore and fork any of them, and how they were made, so you can build the same kind of map from your own data. The maps span 12 themes, from health and natural hazards to history and culture, with data from NASA, NOAA, USGS, CDC, the World Bank, Our World in Data, Eurostat, Wikidata, and more.

Related: [Gallery](#) · [Official Demos collection](#) · [Python package](#)

More videos

The channel also covers GeoAI, DuckDB, and geospatial Python more broadly:

- [Open Geospatial Solutions on YouTube](#)
- [GeoAI playlist](#)

Have a workflow you would like covered? Open a [discussion](#) and suggest it.

<https://geolibre.app/tutorials/first-map/>

Your First Map

This tutorial takes you from an empty workspace to a styled map with inspectable data, in a few minutes. You can do all of it in the [live viewer](#).

1. Open GeoLibre

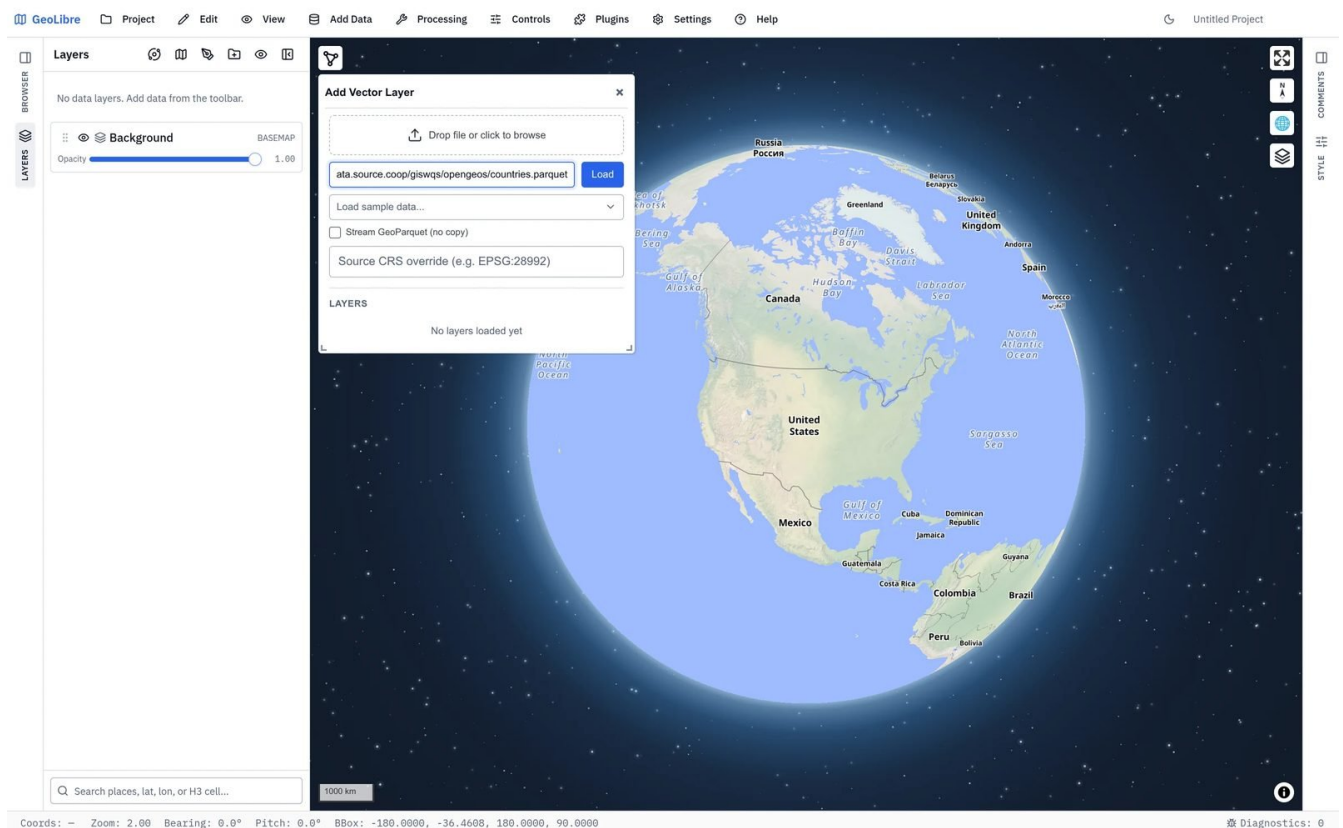
Open web.geolibre.app, or launch the desktop app. You start with a basemap and an empty **Layers** panel.

2. Add a layer

1. Open **Add Data** → **Vector Layer**.
2. In the Add Vector panel, enter a vector URL. You can use the sample countries dataset:

```
https://data.source.coop/giswqs/opengeos/countries.parquet
```

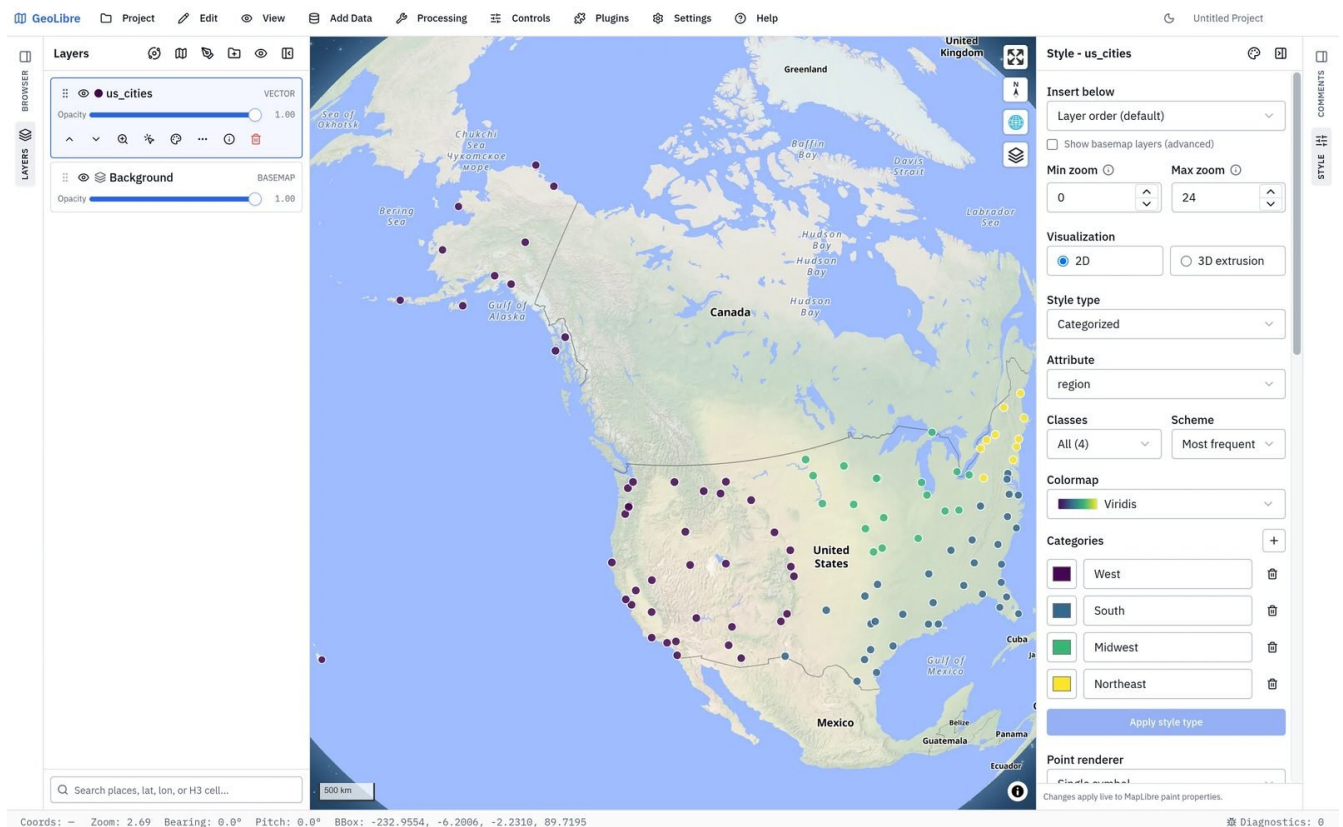
3. Click **Load**. The countries appear on the map and a **countries** layer is added to the Layers panel.



See [Adding Data](#) for every supported source.

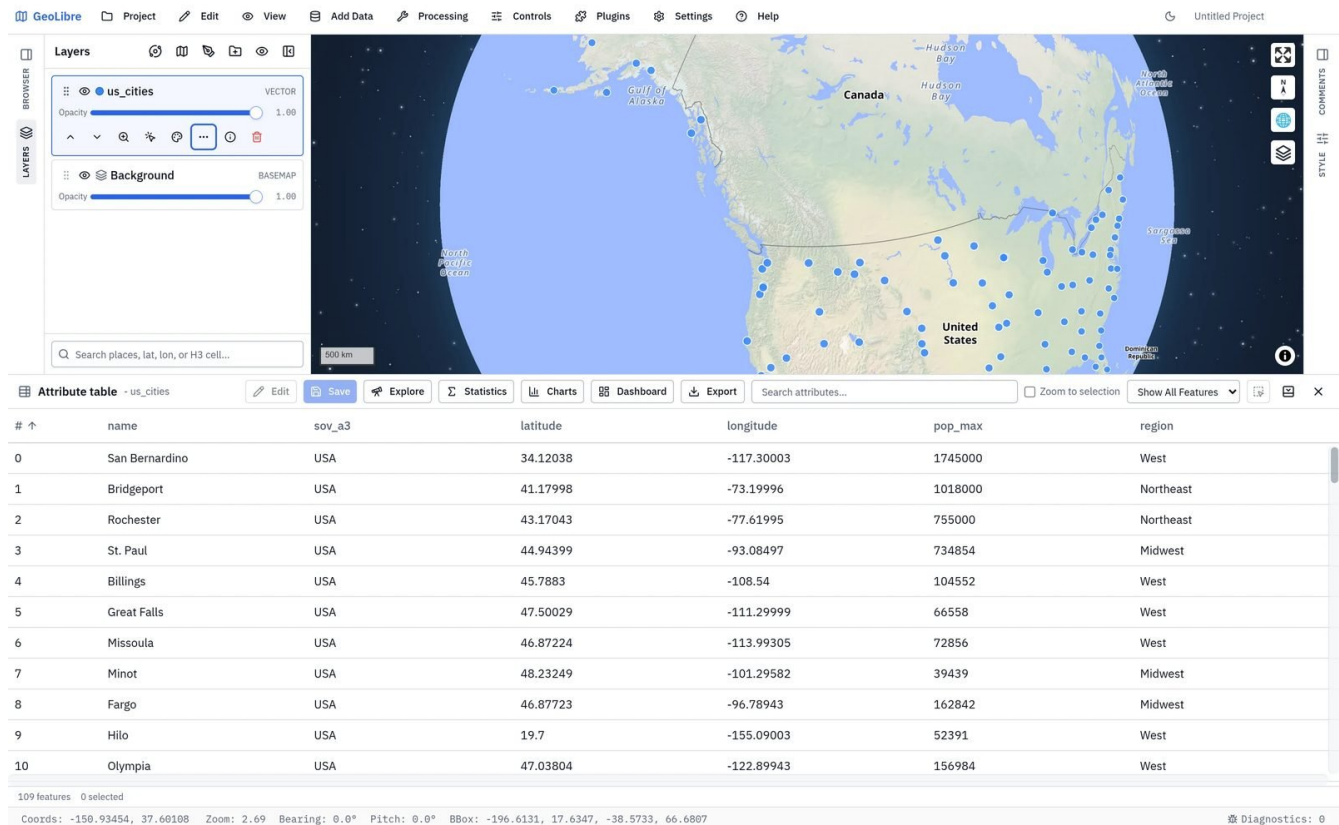
3. Style the layer

1. Select the **countries** layer in the Layers panel, then expand the **Style panel** on the right if it is collapsed.
2. Adjust the **Fill color**, **Outline color**, and **Fill opacity** to taste.
3. To make a choropleth, set **Style type** to **Graduated**, pick a numeric **Attribute** (for example **POP_EST** or **GDP_MD_EST**), choose the number of **Classes**, a **Scheme**, and a **Colormap**, then click **Apply style type**.



4. Inspect the data

1. Click **Attribute table** on the status bar to expand it, then select the **countries** layer to load its records. See [Attribute Table](#).
2. Sort by a column, or type in **Search attributes** to find a feature. Selecting a row highlights it on the map.
3. Click **Explore** to profile every field at once — its type, how complete it is, and how its values are distributed.
4. You can also turn on **Identify features** for the layer and click a country on the map to see its attributes in a popup.



5. Save or share

- Use **Project** → **Save** to write a `.geolibre.json` file. The desktop app saves to disk; the browser build uses the browser's save dialog where it has one and otherwise downloads the file. See [Projects](#).
- Anywhere, use **Project** → **Share** to upload the project and get a public link. See [Sharing & Embedding](#).

Next steps

- Load cloud-native and remote formats in [Cloud-Native Data](#).
- Run geometry tools in [Vector Analysis](#).
- Query your data in [Spatial SQL](#).

<https://geolibre.app/tutorials/cloud-native-data/>

Cloud-Native Data

GeoLibre is built for cloud-native geospatial formats: GeoParquet and FlatGeobuf for vector, Cloud-Optimized GeoTIFF (COG) for raster, and PMTiles for tiles. This tutorial loads them from remote URLs and converts a local dataset.

Load a remote GeoParquet

GeoParquet is a compressed, columnar vector format that reads well over HTTP.

1. Open **Add Data → GeoParquet Layer** (or **Vector Layer**).
2. Enter a GeoParquet URL, for example:

```
https://data.source.coop/giswqs/opengeos/countries.parquet
```

3. To avoid copying a large file into memory, enable **Stream GeoParquet (no copy)**, which queries it in place with HTTP range requests. This works best for large remote files whose rows are spatially sorted (for example, Hilbert order, as written by GeoLibre's own Conversion tools), so only the relevant row groups are fetched.
4. Click **Load**.

Load a FlatGeobuf

FlatGeobuf is a streaming-friendly vector format with a spatial index.

1. Open **Add Data → FlatGeobuf Layer**.
2. Enter a `.fgb` URL and load it. GeoLibre fetches only the features in view where the format allows.

Load a COG

A Cloud-Optimized GeoTIFF is a regular GeoTIFF organized so clients can read just the tiles they need.

1. Open **Add Data → Raster Layer**.
2. Enter the URL of a COG (`.tif`) and load it. You can then adjust brightness, contrast, saturation, and hue in the [Style panel](#).

Drag and drop

You can also drag a local GeoTIFF or COG onto the map to add it as a raster layer. See [Adding Data](#).

Convert local data to cloud-native

Use **Processing** → **GeoLibre Toolbox** → **Conversion** to write cloud-native files. See [Processing Tools](#).

- In the browser build, **Vector to GeoParquet** and **CSV to GeoParquet** run on DuckDB-WASM, and **Vector to FlatGeobuf**, **Vector to PMTiles**, and **Raster to COG** run on WebAssembly (`geolibre-wasm`), so none of them needs a sidecar.
- The desktop app runs these on the Python sidecar instead, which reads more input formats (Raster to COG in the browser reads GeoTIFF only) and preserves data types the browser writers cannot.

For example, to publish a local GeoJSON as GeoParquet:

1. Open **Processing** → **GeoLibre Toolbox** → **Conversion** → **Vector to GeoParquet**.
2. Choose the input file and an output path.
3. Run the conversion, then add the resulting GeoParquet back to the map to verify it.

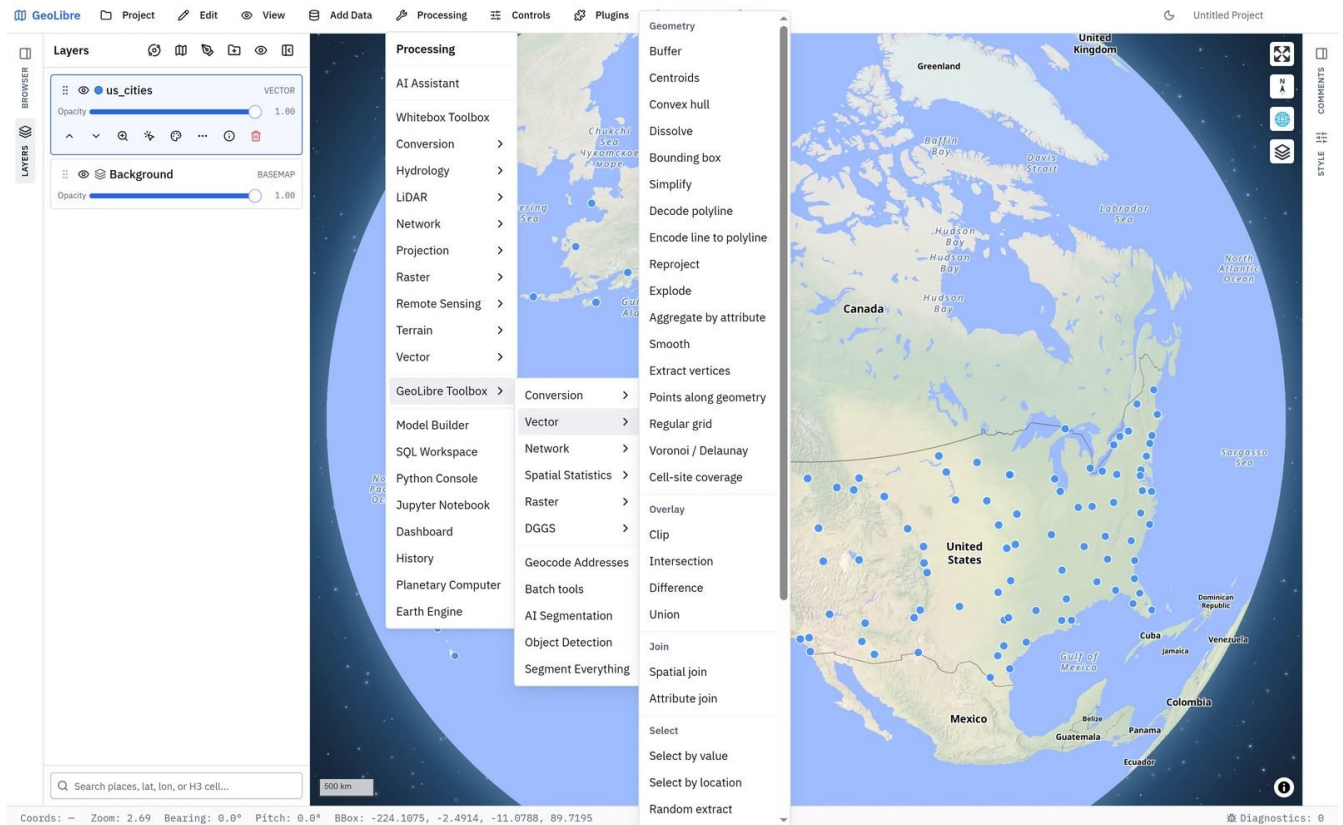
Next steps

- Query these formats directly in [Spatial SQL](#); the SQL Workspace reads Parquet, CSV, JSON, and GeoJSON from URLs.
- Animate a time series of COGs with the Time Slider plugin. See [Data Integrations](#).

<https://geolibre.app/tutorials/vector-analysis/>

Vector Analysis

This tutorial runs a small vector workflow: buffer a layer, overlay it with another, and export the result. It uses the [Vector tools](#) under **Processing** → **GeoLibre Toolbox** → **Vector**.



Two toolboxes share this menu

Processing → **Vector** (above the separator) is the Whitebox catalog's vector category; **Processing** → **GeoLibre Toolbox** → **Vector** is the dialog this tutorial uses. See [Two toolboxes in one menu](#).

1. Load input data

Add at least one vector layer (see [Adding Data](#)). For an overlay you will need two layers, for example a set of points or lines and a polygon layer to clip against.

2. Buffer a layer

1. Open **Processing** → **GeoLibre Toolbox** → **Vector** → **Buffer**.
2. Set **Input layer** to your layer.
3. Set the **Distance** and **Units** (kilometers, meters, or miles).

4. Choose an **Engine**:
5. **Client (Turf.js)** runs in the browser with no setup.
6. **Sidecar (GeoPandas)** runs on the Python sidecar for projection-aware distances (desktop app).
7. **Python (Pyodide)** runs the same GeoPandas code as the sidecar, but in your browser, so the results match the sidecar without one being installed. The runtime downloads once on first use.
8. Click **Run**. A buffered layer is added to the map.

Projection-aware distances

The client engine buffers in geographic coordinates. For accurate metric distances over large areas, use one of the two GeoPandas engines, which reproject before buffering: **Sidecar (GeoPandas)** on the desktop app, or **Python (Pyodide)** anywhere, including the web build.

3. Overlay two layers

With the buffer (or any polygon layer) and a second layer, run an overlay:

- **Clip** keeps the part of the input that falls inside the overlay, preserving the input's attributes.
- **Intersection** keeps only the overlapping areas of two polygon layers.
- **Difference** removes the overlay's area from the input.
- **Union** merges two polygon layers into one (attributes are not preserved, on either engine).
- **Spatial join** attaches a join layer's attributes to each input feature based on a spatial relationship (intersects, within, or contains) — for example, tagging each point with the polygon that contains it. Works with any geometry type.
- **Attribute join** attaches a join table's attributes to each input feature where a key field matches — no geometry involved (for example, joining census statistics to boundary polygons by a shared FIPS code). It is one-to-one (the first matching join row wins); pick the key field on each side, optionally list which fields to bring over, and choose an inner or left join.

Open the tool from **Processing** → **GeoLibre Toolbox** → **Vector**, pick the input and overlay layers, and **Run**.

4. Inspect and refine

Open the [Attribute table](#) on the result layer to check the output, and adjust its [style](#) so it stands out from the inputs. Every run is also recorded in **Processing** → **History**, which re-runs an entry with one click and copies the equivalent Python code.

5. Export the result

To save the output as a cloud-native file, use **Processing** → **GeoLibre Toolbox** → **Conversion** (for example **Vector to GeoParquet** or **Vector to FlatGeobuf**). See [Cloud-Native Data](#). You can also export records from the [Attribute table](#) or the [SQL Workspace](#).

Next steps

- Do the same kind of analysis in SQL with [Spatial SQL](#).
- Move to raster analysis in [Terrain Analysis](#).

<https://geolibre.app/tutorials/terrain-analysis/>

Terrain Analysis

This tutorial derives terrain products from a digital elevation model (DEM): a hillshade, a slope map, and contour lines. It uses the [Raster tools](#) under **Processing → GeoLibre Toolbox → Raster**.

Which engine runs these

Hillshade, **Slope**, and **Aspect** each offer a **Client (browser)** engine, so this tutorial's first three steps work in the browser build with nothing installed. **Contour** and **Clip by mask layer** have no client engine and need the rasterio Python sidecar, which the desktop app manages — see [Getting Started](#). The engine picker at the top of each tool shows what is available; prefer the sidecar for large rasters or for terrain in a geographic CRS.

1. Load a DEM

Add an elevation raster as a layer, for example a GeoTIFF or COG DEM (see [Adding Data](#)). The raster tools read a GeoTIFF file, so a local or accessible raster works best. Tools with a browser engine (Hillshade, Slope, Aspect, Clip by extent) can also use **Use a layer on the map** to pick a raster layer you have already added.

2. Hillshade

1. Open **Processing → GeoLibre Toolbox → Raster → Hillshade**.
2. Choose the DEM as input and set the azimuth, altitude, and z-factor if you want to adjust the lighting.
3. Run it. The shaded-relief raster is added to the map. Place it under your other layers and set the top one's [blend mode](#) to **Multiply**, which lets the relief show through the colour instead of washing both out the way lowered opacity would.

3. Slope and aspect

- **Processing → GeoLibre Toolbox → Raster → Slope** computes steepness from the DEM.
- **Processing → GeoLibre Toolbox → Raster → Aspect** computes the compass direction of the steepest slope.

Run either against the DEM and style the output with a [colormap](#). Open the **Colorbar** from the [Controls menu](#) to show the value scale.

4. Contours

1. Open **Processing** → **GeoLibre Toolbox** → **Raster** → **Contour**.
2. Choose the DEM and set the contour **interval** (the elevation difference between lines).
3. Run it to generate contour lines as a vector layer, which you can label and style like any vector data.

Contour runs on the sidecar only, so this step needs the desktop app.

5. Clip to an area of interest

To restrict outputs to a study area, use **Processing** → **GeoLibre Toolbox** → **Raster** → **Clip by extent** (a bounding box, which also runs in the browser) or **Clip by mask layer** (a vector mask, sidecar only). See [Processing Tools](#).

Next steps

- Convert raster outputs to vectors with **Polygonize**, or write a **Raster to COG** for sharing. See [Cloud-Native Data](#).
- Animate a time series of rasters with the Time Slider plugin. See [Data Integrations](#).

<https://geolibre.app/tutorials/spatial-sql/>

Spatial SQL

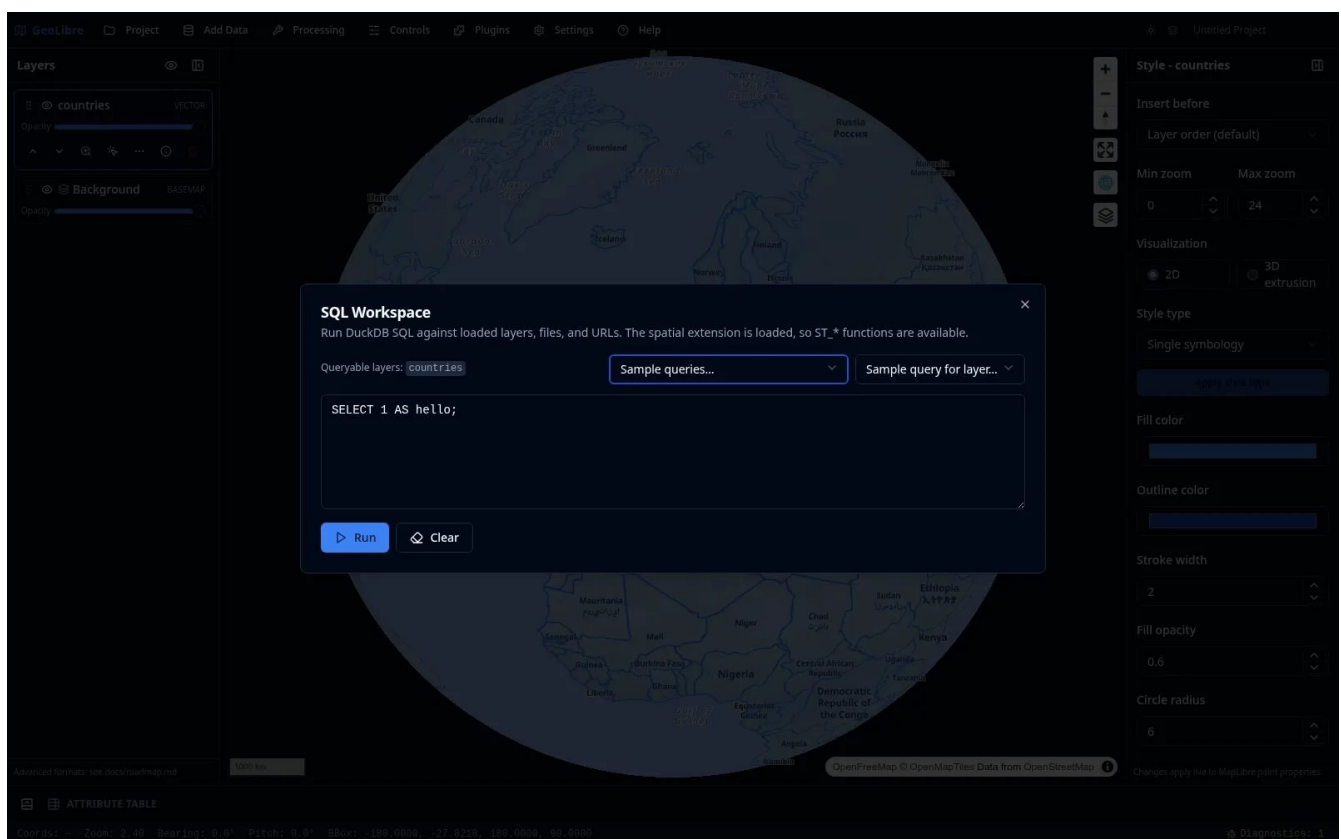
The [SQL Workspace](#) lets you analyze data with DuckDB Spatial SQL and add the results to the map. It runs on DuckDB-WASM, so it works in the browser as well as the desktop app. Open it from **Processing → SQL Workspace**.

1. Query a loaded layer

Every loaded vector layer is a queryable table. Load the sample countries layer (see [Your First Map](#)), then run:

```
SELECT NAME, CONTINENT, GDP_MD_EST
FROM countries
ORDER BY GDP_MD_EST DESC
LIMIT 10;
```

Click **Run** to see the ten countries with the highest estimated GDP.



Sample dataset columns

`NAME`, `CONTINENT`, `POP_EST`, and `GDP_MD_EST` are Natural Earth field names in the sample `countries.parquet`, whose geometry column is `geom`. A different dataset will have its own column names. To discover them, run `DESCRIBE SELECT * FROM 'your-file-url'` first.

2. Query a remote file directly

You do not have to load a file first. The Workspace detects a bare URL in a `FROM` clause and wraps it in the right reader automatically, then streams the file over HTTP range requests so you do not download it in full. This bare-URL shorthand is a SQL Workspace convenience; in the standard DuckDB CLI you would write `read_parquet('https://...')` explicitly.

```
SELECT COUNT(*) AS n
FROM https://data.source.coop/giswqs/opengeos/countries.parquet;
```

3. Use spatial functions

The spatial extension is loaded, so `ST_*` functions are available. For example, compute area and keep the `geom` column so the result can be mapped:

```
-- area is in square degrees because the data is in EPSG:4326;
-- the ranking is valid, but for km2 use
-- ST_Area_Spheroid(ST_FlipCoordinates(geom)) / 1e6 (it expects lat/lon order).
SELECT NAME, ST_Area(geom) AS area, geom
FROM https://data.source.coop/giswqs/opengeos/countries.parquet
WHERE CONTINENT = 'Africa'
ORDER BY area DESC;
```

4. Add the result to the map

When a query returns a geometry column, click **Add as layer** to create a new layer from the result, optionally naming it. The result layer supports [identify](#), [selection](#), and [the attribute table](#) like any vector layer, and you can add several result layers at once.

5. Export

Export the query result as **CSV** or **GeoParquet** straight from the workspace — the **Export CSV** and **Export GeoParquet** buttons sit beside **Add as layer**. See [SQL Workspace](#).

Sample queries and history

Use the **Sample queries...** menu to start from a working query, and **History...** to rerun a previous one.

Next steps

- Do equivalent geometry operations with the menu-driven [Vector Analysis](#) tools.
- Convert results to cloud-native formats in [Cloud-Native Data](#).

<https://geolibre.app/tutorials/sharing-embedding/>

Sharing & Embedding

Once you have a map you like, you can publish it as a public link and embed it in any web page. This tutorial covers both. See [Embedding & Sharing](#) for the full reference.

1. Connect your account

Sharing uploads to `share.geolibre.app`.

- **Web app:** open **Project → Share...** and click **Sign in to share.geolibre.app**. Approve the access request in the popup that opens — you are connected for the browser session.
- **GeoLibre Desktop:** click the same **Sign in to share.geolibre.app** button. The sign-in opens in your system browser, and the desktop app keeps the session in your OS credential store.
- **Fallback (any build):** paste a personal API token into **Settings → Environment Variables → Share.GeoLibre API token**. Create one under **Settings → API tokens** at share.geolibre.app/settings.

2. Share the project

1. Build your map: add layers, style them, and set the map view you want viewers to land on.
2. Open **Project → Share...**
3. Confirm the project title and upload. GeoLibre returns a public URL to a `.geolibre.json` file, for example:

```
https://share.geolibre.app/you/my-map.geolibre.json
```

The shared file captures the same layers, styles, plugin state, and map view as a local save. Layers that read files on your computer are the exception: the dialog lists them as missing before you share, since `share.geolibre.app` stores the project file and never your data files. See [Sharing local data](#) for how to host them instead.

3. Open the shared map

Anyone can open the shared project in the live viewer by passing it as the `url` parameter:

```
https://web.geolibre.app/?url=https://share.geolibre.app/you/my-map.geolibre.json
```

4. Embed it in a page

Use an `<iframe>` and the embed parameters to control the chrome. For a clean, map-only embed:

```
<iframe
  src="https://web.geolibre.app/?url=https://share.geolibre.app/you/my-
map.geolibre.json&maponly"
  title="Geolibre map"
  width="100%"
  height="600"
  style="border: 0;"
  loading="lazy"
  allow="fullscreen; geolocation"
></iframe>
```

Adjust the look with parameters (they combine):

- `maponly` hides all chrome, leaving only the map.
- `layout=viewer` gives a read-only map: Layers, View, Controls, basemaps, and search/identify stay, while everything that edits the project is hidden.
- `layout=compact` keeps a slim, icon-only toolbar.
- `toolbar=none` hides the top toolbar while keeping panels and the status bar.
- `panels=none` hides the side and bottom panels but keeps the toolbar.
- `theme=dark` forces the dark theme on load.

Reach for `layout=viewer` when readers should be able to toggle layers and explore the data but not change it, and `maponly` when the map is a figure in your page rather than something to interact with.

See the full [parameter table](#).

5. Drive the map from your page

URL parameters configure the embed once, at load. To keep talking to it — fly to the record someone just clicked in your own UI, and hear what they do inside the map — install the typed client:

```
npm install @geolibre/embed
```

```
import { connect } from "@geolibre/embed";

const map = await connect(document.querySelector("iframe"), {
  origin: "https://web.geolibre.app",
});

map.on("selectionChanged", ({ featureIds }) => showRecordFor(featureIds[0]));

async function focusField(field) {
  await map.setView({ bbox: field.bbox });
  await map.highlightFeature({ layerId: "fields", filter: { parcel_id: field.id }, fit: true });
}
```

This works only against a deployment that has allowlisted your page's origin — `web.geolibre.app` has not, so the runtime API is for your own hosted build. See [Talking to the map at runtime](#) for the allowlist, the full command list, and the raw `postMessage` protocol if you would rather not add a dependency.

Next steps

- Tune which controls appear before sharing with the [Controls menu](#).
- Revisit [Your First Map](#) to build the map you want to share.

<https://geolibre.app/tutorials/supported-formats-en/>

GeoLibre Supported GIS Data Formats — Complete Reference

Abstract: This article provides a comprehensive inventory of all GIS data formats natively supported by GeoLibre, covering vector, raster, and cloud-native geospatial formats, along with the internal parsing logic for each format. It serves as a complete technical reference for WebGIS developers.

Original source: "GIS开发手记", original link: <https://mp.weixin.qq.com/s/GrtQzetCBgKtYruEhjsOAw>

Preface

This article systematically catalogs every geospatial data format built into GeoLibre, distinguishing between vector, raster, and cloud-native geospatial formats, while documenting each format's corresponding parsing dependency, read priority, and applicable scenarios — a ready reference for WebGIS developers.

1. Runtime Configurations & Platform Overview

Before listing formats, it's important to clarify "where it runs," since many formats have platform-specific differences.

Runtime	How	Notes
Browser	Open <code>web.geolibre.app</code>	No installation needed; works offline after initial load
Desktop	Tauri v2 native application	Windows / macOS / Linux; available via Microsoft Store, Mac App Store, Homebrew, winget, AUR, Flatpak
Android	Google Play native app	~40 MB per ABI
iOS	App Store native app	iPhone and iPad, same codebase via Tauri v2 mobile
Jupyter	<code>pip install geolibre</code>	The entire application embedded in a notebook cell

No accounts, no servers, no fees for the core application. Local files are read in place and stay on your machine, and once the app has loaded, local workflows keep working offline. The optional remote pieces are the exception: online catalogs (STAC, Source Cooperative, Overture, Planetary Computer), basemap and tile downloads from a CDN, and services such as Earth Engine or an authenticated ArcGIS endpoint all need network access, and some need their own credentials or OAuth sign-in.

2. Vector Data Format Support

Vector data is GeoLibre's most mature capability. The file picker has a built-in unified format allowlist, supports drag-and-drop auto-detection (no manual format selection needed), and covers 17 mainstream vector extensions.

```
geojson, json, gpkg, geoparquet, parquet, fgb, flatgeobuf,
csv, tsv, kml, kmz, gml, gpx, dxf, tab, shp, zip
```

Just drag and drop — no need to first select "which format to import."

What's truly interesting is **which engine reads each format behind the scenes** — this determines actual performance:

Format	Extension	Read Engine	Notable Details
GeoJSON	<code>.geojson</code> <code>.json</code>	Native <code>JSON.parse</code>	Files with legacy top-level <code>crs</code> members are automatically reprojected to WGS84 via DuckDB
GeoParquet / Parquet	<code>.parquet</code> <code>.geoparquet</code>	DuckDB <code>read_parquet</code>	Remote files use HTTP Range streaming reads; no full download needed
FlatGeobuf	<code>.fgb</code> <code>.flatgeobuf</code>	DuckDB <code>ST_Read</code>	—
GeoPackage	<code>.gpkg</code>	sql.js (SQLite WASM), not GDAL	Multi-layer files prompt a layer selector; auto-repairs <code>gpkg_ogr_contents</code>
Shapefile (individual files)	<code>.shp</code>	shpjs	Desktop auto-reads companion <code>.dbf</code> / <code>.shx</code> / <code>.prj</code> / <code>.cpg</code> ; 3D MultiPatch falls back to DuckDB
Shapefile (zip archive)	<code>.zip</code>	fflate decompress → shpjs	<code>.prj</code> determines projection, <code>.cpg</code> determines DBF encoding (Chinese attribute encoding issues resolved); auto-skips macOS <code>__MACOSX</code>
KML	<code>.kml</code>	Custom parser	Preserves embedded styling and Folder structure; placemarks with <code><TimeSpan></code> / <code><TimeStamp></code> animate on the Time Slider; also extracts GroundOverlay images and <code><Model></code> 3D models

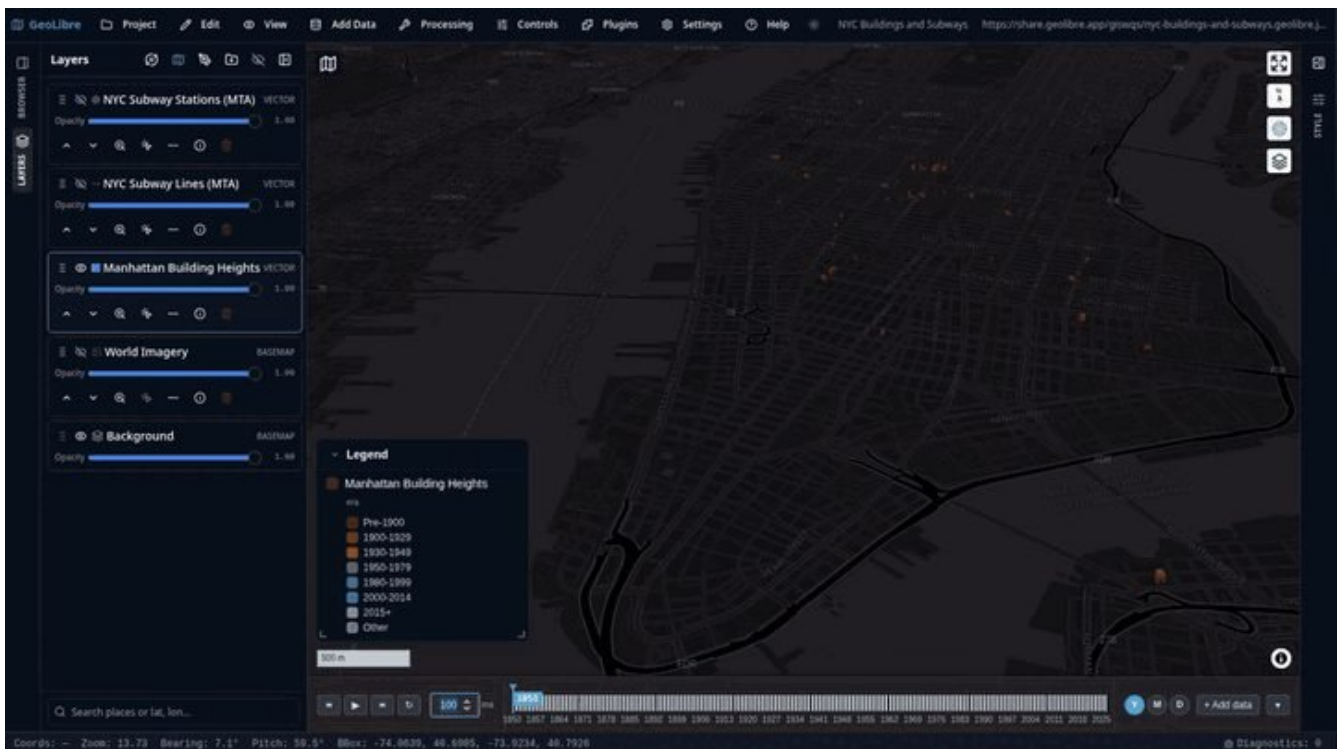
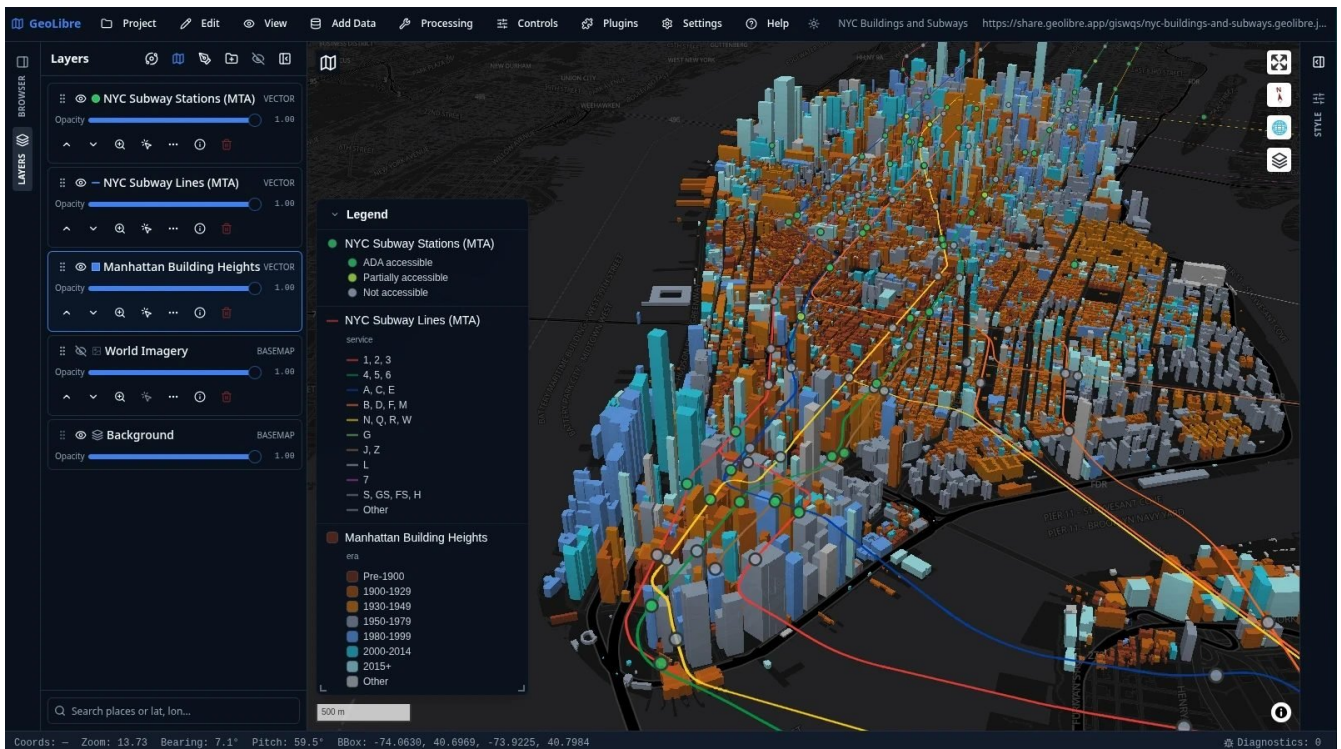
Format	Extension	Read Engine	Notable Details
KMZ	<code>.kmz</code>	fflate decompress	Custom icons and formatted descriptions are preserved
GML	<code>.gml</code>	DuckDB <code>ST_Read</code>	—
GPX	<code>.gpx</code>	Pure JS	Auto-splits into three layers: waypoints / tracks / routes
CSV / TSV	<code>.csv</code> <code>.tsv</code> <code>.txt</code>	Custom + DuckDB fallback	Auto-detects delimiter and lat/lon columns; WKT geometry columns go through DuckDB; dialog allows specifying source CRS
Excel	<code>.xlsx</code> <code>.xls</code>	SheetJS (<code>@e965/xlsx</code>)	Opened from the Delimited Text panel: pick a worksheet, then the coordinate columns
CAD (DXF/DWG)	<code>.dxf</code> <code>.dwg</code>	DuckDB <code>ST_Read</code>	Presents a layer list for selection; CAD files lack coordinate systems — EPSG must be selected manually
MapInfo TAB	<code>.tab</code>	<code>ST_Read</code>	—
Esri File Geodatabase	<code>.gdb</code> folder	Python sidecar	Desktop-only, requires sidecar; hidden in Mac App Store builds
OSM PBF	<code>.osm.pbf</code> <code>.pbf</code>	osmix, runs in Web Worker	Auto-splits into point/line/polygon layers; prompts confirmation above 50 MB; 5-minute timeout protection
GeoRSS	<code>.xml</code> <code>.rss</code> <code>.atom</code>	Pure JS	Supports RSS 2.0 / Atom / RDF, GeoRSS Simple + GML geometries
Geotagged photos	<code>.jpg</code> <code>.jpeg</code> <code>.png</code>	<code>exif</code> <code>.tiff</code> <code>.webp</code>	Reads EXIF GPS data to directly generate a point layer — very practical for UAV/drone photos

Explicitly unsupported: raw `.osm` XML (PBF only).

KML <Model> 3D Models

The KML <Model> handling is worth noting: it loads embedded COLLADA `.dae` files with three.js, then exports them as GLB into the map. The project pulled in three.js just for this one edge case.

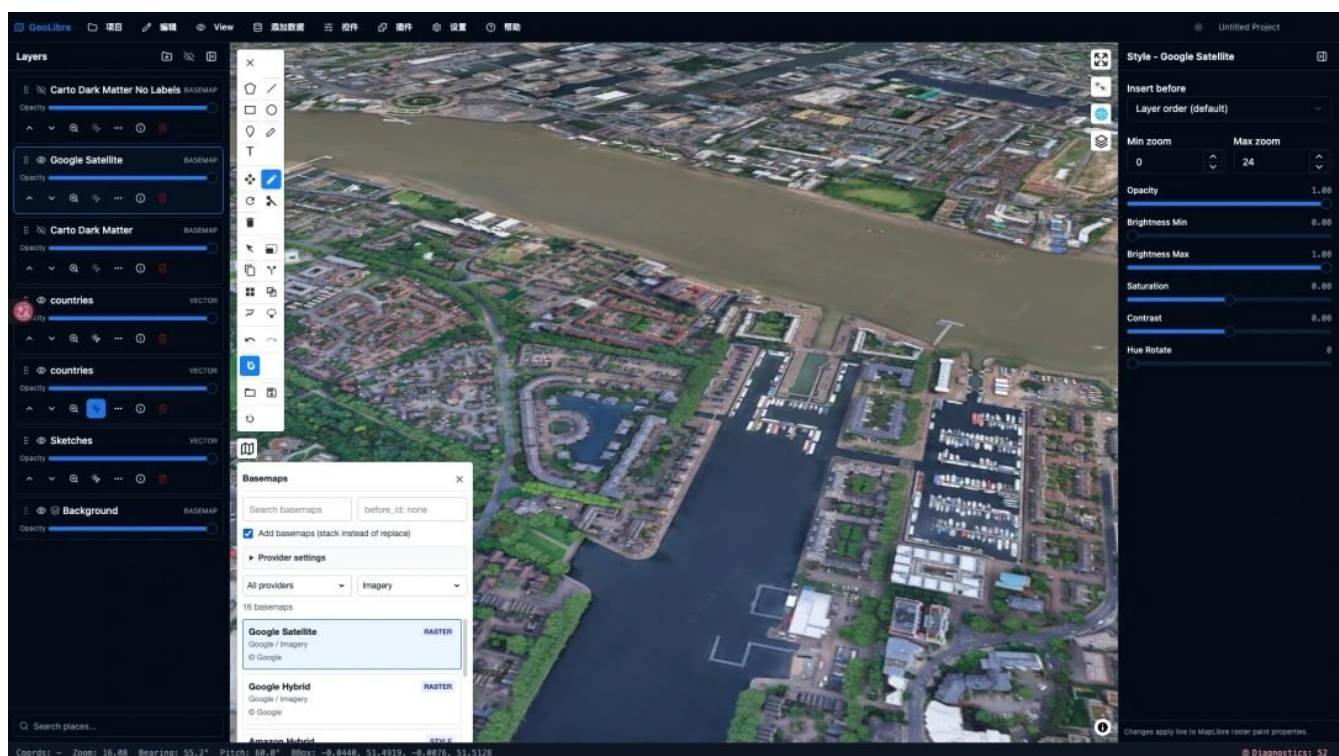
After loading, it's not just "can you see it" — symbology, graduated coloring, and legends all come along:



3. Raster & Imagery

The raster side is narrower than vector but covers the cloud-native mainline.

Format	Extension	Engine	Platform
GeoTIFF / COG	.tif .tiff	Default cog-tiler-wasm; switchable to GPU engine	All platforms; desktop local files use Tauri asset protocol for Range reads
MosaicJSON / STAC item	.json	On-demand stitching at read time	All platforms
Georeferenced images	Any browser-decodable image + GCP .csv / .txt	Least-squares affine; export via gdal3.js	All platforms (gdal3.js loaded from CDN)
KML GroundOverlay	From .kml / .kmz	Four-corner coordinate image layer	Those with <TimeSpan> also support timeline animation
Georeferenced video	.mp4 + .webm	MapLibre video source	URL only
Conversion tool inputs (raster)	.tif .tiff .img .vrt .asc .nc .jp2 .hgt	GDAL / rasterio sidecar	Desktop; browser only accepts .tif/.tiff
Whitebox raster I/O	.tif .tiff .img .bil .flt .sdatt .rdc .asc	whitebox-wasm	All platforms



Note the asymmetry: `.img`, `.vrt`, `.asc`, `.jp2`, `.hgt` are only recognized in conversion tools — they cannot be directly dragged in as layers. Drag-and-drop only works for GeoTIFF.

4. Point Clouds & LiDAR

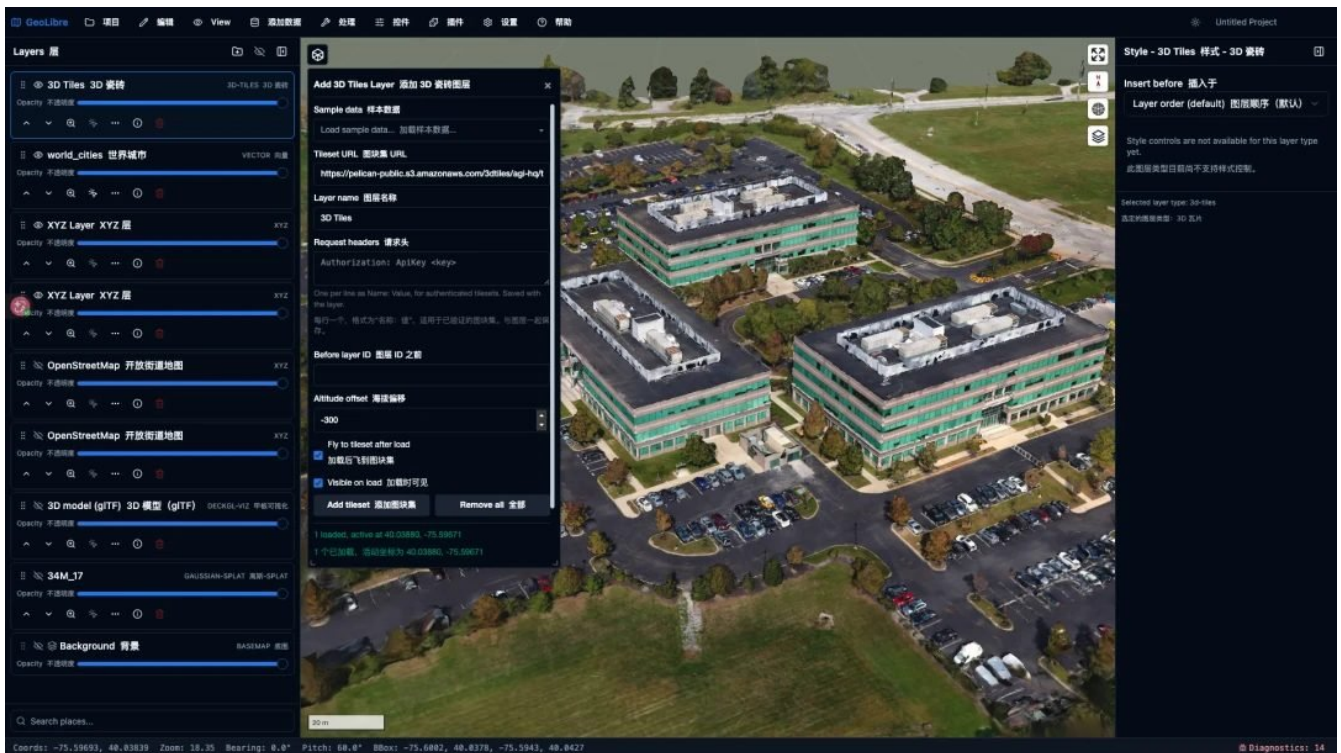
Item	Support	Notes
LiDAR layer	LAS / LAZ / COPC (URL or local file), EPT (<code>ept.json</code> URL)	Rendered through <code>maplibre-gl-lidar</code> + <code>deck.gl</code>
USGS 3DEP	Online point cloud streaming	Standalone plugin built on <code>maplibre-gl-usgs-lidar</code> ; comes with a 3DEP elevation index WMS overlay
Whitebox LiDAR tools	<code>.las</code> <code>.laz</code> <code>.zlidar</code> <code>.copc</code> <code>.e57</code> <code>.ply</code> y, output <code>.laz</code>	The only place in the entire repo where <code>.e57</code> / <code>.ply</code> appear

The LiDAR file picker itself lives in the upstream `maplibre-gl-lidar` package; GeoLibre's wrapper (`packages/plugins/src/plugins/components/lidar.ts`) documents LAS/LAZ/COPC files and EPT `ept.json` URLs, and a dropped local COPC streams by octree node without being copied.

5. 3D Models, Tiles & Gaussian Splats

The opening scenario — "received some 3D Tiles, just want to take a quick look" — this section is the answer.

Format	Input Method	Engine	Notes
OGC 3D Tiles	tileset URL	<code>maplibre-gl-3d-tiles</code> + <code>deck.gl Tile3DLayer</code>	Supports custom request headers — authenticated tilesets work
Google Photorealistic 3D Tiles	Built-in URL	Same as above	Requires Google Maps API key; passed via request headers, never stored to disk
ArcGIS I3S Scene Layer	<code>.../SceneServer</code> URL	<code>deck.gl</code> + <code>loaders.gl</code> <code>I3SLoader</code>	Both integrated mesh and 3D object layers are supported
glTF / GLB	URL or local file	<code>deck.gl ScenegraphLayer</code>	Added through the <code>deck.gl</code> visualization layer's scenegraph type
COLLADA <code>.dae</code>	Only via KML <Model> embedding	<code>three.js</code> → GLB	—
Gaussian Splats	URL	<code>maplibre-gl-splat</code>	Storage layer type is <code>gaussian-splat</code>



This screenshot is quite telling: **in the layer panel on the left, 3D Tiles, vectors, XYZ, glTF models, and Gaussian splats are all stacked in the same list** — and the rendering result is right there on the right. "Loading 3D Tiles requires setting up a server and writing a page" becomes **paste a URL** here.

What's absent: .obj cannot be imported (it appears only as an output of the extruded-layer 3D model export, alongside GLB and STL). .b3dm / .pnts / .cmpt — these 3D Tiles internal formats also have zero hits in the repo; they're transparently handled by upstream loaders, so you don't need to worry about them. The Gaussian splat extension list is likewise in upstream packages.

6. Tiles & OGC Services

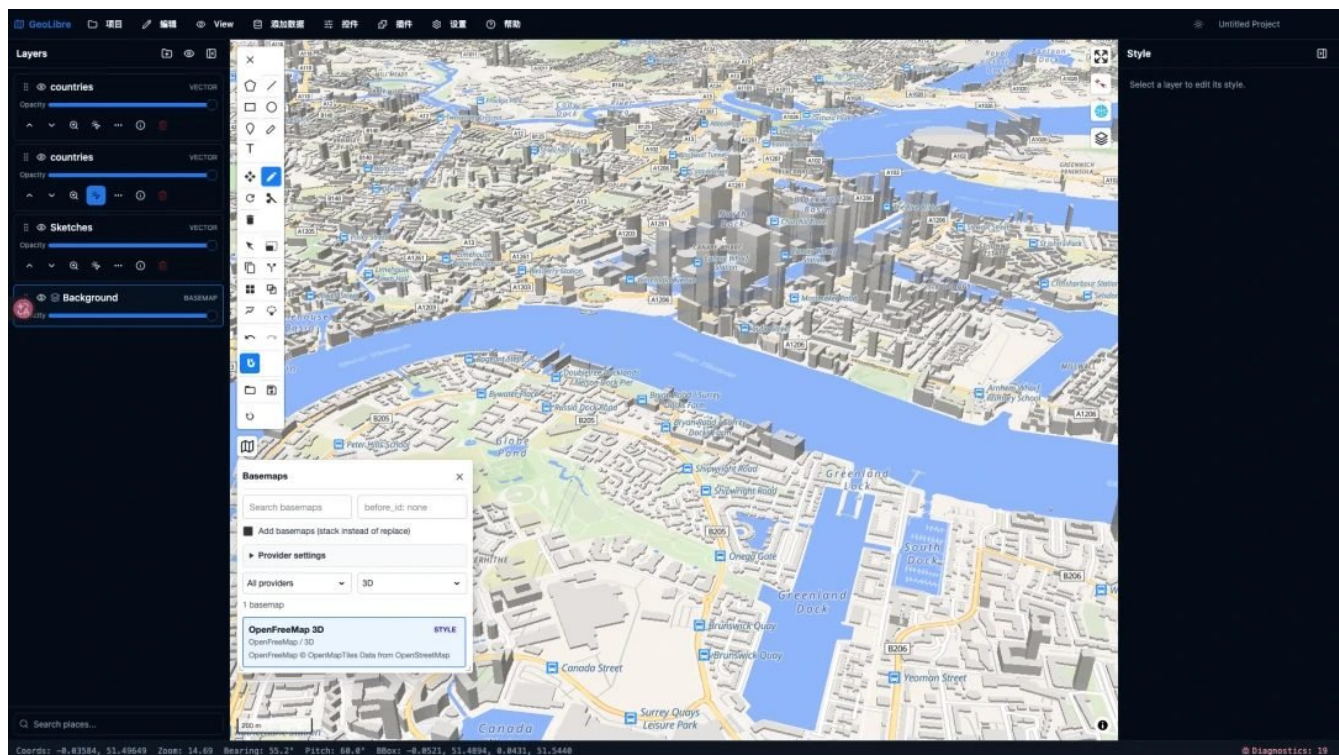
The storage layer type enum has 20 entries:

```
geojson, raster, wms, wmts, xyz, vector-tiles, arcgis, pmtiles,
mbtiles, zarr, lidar, gaussian-splat, 3d-tiles, cog, flatgeobuf,
geoparquet, duckdb-query, deckgl-viz, video, image
```

What you can fill in under "Add Data → Web Services":

Service	Details
XYZ	{z}/{x}/{y} template; raster or vector tiles both work
WMS	GetCapabilities auto-populates the layer dropdown ; click features for GetFeatureInfo; dev mode includes a CORS proxy
WMTS	RESTful tile template

Service	Details
WFS	GetCapabilities fetches typeName; optional auto-refresh
OGC API - Features	Landing page / /collections / single collection / full /items URL are all recognized; auto-follows next links; defaults to 1,000 features
OGC API - Tiles (Vector)	TileJSON or MVT template; can optionally provide a Mapbox style URL to resolve source-layer names
ArcGIS	Four layer types: FeatureServer (downloaded page by page as GeoJSON), VectorTileServer, MapServer, and ImageServer (the last two as raster layers); supports portal item IDs and ArcGIS sign-in
MBTiles	.mbtiles local file, custom protocol + Rust backend reads. Desktop-only
PMTiles	.pmtiles ; vector or raster both work; auto-sniffs file header
PostgreSQL / PostGIS	Connect → select table → outputs MVT via built-in Martin service. Desktop-only
deck.gl visualization layers	14 types: scatterplot, heatmap, hexagon, grid, screen grid, contour, arc, line, great circle, GeoJSON, icon, text, trips, scenegraph



7. Cloud-Native Formats & Data Catalogs

This section is where GeoLibre pulls ahead of traditional desktop GIS.

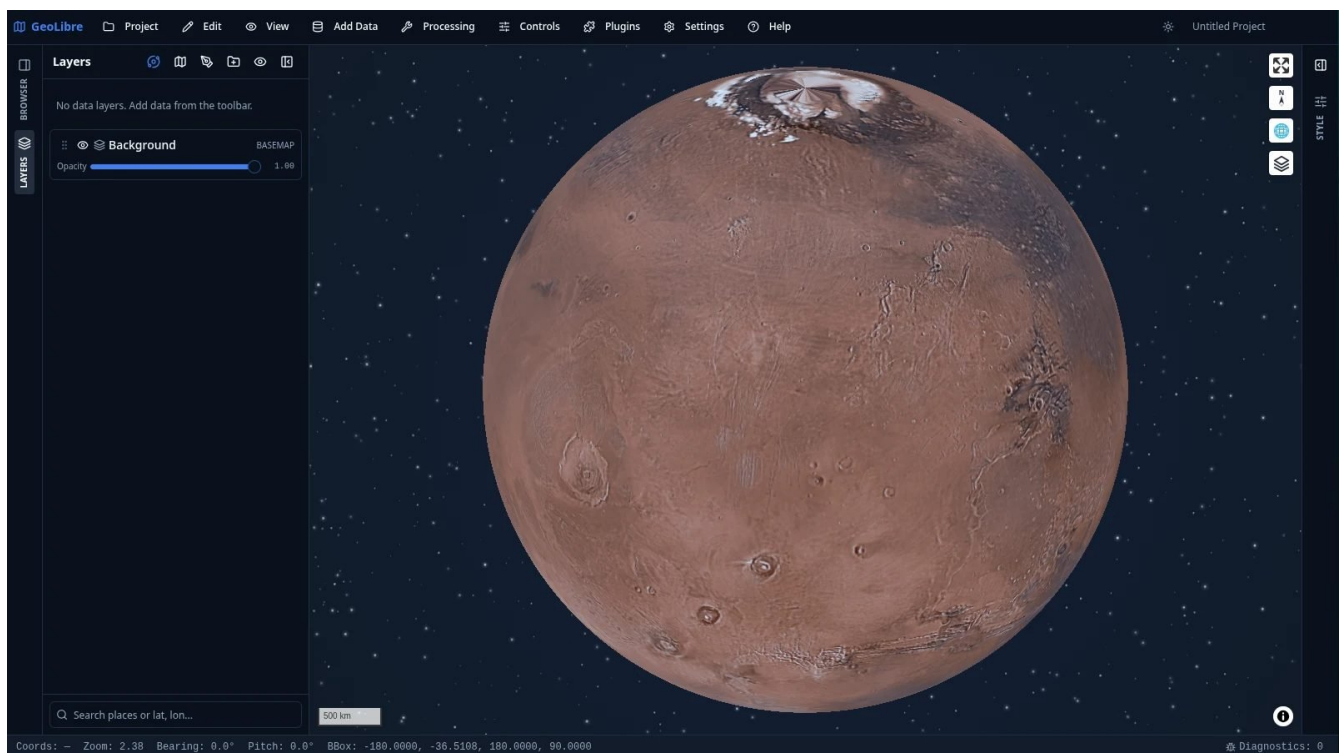
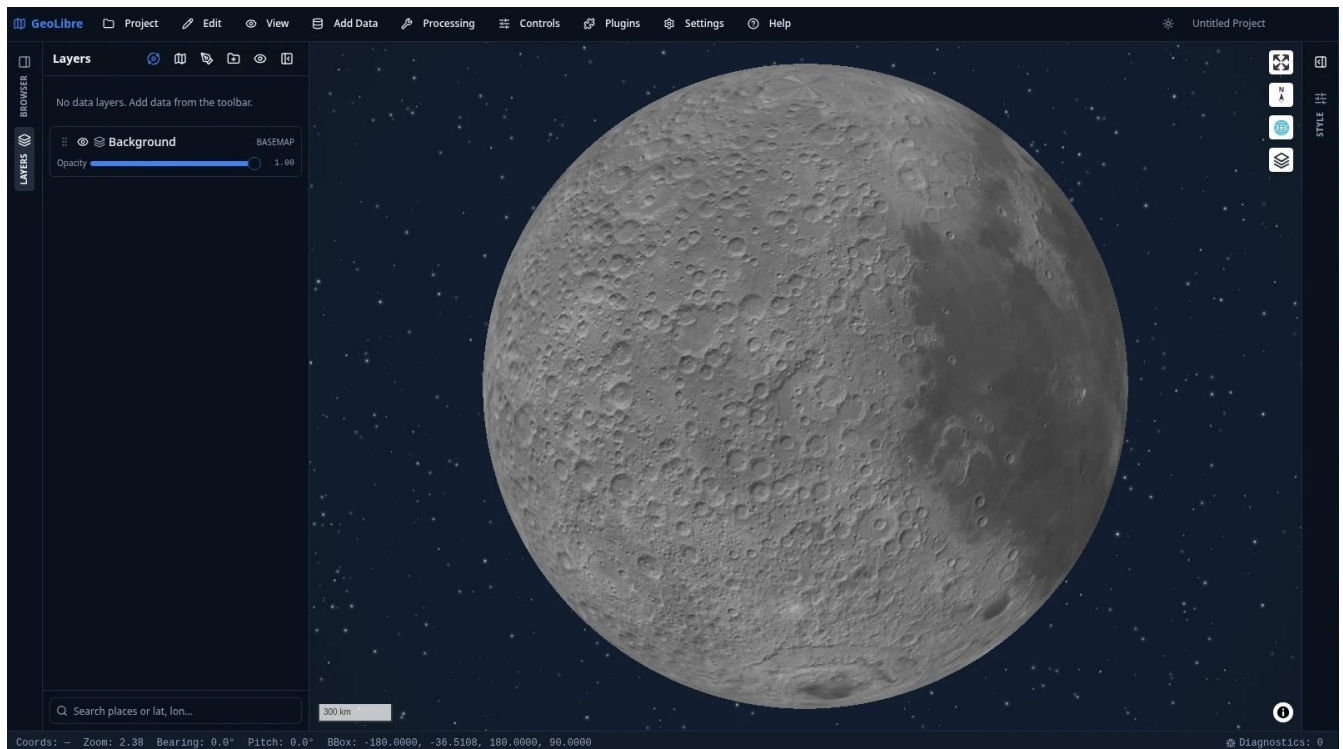
Scientific data formats:

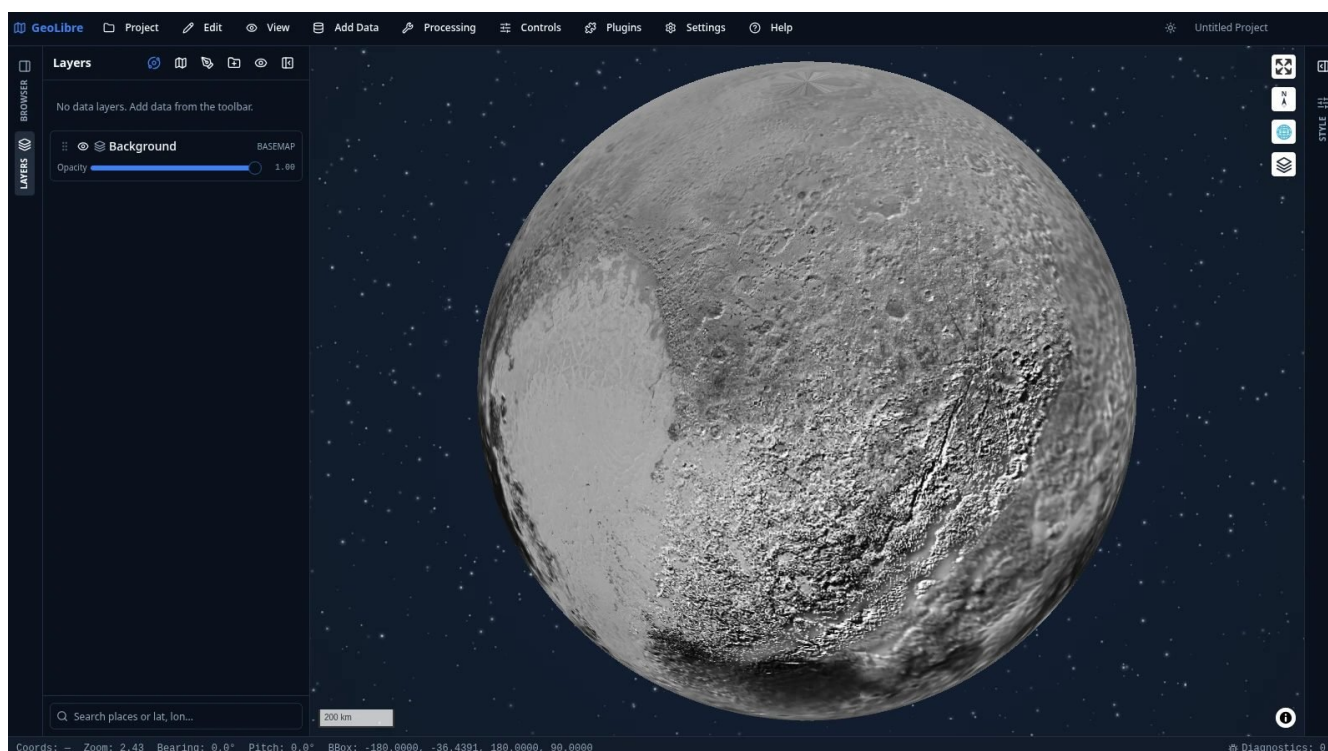
Format	Support Scope
Zarr	Remote store URL, or local folder. Variable and dimension selectors display real coordinate values. <i>Local folders in browser require File System Access API — unavailable in Firefox / Safari</i>
NetCDF / HDF	Two paths: remote via kerchunk reference JSON + HTTP Range (cloud-optimized NetCDF); local supports <code>.nc .nc4 .cdf .h5 .hdf5</code> (NetCDF-3 via netcdfjs, NetCDF-4/HDF5 via h5wasm). HDF4 .hdf is explicitly unsupported
COG	See raster section

Online data catalogs (browsable directly):

- **STAC** — Both static catalogs and STAC API supported; discover catalogs via STAC Index. Only visualizable assets (GeoTIFF / GeoJSON) can be directly added
- **Source Cooperative** — Bucket browser; recognizes `.pmtiles .parquet .geoparquet .tif .geojson .geojsonl .ndjson .fgb .gpkg .csv`
- **Natural Earth** — Essentially the Source Coop panel locked to `opengeos/natural-earth`
- **Hugging Face Hub** — Browse dataset repositories; can even create repos and upload
- **Overture Maps** — Thematic vector tiles; defaults to showing buildings only
- **Microsoft Planetary Computer** — STAC + TiTiler
- **Google Earth Engine** — OAuth login; desktop has native OAuth flow
- **NASA Earthdata GIS** — EOSDIS ImageServer / MapServer / FeatureServer / published web maps
- **OpenAerialMap / Esri Wayback** (historical imagery) / **Mapillary** / **Google Street View**
- **Federal data services** — FEMA flood maps, EPA EnviroAtlas, USGS National Map
- **Weather & time-lapse** — RainViewer real-time radar, NASA GIBS, EOX cloudless annual basemap

Planetary data (this exceeds expectations): **the Moon and Mars come from OpenPlanetaryMap; Mercury, Venus, Galilean moons, Titan, Pluto, and Charon come from USGS Astrogeology.** The key detail is **per-project ellipsoid parameters**, so distance and area measurements are accurate for the celestial body you're measuring on.





Editor's note: Initially this seemed like a surface-level feature. The per-project ellipsoid parameters changed that impression — the project treats coordinate systems seriously, not as a mere basemap overlay.

8. Project & Style Interoperability

This is easy to overlook but determines whether GeoLibre can fit into your existing workflow.

Format	Extension	Direction
GeoLibre project	<code>.geolibre</code> / <code>.geolibreproj</code>	Read + write
QGIS project	<code>.qgz</code> <code>.qgs</code>	Import only. Parses XML with DOMParser; no QGIS code is executed; only imports the 17 recognized vector formats and GeoTIFF
Mapbox GL / MapLibre style	<code>.json</code>	Import + export
OGC SLD	<code>.sld</code> <code>.xml</code>	Import + export
QGIS QML	<code>.qml</code>	Import + export

Key insight: Symbology configured in QGIS can be brought over, and vice versa. Style import format is **determined by content, not by extension** — a file with the wrong extension will still be recognized.

Additionally, there are numerous `.json`-based libraries: style libraries, layer libraries ("My Data"), service libraries, legends, camera fly-throughs, story maps — all importable and exportable.

9. Export & Format Conversion: Essentially a Free GDAL Frontend

The official site underplays this, but it's one of the most practical capabilities.

Direct Layer Export

Right-click a layer to export — **all done in the browser, no backend needed:**

`.geojson` · `.csv` · `.kml` · `.kmz` · `.parquet` (GeoParquet) · `.gpkg` (GeoPackage) · `.zip` (Shapefile)

GeoPackage and Shapefile writing are pure JS implementations — because DuckDB-WASM cannot write GeoPackage, the author hand-wrote a GeoPackage 1.3 writer using `sql.js`; for Shapefile, five files (`.shp` / `.shx` / `.dbf` / `.prj` / `.cpg`) are manually assembled and then zipped.

Format Conversion Tools (9)

Tool	Input	Output
Vector → Vector	<code>geojson</code> <code>geojsonl</code> <code>json</code> <code>parquet</code> <code>geoparquet</code> <code>fgb</code> <code>gpkg</code> <code>shp</code> <code>zip</code> <code>kml</code> <code>gml</code> <code>gpx</code>	Desktop: 14 drivers — GeoJSON, GeoJSONSeq, FlatGeobuf, GPKG, Shapefile, GML, KML, CSV, SQLite, GMT, DXF, MapInfo, JML, GPX
Vector → GeoParquet	<code>parquet</code> <code>geoparquet</code> <code>geojson</code> <code>json</code> <code>shp</code> <code>gpkg</code> <code>fgb</code> <code>gml</code> <code>kml</code>	<code>.parquet</code> ; compression: <code>zstd</code> / <code>snappy</code> / <code>gzip</code> / <code>lz4</code> / <code>uncompressed</code>
Vector → FlatGeobuf	Same as above	<code>.fgb</code>
Vector → Shapefile	Same as above	<code>.zip</code>
Vector → GeoPackage	Same as above	<code>.gpkg</code>
CSV → GeoParquet	<code>csv</code> <code>tsv</code> <code>txt</code>	<code>.parquet</code>
Vector → PMTiles	<code>parquet</code> <code>geoparquet</code> <code>geojson</code> <code>json</code> <code>gpkg</code> <code>fgb</code> <code>shp</code>	<code>.pmtiles</code> . Desktop up to zoom level 24 ; browser uses WASM tiler with shallower levels
Raster → PMTiles	Only accepts <code>.tif</code> / <code>.tiff</code>	<code>.pmtiles</code> (single-band through color ramp, not true color)

Tool	Input	Output
Raster → COG	Desktop tif tiff img vrt asc nc jp2 hgt ; browser tif only	.tif ; compression: deflate zstd lzw webp jpeg packbits raw

Browser-side output format is a subset: geojson / json / csv / parquet / geoparquet / gpkg / zip / fgb. For the full 14-driver set, use the desktop version (via Python sidecar).

Other Exports

Export raster layers as GeoTIFF, clip raster subsets by bounding box, extract basemaps as PMTiles offline packages, print layout to PNG/PDF/multi-page ZIP, **export entire project as a standalone HTML file**, story maps to HTML/PDF, map screen recording to WebM, charts to SVG/PNG.

Export Project as Standalone HTML

This feature is severely underrated — sharing a single-file webpage with a collaborator to view results is far more practical than asking them to install specialized software.

10. Platform Differences: Points of Caution

Mentioned piecemeal above, consolidated here. **This is where things most easily go wrong.**

Limitation	Impact
Desktop (Tauri) exclusive	Native file/folder dialogs, local MBTiles, local raster reads, Shapefile companion file auto-discovery, PostGIS/Martin, file geodatabase, local file watch reload
Requires Python sidecar	File geodatabase, all desktop conversion tools (preferred path), raster tools without a browser engine (rasterio), AI segmentation, PostGIS, Sedona
Mac App Store build	No Python sidecar: hides PostgreSQL and GDB data sources, hides AI segmentation; Whitebox, conversion, raster, and vector tools all fall back to their browser/WASM engines; Shapefile companion files must be manually multi-selected
Android / iOS (mobile)	Hides raster tools, conversion tools, AI segmentation, PostgreSQL — all sidecar-backed. The Whitebox toolbox is WASM-backed and stays available
Browser	No local MBTiles/GDB/PostGIS; conversion output is subset; vector conversion doesn't accept .zip ; raster-to-COG only accepts GeoTIFF; Zarr local folders unavailable in Firefox/Safari

11. Architecture: How It Manages to Support So Many Formats

After listing over a hundred formats, the natural reaction is "how complex must this be." Here's a brief explanation of how it holds together.

The core is an engine-agnostic store. The `@geolibre/core` package depends on neither MapLibre, nor Cesium, nor deck.gl — it stores only two things: a set of flat layer records (id, type, source, style, visibility...) and a five-field `MapViewState` (center / zoom / bearing / pitch / bbox).

All UI operations modify only the store; a sync layer pushes changes to the rendering engine. Strictly one-way. Dragging an opacity slider in the UI modifies the store, not a MapLibre object.

The dividend of this design: **adding a format is cheap.** As long as a new format can be represented as a layer record, it automatically gains the full capabilities of the layer panel — opacity, ordering, project save, style export.

Division of labor among four rendering engines, behind a shared `MapEngine` interface:

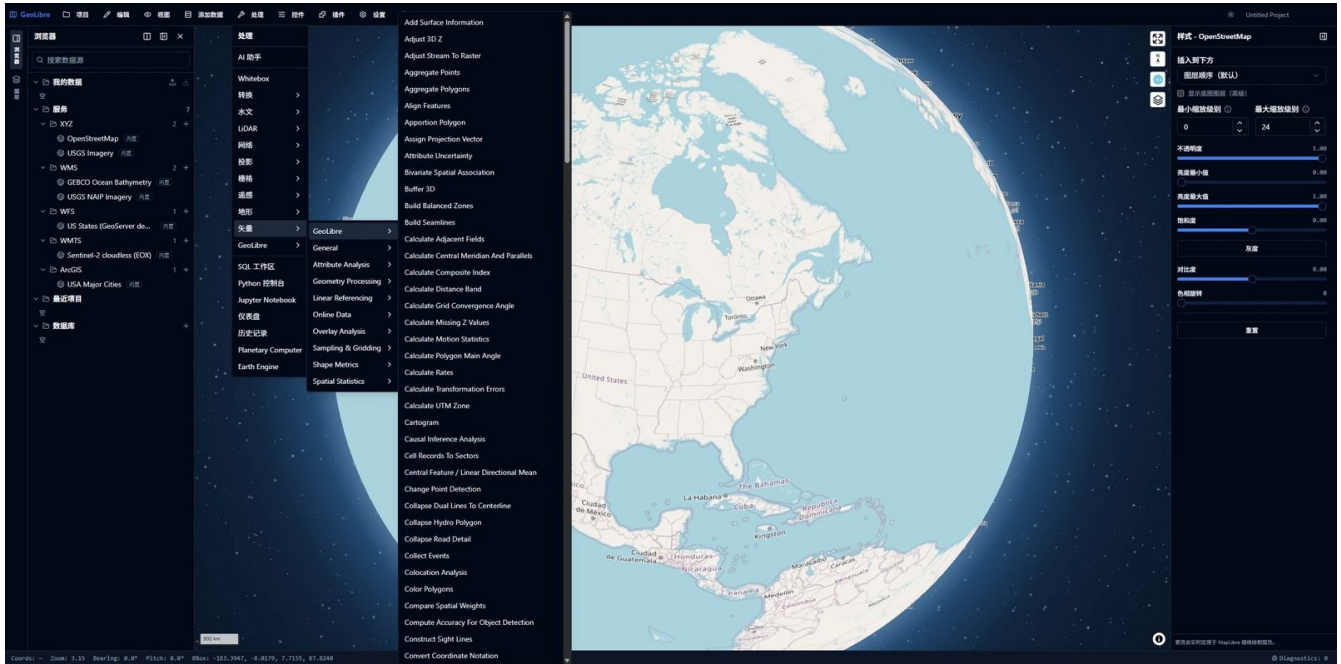
- **MapLibre** is the default and has the broadest layer, tool, and plugin support.
- **Mapbox** uses Mapbox GL JS for Mapbox styles and services while sharing much of MapLibre's Style Specification surface.
- **Cesium** is the native 3D-globe engine for terrain, 3D Tiles, CZML, ion assets, I3S, and globe-oriented workflows.
- **ArcGIS** uses the ArcGIS Maps SDK for native ArcGIS services and 2D or 3D Esri views.

deck.gl is not a fifth engine. It is an overlay hosted by compatible engines for COG, point-cloud, 3D, and visualization layers. Interleaved producers on the same map must share one overlay instance or a later instance can replace layers owned by an earlier one.



A noteworthy detail: **Cesium's camera sync is not aligned by zoom level, but by ground resolution (meters/pixel)**, so split-screen panels at different heights maintain the same on-screen scale.

Five compute engines, all under the same UI: DuckDB-WASM Spatial (the workhorse, running in a dedicated Worker), PGLite + PostGIS, Apache Sedona (sidecar or browser WASM version), Pyodide (running GeoPandas/Shapely in the browser), Whitebox WASM (1,000+ tools).



A clever approach: the Python implementation of vector operators is a "framework-free" module; the sidecar and the browser Pyodide execute the exact same code, so results from both paths are completely consistent — no "local computation doesn't match server computation" discrepancies.

12. Performance: Some Real Numbers

A quick overview, all verifiable numbers from the source — not estimates.

How large vectors are handled. Above 50,000 features, it stops using MapLibre's native GeoJSON source and instead tiles on-the-fly client-side — on a Web Worker, geojson-vt generates tiles (point layers use Supercluster for aggregation), vt-pbf encodes to MVT, then feeds MapLibre through a custom protocol. Max zoom 16, 4096 extent.

Two details that show engineering care: tile index objects are **deliberately excluded from the store** (too large, non-serializable, can't be written into a project file); encoding checks an abort signal before proceeding, since MapLibre cancels tile requests that scroll off-screen.

Other guard thresholds (these numbers are themselves good reference points):

Threshold	Value
Tiling render trigger	50,000 features
DuckDB result materialization confirmation prompt	500,000 rows
Browser-side Sedona cap	50,000 features
OSM PBF warning / timeout	50 MB / 5 minutes
Remote vector file cap	2 GiB (DuckDB-WASM uses 32-bit for remote file sizes)
Local COG cap	2 GiB
Undo history soft budget	500,000 features

Attribute table is virtualized, but sorting, filtering, and selection operate on the full data model — virtualization only governs rendering.

Undo history has a memory cap. Because every snapshot holds the full GeoJSON of layers, repeated editing pins multiple copies in memory. The approach: set a feature-count soft budget; when exceeded, discard the oldest snapshots, but **always retain the newest one**, guaranteeing at least one undo step even after a large edit. Slider drags are coalesced into a single undo record within 400 ms.

Bundle size is controlled through CDN offloading. Three heavyweight WASM engines (PGLite+PostGIS ~25 MB, CereusDB ~40 MB, gdal3.js ~28 MB + 12 MB) are not bundled. They're fetched from CDN at runtime by exact version; after first use, Service Worker caches them for offline availability. Two historical notes in source comments: **bundling PGLite would inflate the binary from 42 MB to 63 MB; CereusDB would increase the installer from 27 MB to 36 MB** — that's how the 30 MB installer figure is achieved.

One honest performance issue. The documentation has an entire section on Linux/WebKitGTK investigation: empty map steady at 60 FPS at any zoom, but with any tile layer present, **FPS drops to single digits during tile loading**, immediately returning to 60 once loading stops. Root cause: WebKitGTK performs GPU uploads for each new tile on the main thread (textures for raster, vertex buffers for vectors), **with a single tile integration cycle measuring ~125 ms versus Chromium's few ms**. The author ruled out software rendering, GPU saturation, Tauri IPC, JSON.parse, compositor latency, and numerous other possibilities. Mitigations (increase tile cache, 512px raster tiles, disable fade-in) are documented but **not yet implemented**. macOS/Windows WebViews have not been tested.

Editor's note: Projects that willingly document negative findings like this in their architecture docs are rare. This practice deserves recognition.

13. Its Boundaries

Having covered the strengths, let's discuss the limitations.

It Is Not a QGIS Replacement

This is the consensus across multiple reviews, and the author has never claimed otherwise.

- 1. The feature scope is deliberately narrow.** It focuses on browser workflows, local processing, cloud-native formats, spatial SQL, modern visualization, and portability. Complex professional workflows still belong in QGIS.
- 2. The iteration speed is a double-edged sword.** When the original article was written, it had gone from 0 to 2.4.0 in just over two months; 2.0.0 to 2.1.0 was separated by roughly 19 hours. If you're considering it as a long-term production dependency, factor in this churn risk.
- 3. Several clear format gaps.** `.obj` cannot be imported, HDF4 is unsupported, raw `.osm` XML is unsupported.
- 4. Platform capability asymmetry.** See the table in Section 10. Don't extrapolate the browser version's experience to represent the whole.
- 5. Cesium terrain and imagery need an ion token outside the web version.** The hosted web version bundles a demo token, but the desktop and mobile apps need your own; without one the globe still works, just without Cesium World Terrain and Ion imagery. The free tier is sufficient for individual use; teams need to budget for quotas and pricing.
- 6. The China-specific environment.** Default sources for basemaps, terrain, and Photorealistic 3D Tiles are all outside the firewall; coordinates use standard WGS84 — **GCJ-02 offsets must be handled separately**. These two issues need to be addressed first for serious use. Reliable first-hand data on this is unavailable; further input from actual users is welcome.

In Closing

You receive a zip archive containing a mix of shp, tif, gpkg, and tileset.json — previously requiring three or four different applications. The answer now: **open a 30 MB application (or just a webpage), and drag everything in.**

GeoLibre's real value is not that it's better than QGIS (it isn't) — it's that it drives **the cost of "just take a quick look at the data" down to near zero**. This niche was previously empty.

For those working with multi-engine or 3D GIS development, there's an additional layer of reference value: **the engine-agnostic store design**. Storing state as plain layer records and view state, rather than tying it to a specific engine's objects, lets another renderer plug in without changing the project model. This approach is worth adopting.

Usage paths by scenario:

1. **Just want to take a look** — Open `web.geolibre.app` directly; no installation, no registration
2. **Need to get real work done** — Download the desktop version from GitHub Releases, or via Microsoft Store / Homebrew / winget; a two-minute process
3. **Python users** — `pip install geolibre`; for GeoPandas support, `pip install "geolibre[all]"`; requires Python 3.11+
4. **Intranet / offline environments** — `VITE_PYODIDE_INDEX_URL` and `VITE_DUCKDB_SPATIAL_EXTENSION_PATH` can point Pyodide and the DuckDB spatial extension to internal mirrors, **no rebuild required**; an official Docker image is also available at `ghcr.io/opegeos/geolibre:latest`
5. **Secondary development** — npm workspaces monorepo, so use **npm** (the repo tracks `package-lock.json`) on **Node 22+**; main application in `apps/geolibre-desktop`; MIT licensed

Final takeaway: for data preview and exploration, highly recommended; for production deployment, wait a bit longer — at least until the version iteration cadence stabilizes.

<https://geolibre.app/tutorials/supported-formats-zh/>

GeoLibre 支持 GIS 数据格式完整清单

摘要：本文完整梳理 GeoLibre 内置支持的全部 GIS 数据格式，涵盖矢量、栅格、云原生地理空间格式及每种格式的内部解析逻辑，为国内 WebGIS 开发者提供完整的技术参考清单。

原文来源："GIS开发手记"，原文链接：<https://mp.weixin.qq.com/s/GrtQzetCBgKtYruEhjsOAw>

本文使用简体中文撰写。

前言

本文完整梳理 GeoLibre 内置支持的全部地理空间数据格式，区分矢量、栅格、云原生地理格式，同时说明每种格式对应的解析依赖、读取优先级与适用场景，可供 WebGIS 开发人员快速查阅。

一、运行形态与平台说明

列出格式之前先说清楚「在哪儿跑」，因为后面很多格式有平台差异。

形态	怎么跑	说明
浏览器	打开 <code>web.geolibre.app</code>	什么都不用装，加载完可离线用
桌面	Tauri v2 原生应用	Windows / macOS / Linux，微软商店、Mac App Store、Homebrew、winget、AUR、Flatpak 都有
安卓	Google Play 原生 App	每 ABI 约 40MB
iOS	App Store 原生 App	iPhone 与 iPad，同一套代码经 Tauri v2 mobile 构建
Jupyter	<code>pip install geolibre</code>	整个应用嵌进 notebook 单元格

核心应用没有账号、没有服务器、没有费用。 本地文件就地读取、不出本机，应用加载完成后本地流程也能离线继续用。例外的是那些可选的远程能力：在线目录（STAC、Source Cooperative、Overture、Planetary Computer）、从 CDN 下载底图与瓦片，以及 Earth Engine、需要鉴权的 ArcGIS 服务等，都需要联网，部分还需要各自的凭据或 OAuth 登录。

二、矢量数据格式支持

矢量数据是 GeoLibre 适配最完善的能力模块，文件选择器内置统一的格式白名单，支持拖拽自动识别，无需手动选择文件类型，覆盖 17 种主流矢量扩展名。

```
geojson, json, gpkg, geoparquet, parquet, fgb, flatgeobuf,
csv, tsv, kml, kmz, gml, gpx, dxf, tab, shp, zip
```

拖进去就行，不用先选「我要导入什么格式」。

但真正有意思的是**每种格式背后是谁在读**——这决定了它的实际表现：

格式	扩展名	读取引擎	值得注意的细节
GeoJSON	.geojson .json	原生 JSON.parse	带老式顶层 crs 成员会自动走 DuckDB 重投影到 WGS84
GeoParquet / Parquet	.parquet .geoparquet	DuckDB read_parquet	远程文件走 HTTP Range 流式读，不用整个下载
FlatGeobuf	.fgb .flatgeobuf	DuckDB ST_Read	—
GeoPackage	.gpkg	sql.js (SQLite WASM) ，不是 GDAL	多图层会弹选择器；会先修复 gpkg_ogr_contents
Shapefile (散文件)	.shp	shpjs	桌面端自动读同名 .dbf/.shx/.prj/.cpg；3D MultiPatch 改走 DuckDB
Shapefile (压缩包)	.zip	fflate 解压 → shpjs	.prj 决定投影，.cpg 决定 DBF 编码（中文属性乱码可以得到正确处理）；自动跳过 macOS 的 __MACOSX
KML	.kml	自研解析器	保留内嵌符号化和 Folder 结构；带 <TimeSpan> / <TimeStamp> 的地标能接时间轴动画；还能吐出 GroundOverlay 图像和 <Model> 三维模型
KMZ	.kmz	fflate 解压	自定义图标、格式化描述都保留
GML	.gml	DuckDB ST_Read	—
GPX	.gpx	纯 JS	自动拆成三个图层：航点 / 轨迹 / 路线
CSV / TSV	.csv .tsv .txt .dat	自研 + DuckDB 兜底	自动识别分隔符和经纬度列；WKT 几何列走 DuckDB；对话框可指定源坐标系
Excel	.xlsx .xls	SheetJS (@e965/xlsx)	从分隔文本面板打开：先选工作表，再选坐标列
CAD (DXF/DWG)	.dxf .dwg	DuckDB ST_Read	会读出图层清单让你挑；CAD 不带坐标系，需要手动选 EPSG
MapInfo TAB	.tab	ST_Read	—

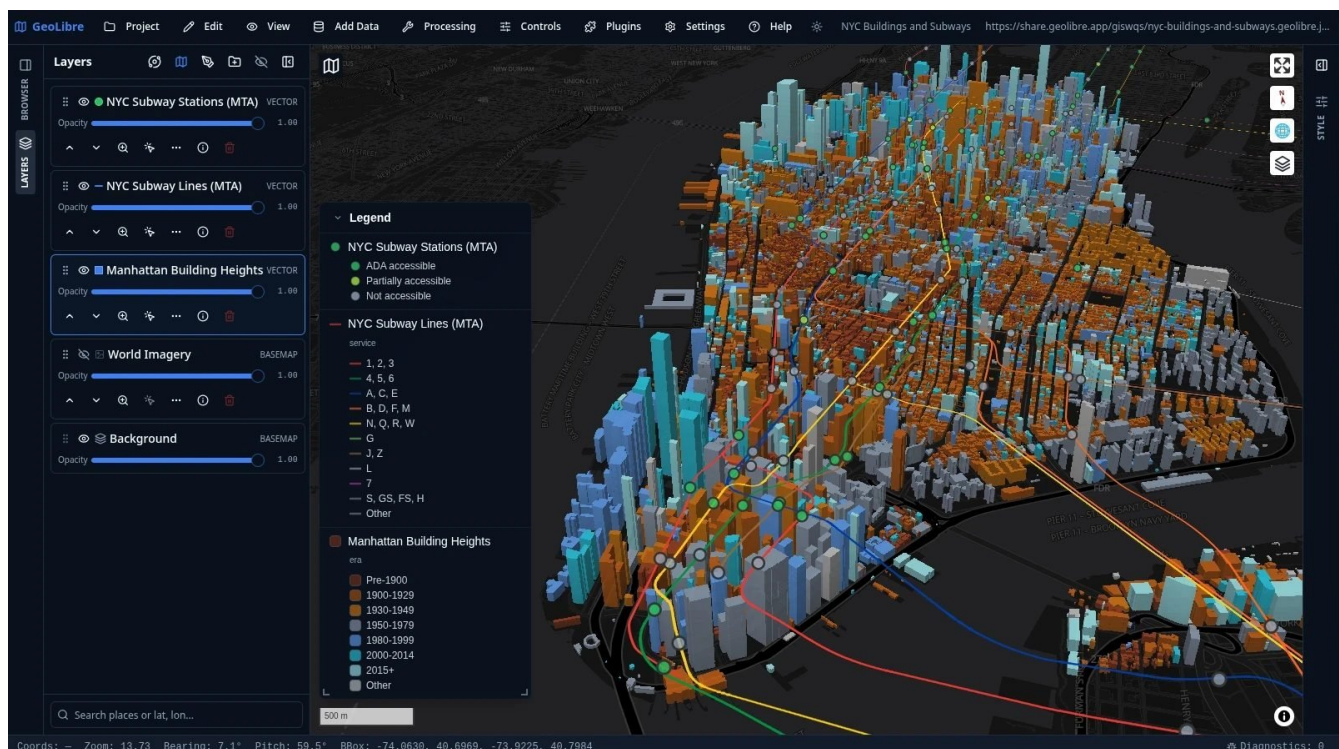
格式	扩展名	读取引擎	值得注意的细节
Esri 文件 地理数据库	.gdb 文件夹	Python sidecar	桌面端专属，且要 sidecar；Mac App Store 版本里是隐藏的
OSM PBF	.osm.pbf .pbf	osmfix，跑在 Web Worker	自动拆成点/线/面三层；超 50MB 会提示确认，5 分钟超时保护
GeoRSS	.xml .rss .atom	纯 JS	RSS 2.0 / Atom / RDF 都吃，GeoRSS Simple + GML 几何
地理标记 照片	.jpg .jpeg .png .tif .tiff .webp .heic	读取 EXIF GPS 直接生成点图层，无人机照片很实用	

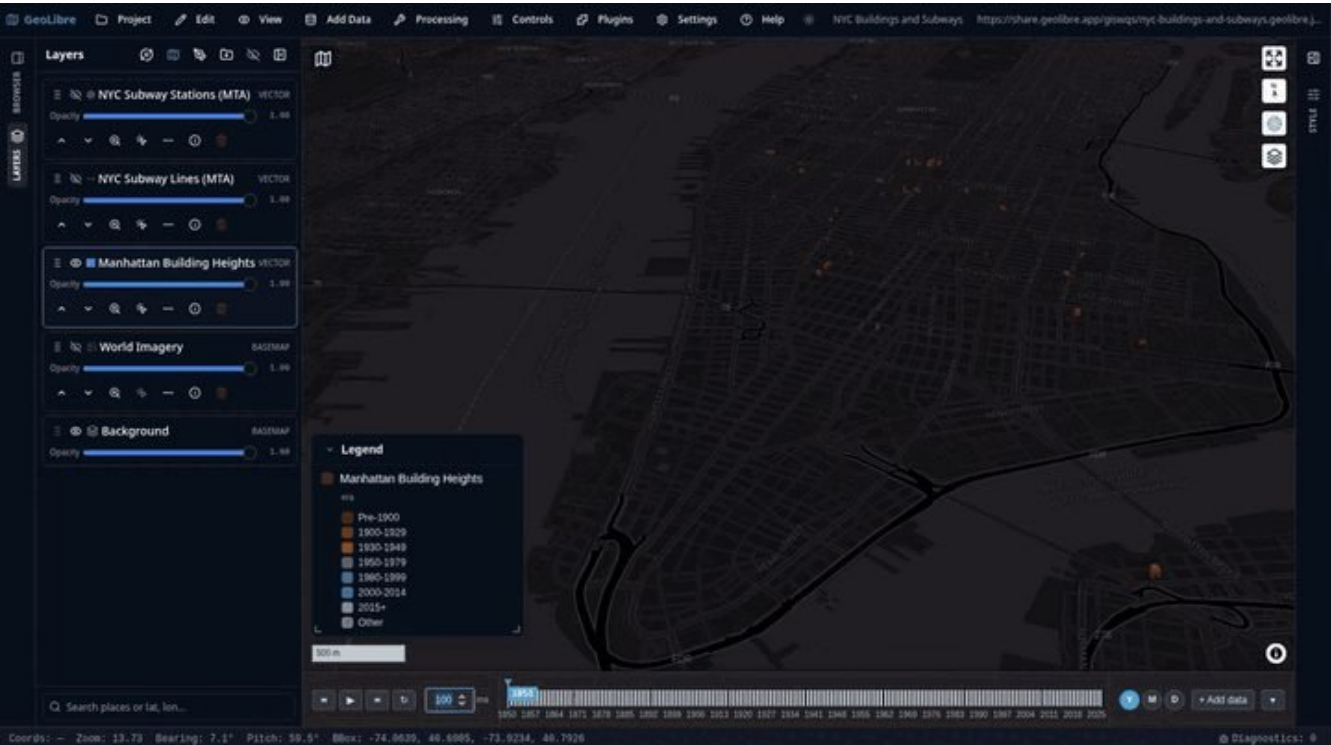
明确不支持的：原始 .osm XML（只认 PBF）。

KML <Model> 的三维模型

KML 的 <Model> 这条值得关注：它会把内嵌的 COLLADA .dae 用 three.js 加载再导出成 GLB 加载到地图中。为了这一个边缘场景引入了 three.js。

加载完之后不是「能看见就完事」——符号化、分级配色、图例都是跟着走的：

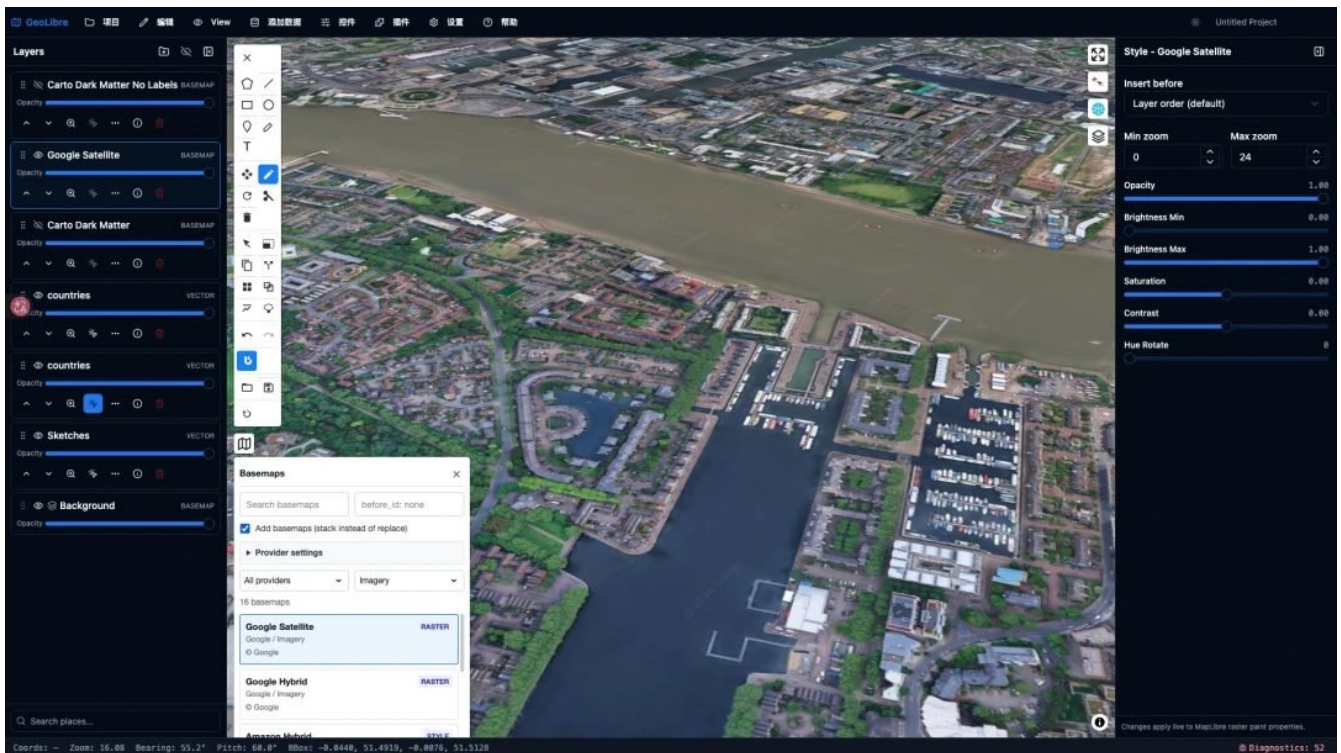




三、栅格与影像

栅格这块比矢量窄，但覆盖了云原生那条主线。

格式	扩展名	引擎	平台
GeoTIFF / COG	.tif .tiff	默认 cog-tiler-wasm，可切 GPU 引擎	全平台；桌面端本地文件走 Tauri asset 协议做 Range 读
MosaicJSON / STAC item	.json	读取时按需拼接	全平台
地理配准图像	任意浏览器能解码的图 + GCP 的 .csv / .txt	最小二乘仿射，导出用 gdal3.js	全平台（gdal3.js 从 CDN 加载）
KML GroundOverlay	来自 .kml / .kmz	四角坐标图像图层	带 <TimeSpan> 的还能接时间轴动画
地理配准视频	.mp4 + .webm	MapLibre video source	只支持 URL
转换工具输入（栅格）	.tif .tiff .img .vrt .asc .nc .jp2 .hgt	GDAL / rasterio sidecar	桌面端；浏览器只收 .tif/.tiff
Whitebox 栅格 I/O	.tif .tiff .img .bil .flt .sdat .rdc .asc	whitebox-wasm	全平台



注意这个不对等：`.img`、`.vrt`、`.asc`、`.jp2`、`.hgt` 这些只在**转换工具**里认，不能直接当图层拖进去。直接拖只认 *GeoTIFF*。

四、点云与 LiDAR

项	支持	说明
LiDAR 图层	LAS / LAZ / COPC (URL 或本地文件)、EPT (ept.json URL)	走 maplibre-gl-lidar + deck.gl 渲染
USGS 3DEP	在线点云流式加载	基于 maplibre-gl-usgs-lidar 的独立插件，会附带 3DEP 高程索引 WMS 覆盖图
Whitebox LiDAR 工具	.las .laz .zlidar .copc .e57 .ply，输出 .laz	这是全库唯一出现 .e57 / .ply 的地方

LiDAR 的文件选择器本身在上游 maplibre-gl-lidar 包里；*GeoLibre* 的封装（`packages/plugins/src/plugins/components/lidar.ts`）写明支持 *LAS/LAZ/COPC* 文件和 *EPT* 的 `ept.json URL`，拖进来的本地 *COPC* 会按八叉树节点流式加载，不整份复制。

五、三维模型、瓦片与高斯泼溅

开篇那个「收到 3D Tiles 想看一眼」的场景，这节就是答案。

格式	输入方式	引擎	说明
OGC 3D Tiles	tileset URL	maplibre-gl-3d-tiles + deck.gl Tile3DLayer	支持自定义请求头，带鉴权的切片也能加
Google 照片级 3D Tiles	内置 URL	同上	需要 Google Maps API key，走请求头传，不落盘
ArcGIS I3S 场景图层	.../SceneServer URL	deck.gl + loaders.gl I3SLoader	整合网格和三维对象图层都支持
glTF / GLB	URL 或本地文件	deck.gl ScenegraphLayer	通过 deck.gl 可视化图层的 scenegraph 类型添加
COLLADA .dae	只能通过 KML <Model> 内嵌	three.js → GLB	—
高斯泼溅	URL	maplibre-gl-splat	存储层类型是 gaussian-splat



上面这张图挺能说明问题：左侧图层面板里 3D Tiles、矢量、XYZ、glTF 模型、高斯泼溅是叠在同一个列表里的，右边直接就是渲染结果。「加载 3D Tiles 要起服务器写页面」这件事在这儿就是粘贴一个 URL。

没找到的：`.obj` 不能导入（它只作为拉伸图层三维模型导出的一种输出格式出现，和 `GLB`、`STL` 并列）。`.b3dm` / `.pnts` / `.cmpt` 这些 3D Tiles 内部格式在仓库里也搜不到——它们被上游加载器透明处理了，不用你操心。高斯泼溅的扩展名清单同样在上游包里。

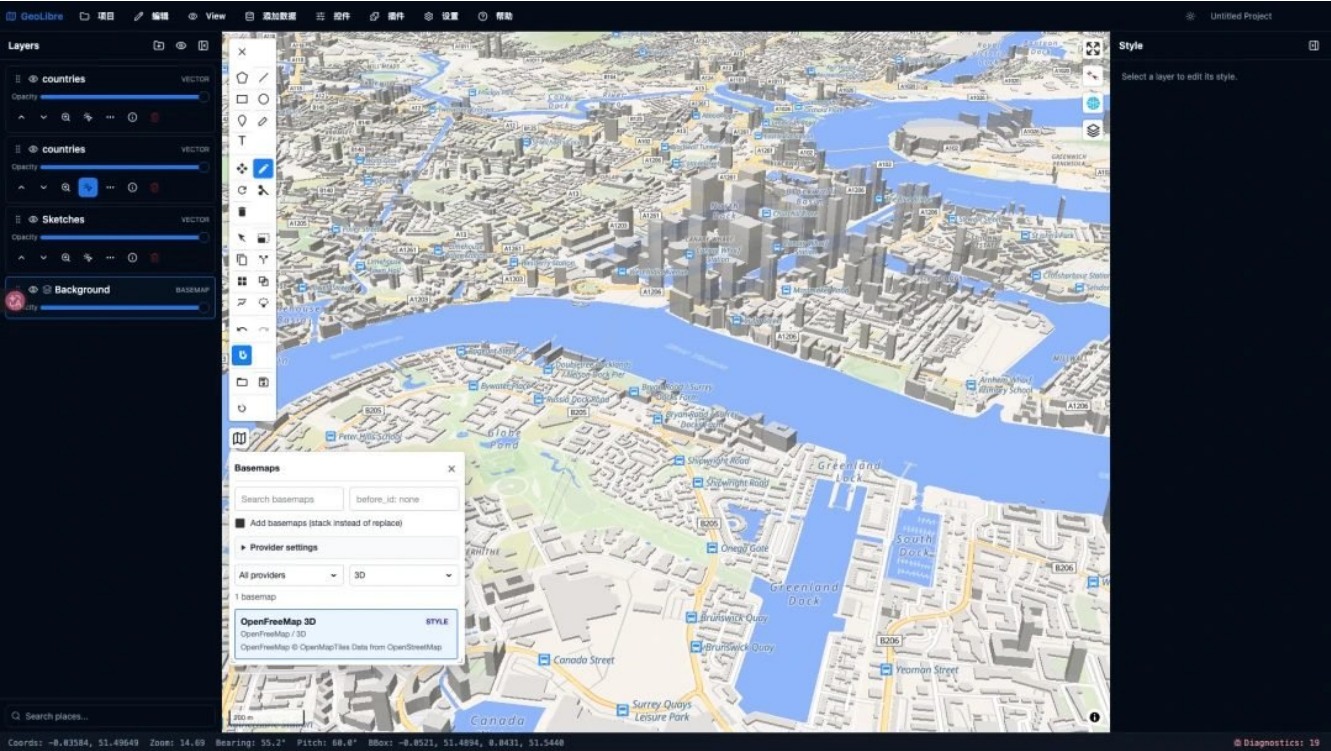
六、瓦片与 OGC 服务

存储层的图层类型枚举一共 20 种：

```
geojson, raster, wms, wmts, xyz, vector-tiles, arcgis, pmtiles, mbtiles, zarr, lidar, gaussian-splat, 3d-tiles, cog, flatgeobuf, geoparquet, duckdb-query, deckgl-viz, video, image
```

「添加数据 → Web 服务」里能填的：

服务	细节
XYZ	{z}/{x}/{y} 模板，栅格或矢量瓦片都行
WMS	GetCapabilities 自动拉图层下拉框；点击要素走 GetFeatureInfo；开发态自带 CORS 代理
WMTS	RESTful 瓦片模板
WFS	GetCapabilities 拉 typeName；可选自动刷新
OGC API - Features	落地页 / /collections / 单个集合 / 完整 /items URL 都认，自动翻 next 链接，默认取 1000 条
OGC API - Tiles (矢量)	TileJSON 或 MVT 模板，可另填 Mapbox style URL 来解析 source-layer 名
ArcGIS	四种图层类型：FeatureServer（按页下载成 GeoJSON）、VectorTileServer、MapServer 和 ImageServer（后两种作为栅格图层）；支持门户项目 ID 和 ArcGIS 登录
MBTiles	.mbtiles 本地文件，自定义协议 + Rust 后端读取。桌面端专属
PMTiles	.pmtiles，矢量栅格都行，自动嗅探文件头
PostgreSQL / PostGIS	连接 → 选表 → 出 MVT，靠内置的 Martin 服务。桌面端专属
deck.gl 可视化图层	14 种：散点、热力、六边形、格网、屏幕格网、等值线、弧线、线、大圆、GeoJSON、图标、文本、轨迹、场景图



七、云原生格式与数据目录

这一节是 GeoLibre 和传统桌面 GIS 拉开距离的地方。

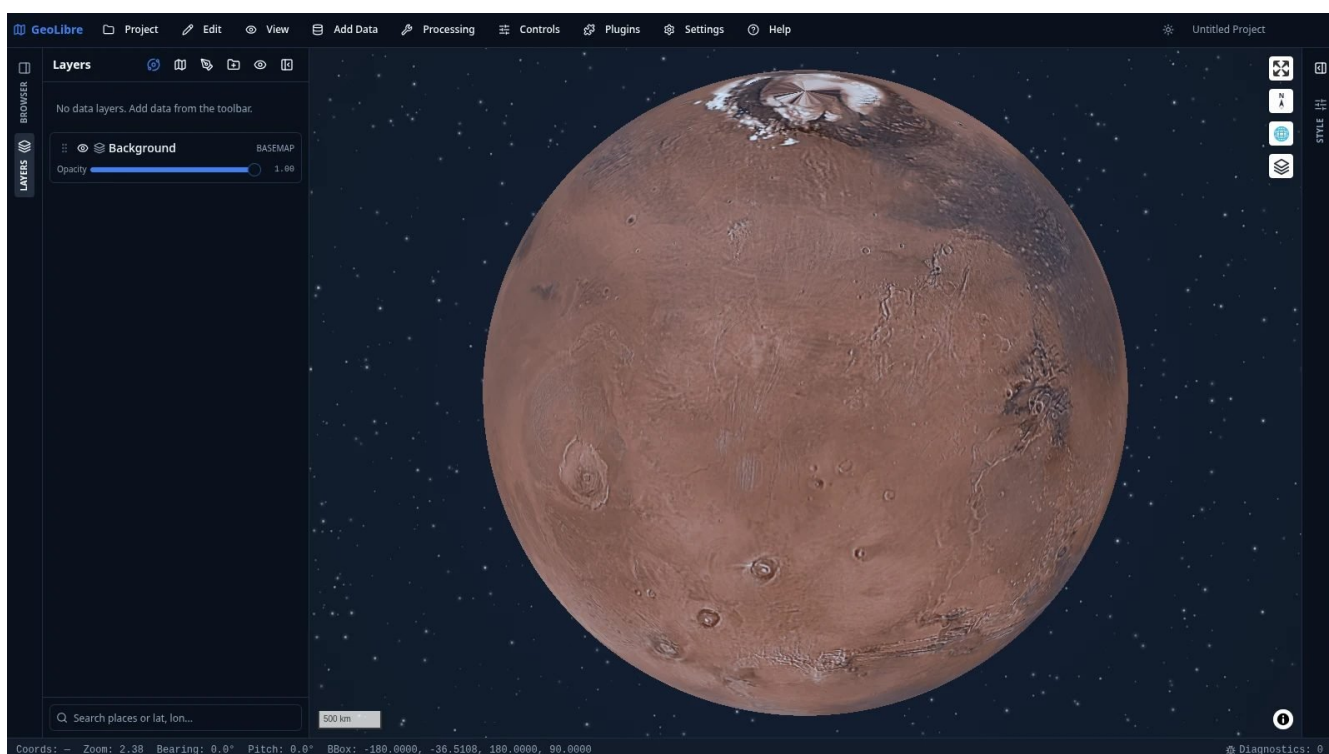
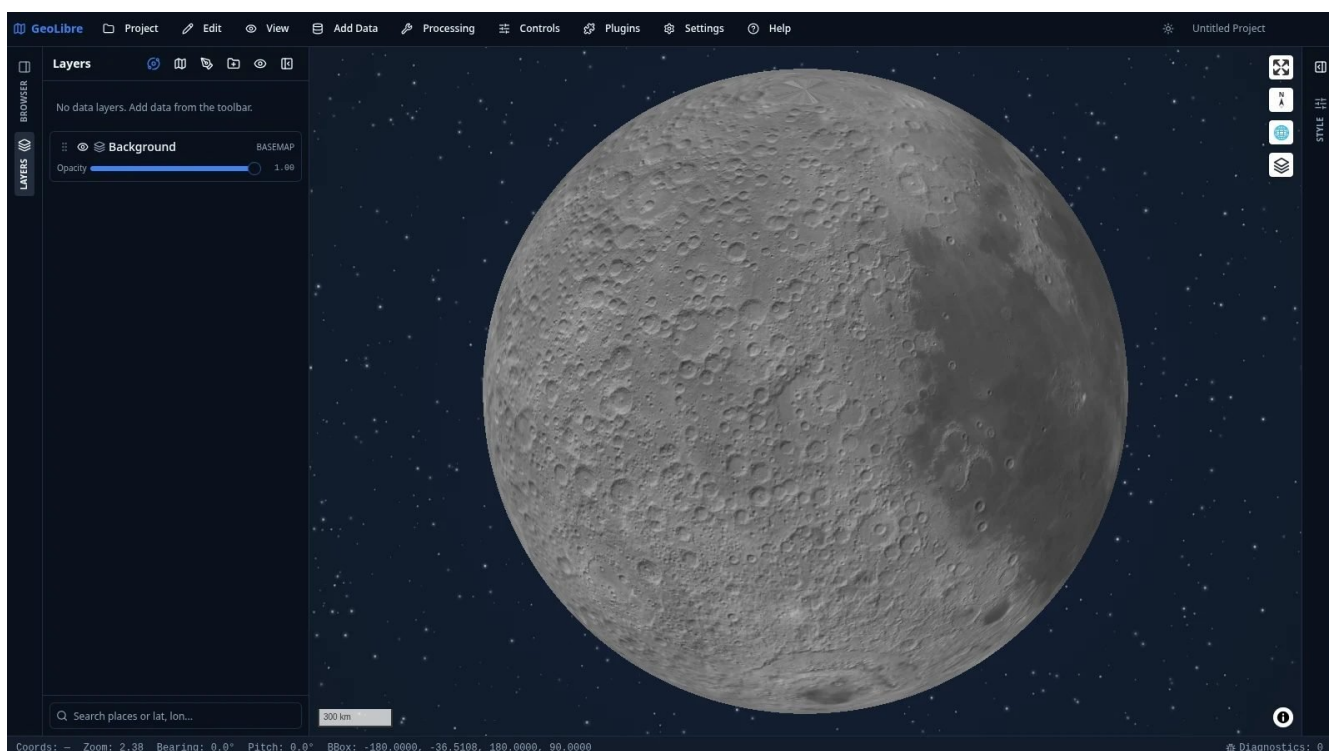
科学数据格式：

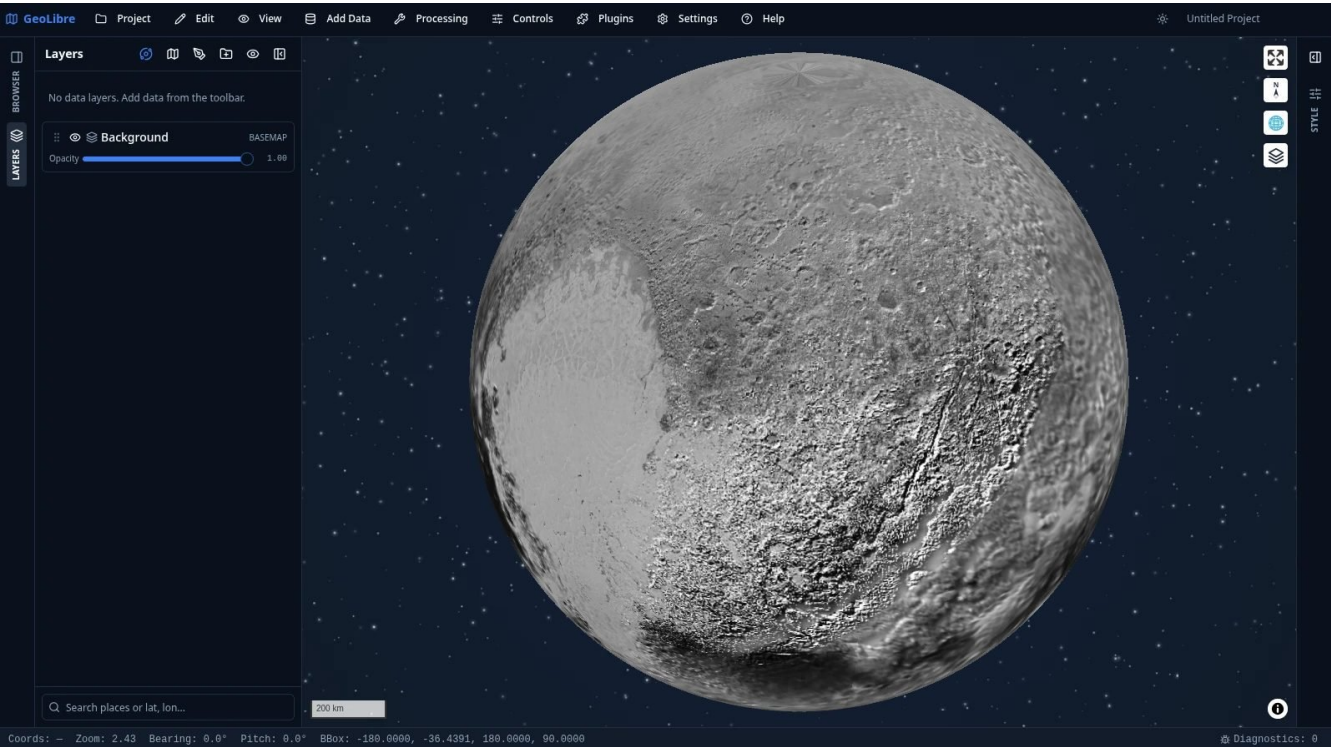
格式	支持范围
Zarr	远程 store URL，或本地文件夹。变量和维度选择器会显示真实坐标值。本地文件夹在浏览器里需要 <i>File System Access API</i> ， <i>Firefox / Safari</i> 用不了
NetCDF / HDF	两条路：远程走 kerchunk 引用 JSON + HTTP Range （云优化 NetCDF）；本地支持 <code>.nc .nc4 .cdf .h5 .hdf5</code> （NetCDF-3 走 netcdfjs，NetCDF-4/HDF5 走 h5wasm）。 HDF4 的 .hdf 明确不支持
COG	见栅格一节

在线数据目录（打开就能浏览的）：

- **STAC** —— 静态目录和 STAC API 都支持，通过 STAC Index 发现目录。只有可视化资产（GeoTIFF / GeoJSON）能直接加
- **Source Cooperative** —— 桶浏览器，认 `.pmtiles .parquet .geoparquet .tif .geojson .geojsonl .ndjson .fgb .gpkg .csv`
- **Natural Earth** —— 本质是 Source Coop 面板锁定到 `opengeos/natural-earth`
- **Hugging Face Hub** —— 浏览数据集仓库，甚至能建仓库上传
- **Overture Maps** —— 主题化矢量瓦片，默认只显示建筑
- **Microsoft Planetary Computer** —— STAC + TiTiler
- **Google Earth Engine** —— OAuth 登录，桌面端有原生 OAuth 流程
- **NASA Earthdata GIS** —— EOSDIS 的 ImageServer / MapServer / FeatureServer / 已发布 web 地图
- **OpenAerialMap / Esri Wayback**（历史影像） / **Mapillary / Google 街景**
- **联邦数据服务** —— FEMA 洪水图、EPA EnviroAtlas、USGS National Map
- **气象与延时** —— RainViewer 实时雷达、NASA GIBS、EOX 无云年度底图

行星数据（这是一个超出预期的设计）：**月球、火星来自 OpenPlanetaryMap，水星、金星、伽利略卫星、土卫六、冥王星、卡戎来自 USGS Astrogeology**。关键细节是**每个工程独立的椭球体参数**，所以距离和面积量算跟你测的那颗星球是对得上的。





编者注：最初以为这只是表面文章，但每个工程独立的椭球体参数这个细节改变了这一看法——项目对坐标系的处理是严谨的，不是简单地贴图了事。

八、工程与样式互通

这条容易被忽略，但决定了你能不能将它集成到现有工作流中。

格式	扩展名	方向
GeoLibre 工程	.geolibre / .geolibre 读写	
QGIS 工程	.qgz .qgs	只能导入。用 DOMParser 解析 XML，不执行任何 QGIS 代码；只导入它认识的 17 种矢量格式和 GeoTIFF
Mapbox GL / MapLibre 样式	.json	导入 + 导出
OGC SLD	.sld .xml	导入 + 导出
QGIS QML	.qml	导入 + 导出

关键启发：你在 QGIS 里配好的符号化可以搬过来，反过来也行。样式导入的格式是按内容判断的，不看扩展名——扔个改错后缀的文件也能认出来。

另外还有一堆 `.json` 形态的库：样式库、图层库（「我的数据」）、服务库、图例、相机漫游、故事地图，都能导入导出。

九、导出与格式转换：相当于一个免费的 GDAL 前端

这块被官网说得太轻，但是最实用的功能之一。

图层直接导出

右键图层就能导出，全部在浏览器里完成，不用后端：

`.geojson` · `.csv` · `.kml` · `.kmz` · `.parquet`（GeoParquet）· `.gpkg`（GeoPackage）· `.zip`（Shapefile）

GeoPackage 和 Shapefile 的写出是纯 JS 实现的——因为 DuckDB-WASM 写不了 GeoPackage，作者干脆用 `sql.js` 手写了一个 GeoPackage 1.3 写入器；Shapefile 则是手工拼 `.shp/.shx/.dbf/.prj/.cpg` 五个文件再打包。

格式转换工具（9 个）

工具	输入	输出
矢量 → 矢量	<code>geojson geojsonl json parquet geoparquet fgb gpkg shp zip kml gml gpx</code>	桌面端 14 种驱动：GeoJSON、GeoJSONSeq、FlatGeobuf、GPKG、Shapefile、GML、KML、CSV、SQLite、GMT、DXF、MapInfo、JML、GPX
矢量 → GeoParquet	<code>parquet geoparquet geojson json shp gpkg fgb gml kml</code>	<code>.parquet</code> ，压缩可选 <code>zstd / snappy / gzip / lz4 / 不压缩</code>
矢量 → FlatGeobuf	同上	<code>.fgb</code>
矢量 → Shapefile	同上	<code>.zip</code>
矢量 → GeoPackage	同上	<code>.gpkg</code>
CSV → GeoParquet	<code>csv tsv txt</code>	<code>.parquet</code>
矢量 → PMTiles	<code>parquet geoparquet geojson json gpkg fgb shp</code>	<code>.pmtiles</code> 。桌面端最高 24 级，浏览器端因为用 WASM 切片器层级要浅一些
栅格 → PMTiles	只收 <code>.tif/.tiff</code>	<code>.pmtiles</code> （单波段过色带，不是真彩色）

工具	输入	输出
栅格 → COG	桌面 tif tiff img vrt asc nc jp2 hgt，浏览器只收 tif	.tif，压缩可选 deflate zstd lzw webp jpeg packbits raw

浏览器端输出格式是子集：*geojson / json / csv / parquet / geoparquet / gpkg / zip / fgb*。想要完整的 14 种驱动得用桌面版（走 *Python sidecar*）。

其它导出

栅格图层导 GeoTIFF、按框裁栅格子集、底图抽取成 PMTiles 离线包、打印布局出 PNG/PDF/多页 ZIP、整个工程导成一个独立 HTML 文件、故事地图导 HTML/PDF、地图录屏 WebM、图表导 SVG/PNG。

工程导成独立 HTML

这条功能被严重低估了——给协作者分享单文件网页查看成果，比让对方安装专用软件现实得多。

十、平台差异：注意事项

前面零散提过，这里集中一次。这是最容易吃亏的地方。

限制	影响什么
桌面端（Tauri）专属	原生文件/文件夹对话框、本地 MBTiles、本地栅格读取、Shapefile 同名文件自动发现、PostGIS/Martin、文件地理数据库、本地文件监听重载
需要 Python sidecar	文件地理数据库、桌面端的全部转换工具（首选路径）、没有浏览器引擎的栅格工具（rasterio）、AI 分割、PostGIS、Sedona
Mac App Store 版本	不带 Python sidecar：隐藏 PostgreSQL 和 GDB 数据源、隐藏 AI 分割；Whitebox、转换、栅格、矢量工具全部退回浏览器/WASM 引擎；Shapefile companion 文件要手动多选
安卓 / iOS 移动端	隐藏栅格工具、转换工具、AI 分割、PostgreSQL——这些都依赖 sidecar。Whitebox 工具箱走 WASM，依然可用
浏览器端	无本地 MBTiles/GDB/PostGIS；转换输出是子集；矢量转换不收 .zip；栅格转 COG 只收 GeoTIFF；Zarr 本地文件夹在 Firefox/Safari 不可用

十一、架构：它凭什么能吃下这么多格式

前面列了一百多种格式，容易让人觉得「过于复杂」。所以简单说一下它如何支撑的。

核心是一个引擎无关的 store。 `@geolibre/core` 这个包不依赖 MapLibre、不依赖 Cesium、不依赖 deck.gl——它只存两样东西：一组扁平的图层记录（id、类型、source、样式、可见性...）和一个五个字段的 `MapViewState`（center / zoom / bearing / pitch / bbox）。

所有 UI 操作都只改 store，再由同步层推给渲染引擎，严格单向。 你在界面上拖透明度滑块，改的是 store，不是 MapLibre 对象。

这个设计的红利就是**加格式很便宜**：新格式只要能变成一条图层记录，就自动获得图层面板、透明度、排序、工程保存、样式导出的全部能力。

四个渲染引擎的分工，统一通过 MapEngine 接口接入：

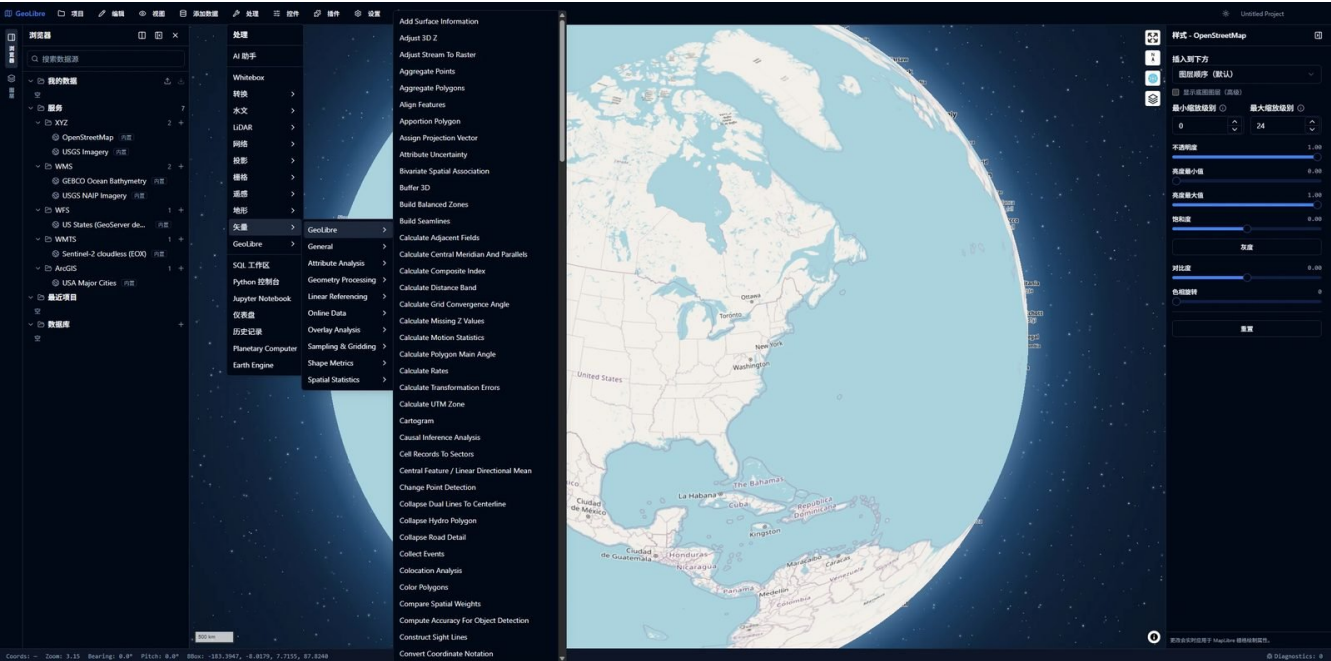
- **MapLibre** 是默认引擎，图层、工具和插件支持最全面。
- **Mapbox** 使用 Mapbox GL JS 提供 Mapbox 样式和服务，同时与 MapLibre 共享大部分 Style Specification 能力。
- **Cesium** 是原生三维球引擎，适合地形、3D Tiles、CZML、ion 资产、I3S 和面向全球的三维 workflow。
- **ArcGIS** 使用 ArcGIS Maps SDK，原生支持 ArcGIS 服务以及 Esri 的二维地图和三维场景。

deck.gl 不是第五个引擎。 它是由兼容引擎承载的 overlay，用于 COG、点云、三维内容和可视化图层。同一地图上交织渲染的生产者必须共用一个 overlay 实例，否则后创建的实例可能覆盖前一个实例管理的图层。



一个值得注意的细节：**Cesium** 的相机同步不是按 zoom 级别对齐的，而是按地面分辨率（米/像素），所以不同高度的分屏面板能保持同样的屏幕比例尺。

计算引擎有五套，都挂在同一套 UI 下：DuckDB-WASM Spatial（主力，跑在独立 Worker 里）、PGLite + PostGIS、Apache Sedona（sidecar 或浏览器 WASM 版）、Pyodide（浏览器里跑 GeoPandas/Shapely）、Whitebox WASM（1,000+ 工具）。



一个很聪明的做法：矢量算子的 Python 实现是一个「无框架」模块，sidecar 和浏览器 Pyodide 执行的是同一份代码，所以两条路径结果完全一致，不会出现「本地算和服务端算不一样」。

十二、性能：几个真实的数字

也简单说几个，都是源码里能查到的实数，不是估算的。

大矢量的处理方式。超过 5 万要素，就不再走 MapLibre 原生 GeoJSON 源，而是在客户端现场切成矢量瓦片——在 Web Worker 里由 geojson-vt 生成瓦片（点图层用 Supercluster 聚合），vt-pbf 编码成 MVT，再通过一个自定义协议喂给 MapLibre。最大 16 级，4096 extent。

两个细节能看出功力：瓦片索引对象故意不放进 store（太大、不可序列化，不能写进工程文件）；编码前会检查中止信号，因为 MapLibre 会取消滚出屏幕的瓦片请求。

其它保护阈值（这些数字本身就是很好的参考）：

阈值	值
切片渲染触发	50,000 要素
DuckDB 结果物化确认提示	500,000 行
浏览器端 Sedona 上限	50,000 要素
OSM PBF 提醒 / 超时	50 MB / 5 分钟
远程矢量文件上限	2 GiB（DuckDB-WASM 用 32 位存远程文件大小）

阈值	值
本地 COG 上限	2 GiB
撤销历史软预算	500,000 要素

属性表是虚拟化的，但排序、筛选、选择是在完整数据模型上跑的，虚拟化只管渲染。

撤销历史有内存上限。因为每个快照都持有图层的完整 GeoJSON，反复编辑会把好几份副本钉在内存里。做法是设一个要素数软预算，超了就丢最老的快照，但**永远保留最新一个**，保证大编辑至少能撤销一步。滑块拖动会在 400ms 内合并成一次撤销记录。

体积是通过 CDN 分发来控制的。三个重量级 WASM 引擎（PGLite+PostGIS 约 25MB、CereusDB 约 40MB、gdal3.js 约 28MB+12MB）都不打进包，运行时按版本号从 CDN 拉，首次用过之后 Service Worker 会缓存下来离线可用。源码注释里留了两处历史记录：**把 PGLite 打进包会让二进制从 42MB 涨到 63MB，CereusDB 会让安装包从 27MB 涨到 36MB**——这就是安装包保持在 30MB 的原因。

一个诚实的性能问题。文档里有一整节 Linux/WebKitGTK 的排查记录：空白地图稳定 60 FPS，但只要有瓦片图层，**加载瓦片期间帧率掉到个位数**，加载完立刻回到 60。根因是 WebKitGTK 在主线程处理每张新瓦片的纹理上传，**一个瓦片集成周期约 125 毫秒，而 Chromium 只要几毫秒**。作者排除了软件渲染、GPU 饱和、Tauri IPC、JSON.parse、合成器延迟等一堆可能。缓解措施（加大瓦片缓存、512px 栅格瓦片、关掉淡入）文档里写了，但**还没实现**。macOS/Windows 的 WebView 未测试。

编者注：能把这种负面结果写进架构文档的项目不多。这一做法值得肯定。

十三、它的边界

说完优势，来谈谈局限性。

它不是 QGIS 的替代品

这是多篇评测的共识，作者自己也没这么宣称。

一、功能范围是故意收窄的。它聚焦在浏览器 workflow、本地处理、云原生格式、空间 SQL、现代可视化和可移植性。复杂的专业流程该用 QGIS 还得用 QGIS。

二、迭代速度是双刃剑。原文写作时，它两个多月就从 0 到了 2.4.0，2.0.0 到 2.1.0 只隔了大概 19 小时。拿它做生产环境的长期依赖，得想清楚这个 churn 风险。

三、几个明确的格式缺口。`.obj` 不能导入、HDF4 不支持、原始 `.osm` XML 不支持。

四、平台能力不对等。见第十节那张表。别拿浏览器版的体验去代表全部。

五、Cesium 的地形和影像在网页版之外需要 Ion token。 网页版内置了演示 token，桌面版和移动版需要自己的 token；没有 token 三维球照样能用，只是没有 Cesium World Terrain 和 Ion 影像。免费额度够个人玩，团队用要算账。

六、国内环境。 底图、地形、Photorealistic 3D Tiles 这些默认源都在墙外；坐标系走标准 WGS84，**GCJ-02 偏移得自己处理。** 想认真用得先解决这两件事。这部分暂无可靠的实测信息，留给实际使用者补充。

写在最后

拿到一个混装了 shp、tif、gpkg、tileset.json 的压缩包，以前要开三四个软件；现在的答案是：**打开一个 30MB 的应用（或者干脆一个网页），全拖进去。**

GeoLibre 真正的价值在于——不在于它比 QGIS 强（它不强），而在于它把「**看一眼数据**」这件事的**成本降低到了接近零**。这个位置以前是空的。

对做多引擎或三维 GIS 开发的人，它还有一层参考价值：**引擎无关的 store 设计**。把状态存成普通的图层记录 and 视图状态，而不是绑死在某个渲染引擎的对象上，就能在不改变工程模型的前提下接入另一个渲染器。这个思路值得借鉴。

使用路径按场景分：

1. **只想看看** —— 直接开 `web.geolibre.app`，不用装不用注册
2. **要真干活** —— GitHub Releases 下桌面版，或走微软商店 / Homebrew / winget，两分钟的事
3. **Python 用户** —— `pip install geolibre`，要 GeoPandas 支持就
`pip install "geolibre[all]"`，需要 Python 3.11+
4. **内网 / 离线环境** —— `VITE_PYODIDE_INDEX_URL` 和 `VITE_DUCKDB_SPATIAL_EXTENSION_PATH` 可以把 Pyodide 和 DuckDB 空间扩展指到内部镜像，**不用重新构建**；官方也有 Docker 镜像 `ghcr.io/opengeos/geolibre:latest`
5. **二次开发** —— npm workspaces 单体仓库，所以要用 **npm**（仓库跟踪 `package-lock.json`）配 **Node 22+**；主应用在 `apps/geolibre-desktop`，MIT 许可

最后还是那句：拿它做数据预览和探索，推荐；用于生产环境，再等等，至少等版本迭代节奏趋于稳定。

<https://geolibre.app/tutorials/source-code-deep-dive-en/>

GeoLibre Source Code Deep Dive: Core Techniques Worth Reusing in WebGIS Projects

Abstract: This article provides an in-depth analysis of the GeoLibre source code architecture, covering dependency selection, the DuckDB-WASM Spatial core engine, web-side performance optimization, state management, offline PWA caching, and cloud-native geospatial data solutions. All source file paths are annotated for developer reference and reuse.

Original source: "GIS开发手记", original link: <https://mp.weixin.qq.com/s/482yiH-VsKP7OmBw9UWG6g>

Preface

This article walks through the GeoLibre repository source code layer by layer, covering dependency selection, the DuckDB-WASM spatial computation kernel, frontend performance optimization strategies, state management design, the three-tier offline caching strategy, and the cloud-native geospatial data solution. All corresponding source file paths are annotated. Every technical approach described can be directly applied to your own WebGIS projects.

1. The Toolbox: Open-Source Libraries Worth Examining Individually

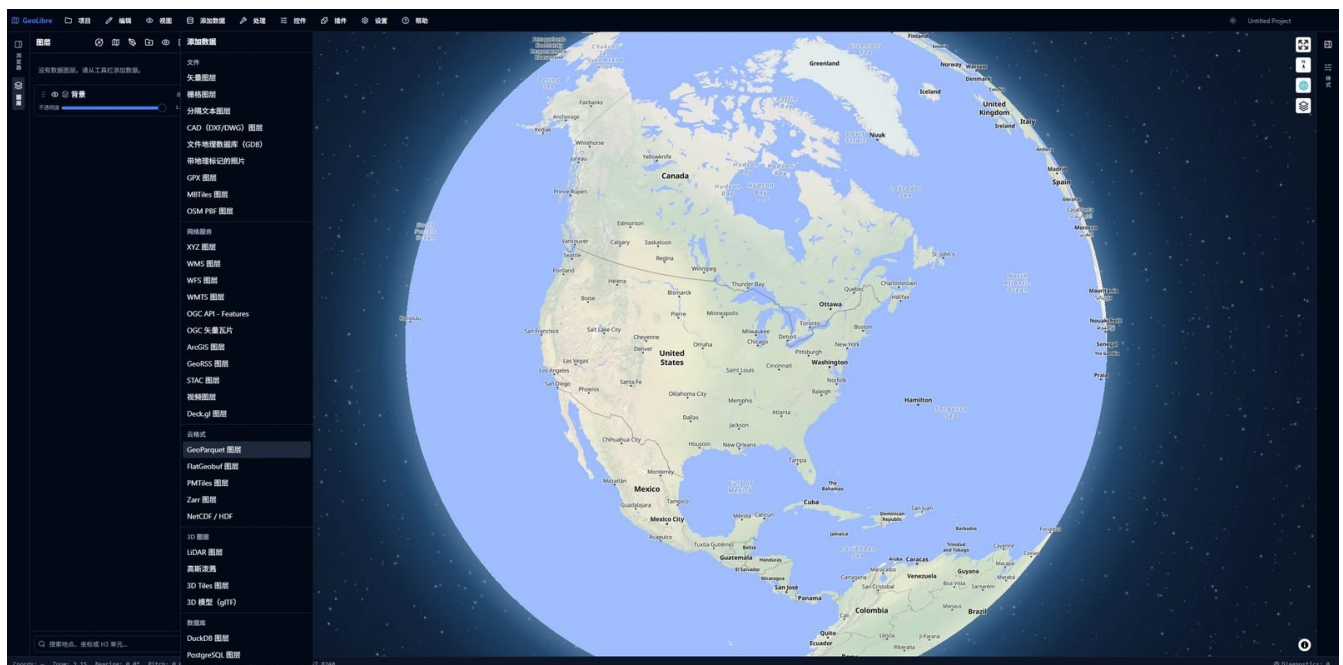
The repository is an npm workspaces monorepo with 7 packages plus a desktop application. Dependencies are categorized by purpose below; the ones marked for attention are libraries worth focused study.

1.1 Format Parsing: Lightweight First, Heavy Artillery as Fallback

Library	Version	Role	Why This One
@duckdb/duckdb-wasm	1.33.1	GeoParquet, FlatGeobuf, GML, DXF, TAB, and spatial SQL	A single library serving as both format driver and compute engine
shpjs	6.2	Shapefile	Pure JS, tens of KB; <code>.prj</code> for projection, <code>.cpg</code> for encoding (Chinese attribute encoding issues resolved)

Library	Version	Role	Why This One
<code>fflate</code>	0.8	zip / kmz / aprx decompression	Extremely small and fast
<code>sql.js</code>	1.14	GeoPackage read and write	GPKG is essentially SQLite — lightweight and efficient
<code>exifr</code>	7.1	Photo EXIF GPS → point layer	Practical for UAV/drone scenarios
<code>gdal3.js</code>	2.8	Only used for georeferenced GeoTIFF/ COG export	WASM ~28MB + data ~12MB, loaded from CDN only, never bundled
<code>geotiff</code>	3.0	GeoTIFF decoding	—
<code>h5wasm / netcdfjs</code>	—	HDF5 / NetCDF-3	Clear division of labor, each handles one format
<code>@osm/pbf @osm-ix/core</code>	—	OSM PBF	Runs in a Web Worker
<code>pmtiles proj4 fast-xml-parser</code>	—	Tile archives / projections / XML	Foundational utilities

The key takeaway here is **"choose the lightest path for each format."** The source file `packages/plugins/package.json` has over 60 dependencies, yet there is no monolithic "unified read layer" — each format takes its own shortest path.



The KML case is particularly illustrative. The DuckDB-WASM section of `docs/architecture.md` states it clearly: KML is handled by a **custom-built parser** to **preserve embedded styling**, outputting `simplestyle-spec` properties (`fill` , `stroke` , `stroke-width`) so that styled KML looks the same in GeoLibre as it does in Google Earth. Only when the custom parser fails does it fall back to DuckDB Spatial, **at the cost of losing style information**.

Key insight: General-purpose libraries inevitably discard format-specific information when unifying data models. And that information is often exactly what users care about most.

The same pattern repeats for Shapefile: **try shpjs first, only fall back to DuckDB Spatial if that fails. A fast path plus a slow-but-comprehensive fallback is a universal pattern for format parsing.**

1.2 Spatial Computation: Four Engine Tiers, Defaulting to the Lightest

This area is easy to misunderstand. Tracing through the source (per the Python sidecar section of `docs/architecture.md`):

Engine	Where It Runs	Positioning
Turf.js (<code>@turf/*</code> , ~25 sub-packages)	Browser, pure JS	Default engine for vector tools , zero dependencies, zero backend
GeoPandas / Shapely	Python sidecar	Upgrade path when projection-aware results are needed
GeoPandas / Shapely	Browser via Pyodide	Same codebase , usable in the web version
DuckDB / PGlite+PostGIS / SedonaDB	Browser or sidecar	Three engines for the SQL Workspace

Note that Turf is imported **per sub-package** (`@turf/buffer` , `@turf/intersect` , etc.), not as a whole. This is important — importing all of Turf at once is substantial; per-package imports are what make it practical.

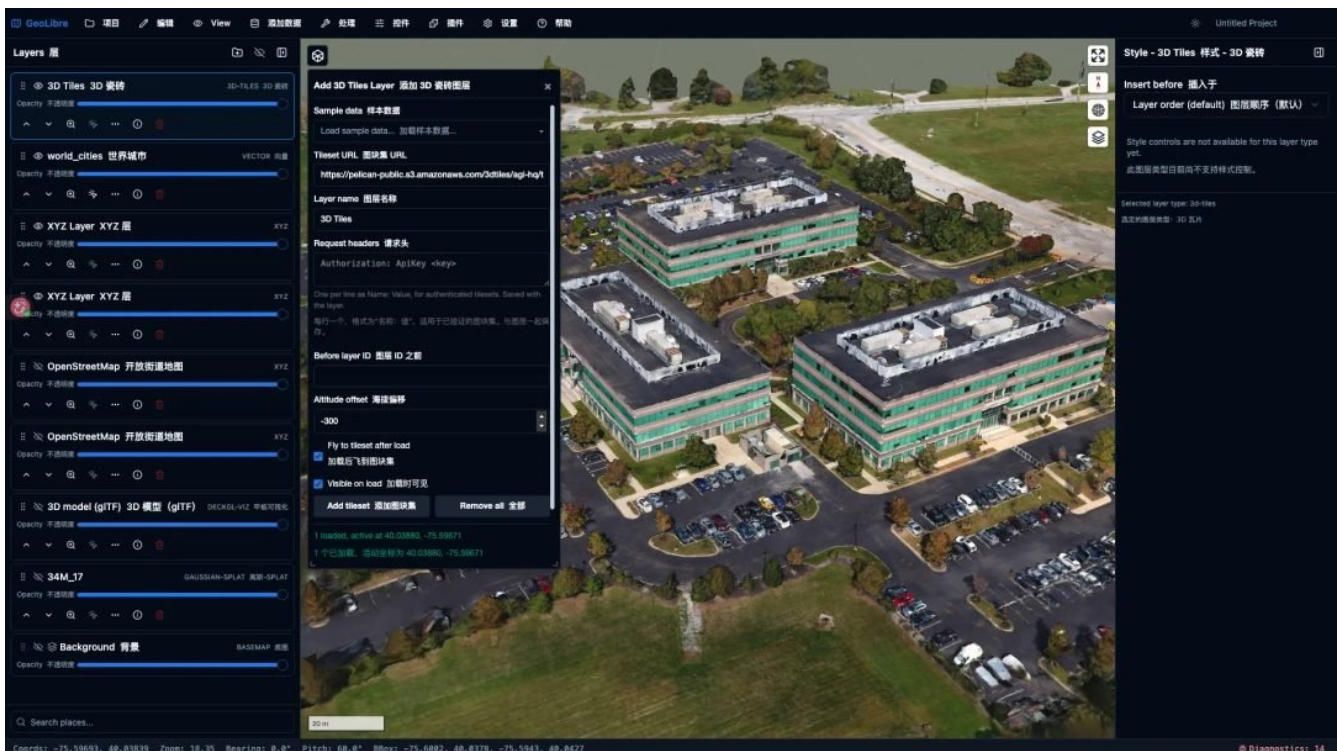
The "one codebase, two runtimes" design is documented in the same section: the geometry logic lives in a **framework-free module** `backend/geolibre_server/geolibre_server/vector_ops.py` . A Vite plugin (`vite-plugins/copy-vector-ops.ts`) copies it into the frontend package, and the browser side uses a classic Web Worker to load Pyodide, install `geopandas` , and call `run_vector_tool` across a JSON string boundary.

Real-world lesson: Anyone who has worked with GIS knows this pitfall — compute a buffer with Turf.js on the frontend, compute the same buffer with PostGIS on the backend, and the areas differ by 0.3%. Two days later you discover it's because the default segment count differs between the two implementations.

1.3 Rendering & Layers: The MapLibre Plugin Ecosystem

Library	Role
<code>maplibre-gl</code> 6.11	Primary map

Library	Role
deck.gl 9.4 (core/layers/geo-layers/mesh-layers/aggregation-layers/mapbox)	COG, 3D Tiles, I3S, visualization layers, interleaved into the MapLibre canvas
maplibre-gl-3d-tiles / -lidar / -splat / -raster / -vector	Without switching engines, directly add 3D Tiles, point clouds, and Gaussian splats onto MapLibre
@developmentseed/deck.gl-geotiff / -raster	COG rendering
@carbonplan/zarr-layer	Zarr scientific data
@loaders.gl/i3s / @esri/maplibre-arcgis	Esri ecosystem integration
@geoman-io/maplibre-geoman-free	Drawing and editing
maplibre-gl-time-slider / -swipe / -layer-control / -basemap-control	Interactive controls
@tanstack/react-virtual	Attribute table virtualization
cesium 1.146	Optional 3D globe split-view, lazy-loaded ~4.7 MB in a separate chunk



Many developers don't realize this about the `maplibre-gl-*` family: 3D Tiles, COPC point clouds, and Gaussian splats can all be loaded directly within the MapLibre canvas — no Cesium required. If your project just needs "a quick look at some 3D Tiles," this path is far lighter than pulling in an entire 3D engine.

Key Takeaways

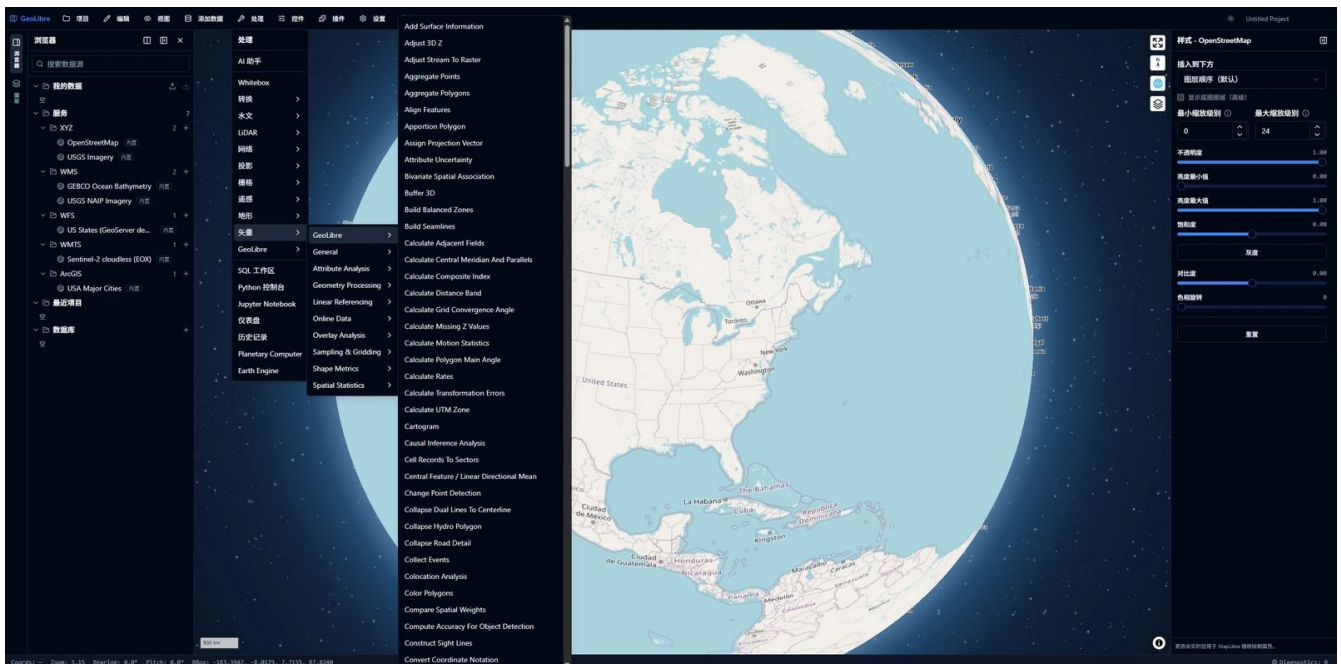
- **1. Choose the lightest path per format.** The size difference between frontend libraries is orders of magnitude (shpjs at tens of KB vs. gdal3.js at 40 MB). The cost of library size is paid by users in first-screen load time.

- **2. Fast path + comprehensive fallback:** 90% of normal data goes through lightweight parsing; the 10% of edge cases fall back to the heavy engine.
- **3. Fidelity over generality.** Building a custom parser to preserve KML styling is worth the effort.
- **4. Import Turf by sub-package,** never as a whole.

2. DuckDB-WASM Spatial: One Library as Both Format Driver and Compute Engine

This section is the core of this article. **If you only read one thing, read this.**

First, let's cover WebAssembly in one sentence: the identifier `wasm` turns up well over a thousand times in the GeoLibre source. **Database, language runtime, native toolchain, codec, and machine learning** — all five capability categories are powered by WASM engines: DuckDB, sql.js (SQLite), PGLite + PostGIS, CereusDB (SedonaDB), Pyodide, `geolibre-wasm` (the Whitebox engine compiled to WASI, plus GeoLibre's own Rust tools), `gdal3.js`, `cog-tiler-wasm`, `h5wasm`, `onnxruntime-web`. **When you trace the native languages, the picture becomes clear: C, C++, Rust. WASM here is not about accelerating JavaScript — it's about bringing decades of accumulated native GIS ecosystem into the browser wholesale.**



Among these dozen or so engines, **one stands out as worth trying right now:** `@duckdb/duckdb-wasm` plus its spatial extension. The reason is simple — other engines solve "a particular category of work," but this one is simultaneously a **format driver** and a **compute engine**. One library can take over both "read data" and "compute on data."

2.1 Clarifying the Three-Layer Relationship First

These three names are often conflated. They are nested:

Layer	What It Is	Relationship
WebAssembly	Binary instruction format in the browser	Just "the ability to run native code"; does nothing GIS-related on its own
DuckDB-WASM	DuckDB (an analytical database written in C++) compiled to WASM	It is "DuckDB implemented via WASM," not WASM itself
Spatial Extension	DuckDB's spatial extension, also a separate <code>.wasm</code> file	Requires a separate <code>INSTALL / LOAD</code> ; without it, there are no spatial functions at all

The most common misconception: thinking that installing duckdb-wasm gives you spatial capabilities. It does not. `ST_Read`, `ST_Transform`, `ST_AsWKB` — these all live in the spatial extension, which is a second WASM artifact fetched from CDN and loaded at runtime.

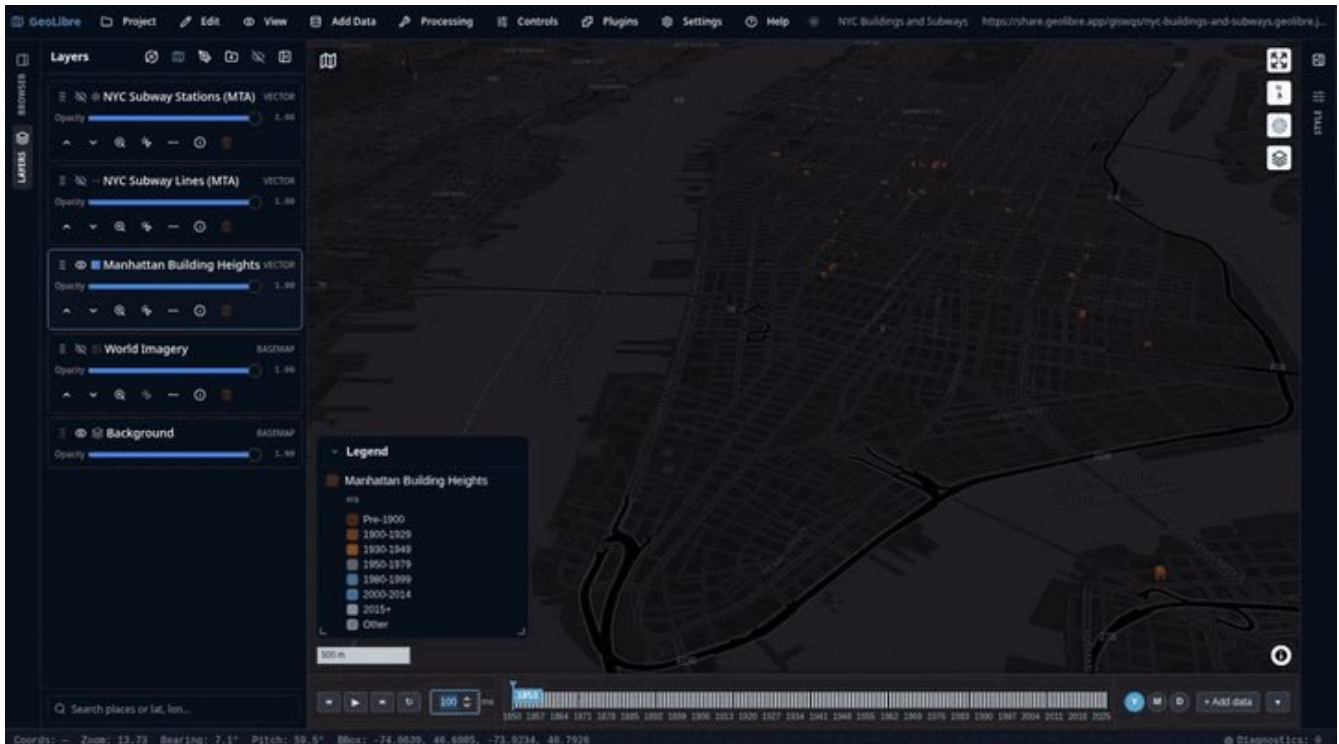
And behind the spatial extension's `ST_Read` lies **a subset of GDAL**. The implication: issue a single SQL statement in the browser, and you can read the vector formats that subset covers. It is the extension's own bundled GDAL, not whatever GDAL is installed on the machine, so the exact list is whatever the loaded build ships — run `SELECT * FROM ST_Drivers()` to see it.

2.2 What GeoLibre Uses It For

DuckDB invocation points in the source code are categorized as follows:

Use Case	Interface Used	Notes
GeoParquet	<code>read_parquet</code>	Both local and remote go through this; remote uses HTTP Range requests
FlatGeobuf / GML / DXF / TAB, etc.	<code>ST_Read</code>	GDAL backend; format coverage depends on what the extension loads
Shapefile (zip)	First <code>shpjs</code> , fall back to Spatial	Lightweight first, heavy weapon as fallback
KML	First custom parser (preserve styling), fall back to Spatial	Spatial only gets geometry; styling is lost
CSV with WKT geometry columns	DuckDB SQL	Converts text geometry directly into a layer
Coordinate system transformation	<code>ST_Transform</code>	GeoJSON with legacy top-level <code>crs</code> members are reprojected through this
SQL Workspace	Full DuckDB SQL	Loaded layers are registered as tables and can be directly JOINed

Key insight: Pay attention to the last row. This is an easily underestimated design choice: layers on the map are simultaneously tables in SQL. Users can write `JOIN` and `ST_Intersects` across two layers, and the result becomes a new layer directly. **"Map" and "database" are two views of the same thing.**



The SQL Workspace also features three "help the user say what they mean" rewrites. If you're building a SQL console, you can follow this approach directly:

- **Bare URLs or local paths after FROM / JOIN just work** — the source code automatically wraps them with the appropriate reader (`read_parquet`, `read_csv_auto`, `ST_Read`, etc.) based on the file extension. Users don't need to memorize which format maps to which function.
- `s3://`, `gs://`, `az://` are translated to their corresponding public HTTPS endpoints, traveling over the same HTTP Range channel.
- **HTTP URLs in reader parameters are rewritten into DuckDB file handles**, with the JS side issuing Range requests rather than going through WASM's built-in httpfs — because the latter fails outright on many servers.

Implementation detail: These three rewrites apply only to **actual reader call arguments**. Strings that look like URLs inside string literals and comments are skipped (the source code implements a full SQL literal masking pass). **Never use raw regex to rewrite SQL.**

2.3 Four Implementation Details Worth Borrowing

First, select bundles based on browser capabilities. duckdb-wasm ships multiple WASM builds (mvp, eh, etc.) with different capabilities and sizes. `duckdb-wasm-bundles.ts` does exactly one thing: passes the mvp and eh artifacts — obtained via Vite's `?url` — to `duckdb.selectBundle()` for runtime selection:

```
const MANUAL_BUNDLES: duckdb.DuckDBBundles = {
  mvp: { mainModule: duckdbWasmMvp, mainWorker: mvpWorker },
  eh: { mainModule: duckdbWasmEh, mainWorker: ehWorker },
};
export async function selectDuckDbBundle() {
  // Imported here so DuckDB-WASM's JS stays off the startup path
  const { selectBundle } = await import("@duckdb/duckdb-wasm");
  return selectBundle(MANUAL_BUNDLES);
}
```

Manually listing bundles rather than using the default CDN resolution ensures Vite produces content-hashed local artifacts for the WASM and worker — which in turn enables safe caching by the Service Worker's CacheFirst strategy (see Section 5). (The `npm run lite:build` variant swaps in `duckdb-wasm-bundles.cdn.ts`, which loads the same pinned version from jsDelivr for hosts with a per-file size cap.)

Second, extension loading must be "per-instance, run-once." `INSTALL spatial` goes over the network, and `LOAD` has state; concurrent calls will conflict. GeoLibre's approach is worth studying:

```
// Memoized per-instance: the shared library and the SQL Workspace
// each have their own DB, and each tracks its own loading state
const spatialExtensionByDb = new WeakMap<duckdb.AsyncDuckDB, Promise<void>>();

export async function ensureSpatialExtension(db, connection, beforeLoad?) {
  let promise = spatialExtensionByDb.get(db);
  if (!promise) {
    promise = (async () => { /* ...INSTALL / LOAD... */ })();
    spatialExtensionByDb.set(db, promise);
  }
  try {
    await promise;
  } catch (error) {
    // Only clear if the memoized entry still points to this failed attempt,
    // otherwise retries would be permanently impossible
    if (spatialExtensionByDb.get(db) === promise) spatialExtensionByDb.delete(db);
    throw error;
  }
}
```

This short piece of code embodies three design decisions. It caches the Promise, not a boolean — concurrent calls share the same loading operation. It uses `WeakMap` keyed by instance — not a module-level singleton; when an instance is rebuilt, the cache automatically invalidates. On failure it clears the memoized entry, but only after verifying identity — preventing it from wiping out a new loading attempt someone else just wrote. **Miss any one of these three, and you'll encounter "spatial function sporadically does not exist" in production.**

Third, leave an escape hatch for loading extensions without network access. `INSTALL spatial` by default downloads from DuckDB's extension repository. GeoLibre adds an environment variable `VITE_DUCKDB_SPATIAL_EXTENSION_PATH`: when set, it directly issues `LOAD '<local path>'`, skipping `INSTALL` entirely.

For intranet, government, and offline desktop deployments, this single configuration setting is the difference between working and not working. The same pattern is reused for the h3 community extension (`INSTALL h3 FROM community`) — DuckDB's extension ecosystem is usable in the browser, and many developers don't know this.

Fourth, manage Workers and lifecycles yourself. DuckDB runs in a dedicated Worker. **The WASM heap is tied to the Worker's lifetime; only `terminate()` truly returns it to the system.** GeoLibre implements reference counting for the SQL Workspace instance: if queries are still running, it marks the instance for "destroy when idle" and waits until the last query releases before shutting down. **This is essential in scenarios requiring instance recreation — a direct `terminate()` would kill in-flight queries.**

2.4 A Real-World Bug Worth More Than Any Tutorial

A noteworthy record found in source code comments, **which illustrates "what it's like to run a database in a browser" better than everything above:**

duckdb-wasm 1.33.1-dev45 will **permanently destroy** an instance's remote `read_parquet` — if that instance executes `LOAD spatial` **before completing its first remote read**. Afterward, all remote Parquet reads fail with `stoi: no conversion`, and this **cannot be recovered in-place**.

GeoLibre counters this with a two-pronged approach:

First, warming up. The `beforeLoad` hook on `ensureSpatialExtension` is exactly for this — before `LOAD spatial`, it runs `SELECT 1 FROM read_parquet(...) LIMIT 0` against whatever remote reader this query uses. `LIMIT 0` **only fetches the Parquet footer, incurring almost no additional traffic**. If this particular query has no remote Parquet, it falls back to reading a small public sample file.

Second, rebuilding. If the bug still strikes (e.g., the warm-up failed previously), `runSqlQuery` catches `stoi: no conversion`, calls `resetSqlDatabase(db)` to replace the entire instance, and then **retries once**. The rebuild goes through the warm-up again, so the second attempt is clean.

Two additional defensive details: the retry is triggered only when the statement **actually contains a remote reader call** (URLs appearing inside string literals don't count), and `resetSqlDatabase` confirms that "the instance to replace is still the current instance" before acting.

Three Transportable Lessons

- **WASM extension loading order has side effects.** Do not assume "it doesn't matter when you LOAD," especially for extensions involving the network subsystem.
- **Treat "warm-up" and "rebuild" as first-class citizens, not patches.** Graceful recovery from irreversible bad state is only possible if the instance is designed to be replaceable.
- **Leave an escape hatch for upstream bugs, but narrow the scope to the symptom.** Match the specific error message + specific conditions + retry only once — don't blindly catch-all and retry.

2.5 Where Its Boundaries Lie

Many articles praise DuckDB-WASM. This section specifically examines the actual limitations revealed in the source code. Understanding these constraints provides more confidence than operating blindly.

Limitation	Value	Reason
Remote file size	2 GiB	DuckDB-WASM's HTTP filesystem stores remote file sizes in 32 bits
Single-tab memory	~4 GiB	WASM address space ceiling
Feature count confirmation threshold	500,000	User must confirm before materializing beyond this
Threading	Single-threaded	Very large data should go back to the server

The 2 GiB limit has an instructive detail: the guard check happens **before ingestion begins** (`_registerSource` runs ahead of the streaming branch, so "streaming reads" can't bypass it either). GeoLibre's approach is to communicate this clearly in the file browser panel rather than letting users click Add and wait for an inevitable failure. **Failures that can be determined in advance should never be deferred to runtime.**

There's also the **copy vs. stream** trade-off: only GeoParquet truly supports "query in place without copying." Other formats silently fall back to full copy even when the stream option is provided — so GeoLibre only shows this button for GeoParquet. **A button whose click makes no difference is worse than no button at all.** In copy mode, memory usage roughly tracks the **decompressed** data size, which can be several times the Parquet disk size. Beyond 100 MB, the UI prompts the user to switch to streaming.

One final instructive comparison: GeoLibre's desktop build exposes a `native-duckdb` compile-time switch to use native DuckDB instead. But the Rust side carries a hard constraint:

```
compile_error!("the `mas` (Mac App Store) build must not enable `native-duckdb`: \
  DuckDB loads its spatial extension as unsigned native code at runtime, \
  which App Sandbox and App Store guideline 2.5.2 forbid.");
```

The native version is faster, but its runtime loading of unsigned native extensions directly violates Mac App Store rules. Conversely, **the WASM version, because it runs inside a sandbox, is the one that "ships everywhere."** This is a consideration rarely factored into technology selection.

2.6 Key Takeaways

- **1. Try DuckDB-WASM Spatial first, then consider alternatives.** One library solving both "read formats" and "compute spatially" is the highest ROI step for web-side GIS today. Remote GeoParquet + `read_parquet` — a dozen lines of code, no backend required.
- **2. Extension loading as "per-instance memoized Promise + identity-verified cache clearing on failure."** This pattern applies to all "async one-time initialization" scenarios, not just DuckDB.
- **3. Always provide a local extension path escape hatch.** Intranet / offline / desktop — this single configuration setting is everything.
- **4. For every WASM engine: lazy load + separate chunk + Worker execution + pinned version + replaceable lifecycle.** Keep it off the main thread and out of the critical rendering path.
- **5. Codify hard limits as named constants and validate upfront.** 2 GiB, 500K features, ~4 GiB heap — failures that can be communicated to the user in advance should never be deferred to runtime.

3. Web-Side Performance: Five Optimizations, Plus One Honest Negative Result

All numbers in this section are traceable to specific file line numbers.

3.1 Threshold-Driven: Set Clear Numbers, Switch Implementation at the Boundary

The core idea in one sentence: **Don't pursue one scheme to rule them all — switch implementations at well-defined thresholds.**

Threshold constants in the source (all verifiable):

Constant	Value	Location	What It Protects
<code>LARGE_VECTOR_FEATURE_THRESHOLD</code>	50,000	<code>core/src/types.ts:783</code>	Main-thread GeoJSON parsing
<code>maxHistoryFeatureCount</code>	500,000	<code>core/src/history.ts:29</code>	Undo stack memory
<code>DUCKDB_VECTOR_FEATURE_WARN_COUNT</code>	100,000	<code>core/src/types.ts:2540</code>	Result materialization memory
<code>MAX_CEREUS_FEATURES</code>	50,000	<code>lib/sedona-workspace.ts:25</code>	WASM heap

Constant	Value	Location	What It Protects
<code>MAX_DERIVED_FEATURES</code>	50,000	<code>map/src/derived-geometry.ts:42</code>	Derived geometry computation
<code>historyCoalesceMs</code>	400 ms	<code>core/src/history.ts:6</code>	Undo record explosion
Remote files	2 GiB	<code>plugins/remote-file-formats.ts</code>	DuckDB-WASM 32-bit

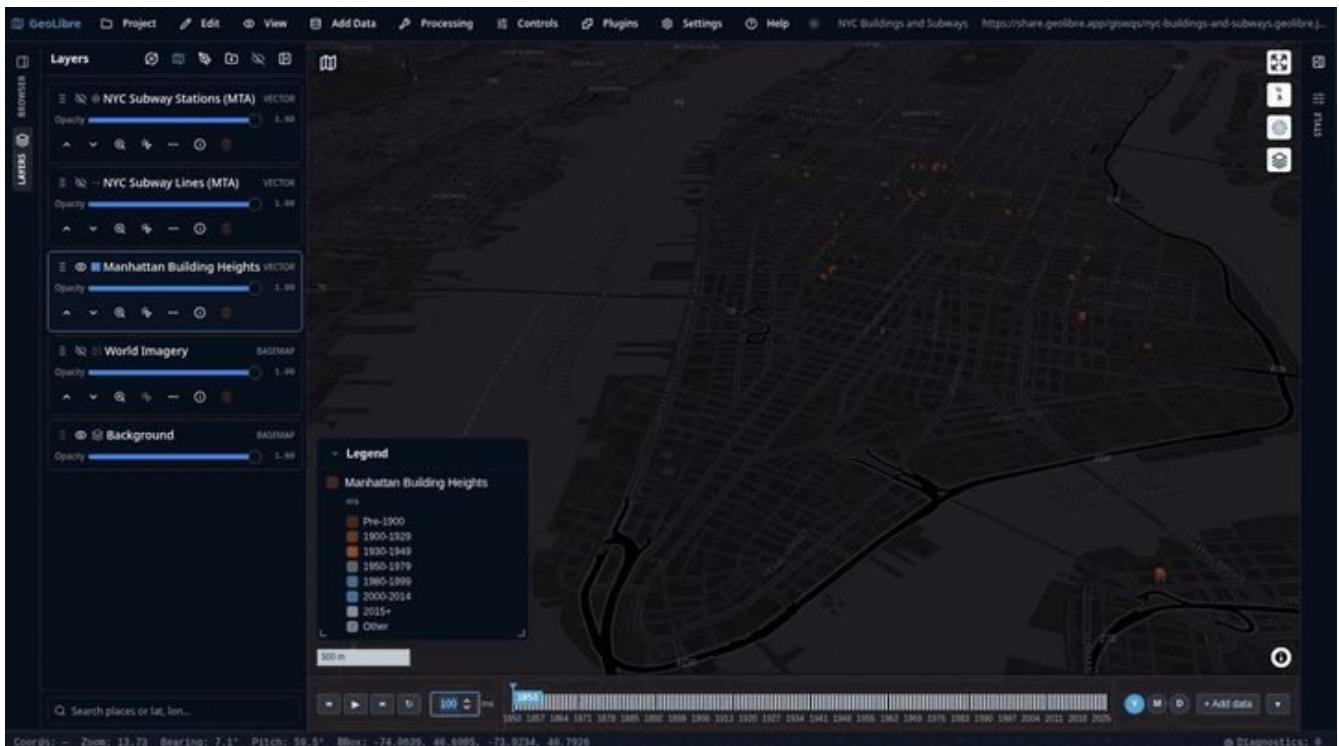
Note that these thresholds are all named constants, all have documentation comments, and both `historyCoalesceMs` and `maxHistoryFeatureCount` have setters for runtime adjustment (set to 0 in tests). This is a tier above magic numbers scattered through the code.

Takeaway: Every threshold should be able to articulate "what it protects." A threshold that can't justify its protective purpose lacks a reliable foundation and is unlikely to survive long-term maintenance.

3.2 Client-Side Tiling: On-the-Fly Tile Generation Above 50K Features

The pipeline lives in `packages/map/src/geojson-vt-protocol.ts` (protocol and registry) and `geojson-vt-index.ts` (index building and encoding), step by step:

- Indexing uses `@maplibre/geojson-vt` (note: this is MapLibre's fork; **Supercluster is included in this package** — the comments say it's "the same engine MapLibre uses internally")
- Point layers use `Supercluster` for indexing; everything else uses `GeoJSONVT`
- Encoding uses `@maplibre/vt-pbf`'s `fromGeojsonVt`
- Fed to MapLibre via a custom protocol `geolibre-gjvt`
- `TILE_EXTENT = 4096`, `TILE_MAX_ZOOM = 16` (beyond this, let MapLibre over-zoom)
- Index building and tile encoding run on a Web Worker (`geojson-vt.worker.ts`), so a large load no longer freezes the main thread; where no Worker is available, the same code runs inline



Two details worth highlighting individually.

First, the per-layer tile registry lives in a module-level Map, not in the store. The source comment puts it this way:

Keyed by layer id. Module-level rather than on the Zustand record because **tile indexes are large, non-serializable objects that must not enter app state or be written to .geolibre.json.**

This provides an excellent heuristic: what belongs in global state? — only what can be written into a project file. Derived artifacts (tile indexes, spatial indexes, WebGL buffers, decoded bitmaps) should always live in module-level caches or on the engine side.

Second, check the abort signal before encoding. One line:

```
// packages/map/src/geojson-vt-protocol.ts
if (signal?.aborted) return empty;
```

MapLibre cancels tile requests that scroll off-screen, and the result would be discarded anyway — so don't waste the computation.

Generalized: every "on-demand production" async pipeline must support cancellation. Applied to real projects: reprojecting off-screen tiles while the user rapidly drags the map, queries from previous pages still running while the attribute table rapidly flips pages, a previous layer's parse result arriving late and overwriting the new layer during rapid layer switching (**classic race condition**).

`AbortController` **Is Not Just for** `fetch`

Any loop that spans more than one frame should check `signal.aborted` in the loop body and exit early.

3.3 Undo Stack: Three Refinements Worth Studying

The file `packages/core/src/history.ts` is worth opening directly — under two hundred lines, high density. It uses `zundo` (Zustand's time-travel middleware).

The problem: **Every snapshot holds the full GeoJSON of layers.** Repeated editing pins multiple copies in memory (the comments reference issue #341).

Treatment 1: Soft budget measured by feature count. The comments explain why not bytes: "Feature count is a cheap proxy for payload size; it avoids serializing geometry on every edit" — **serializing geometry just to measure memory is itself a performance problem.**

Treatment 2: Always retain the newest snapshot:

```
// trimHistoryBySize: walk from newest to oldest, stop when the accumulated budget is exceeded
// The newest snapshot is always retained, even if it alone exceeds the budget.
let total = distinctFeatureCount(pastStates[lastIndex], seen);
```

"When memory is tight, clear the undo stack" is many projects' approach — but that means the user, at the very moment they most need undo (right after a large edit), has exactly no undo available.

Treatment 3: Deduplicate by object reference. `distinctFeatureCount` uses a `Set` to track seen payloads: unchanged layers share the same reference across snapshots and are counted only once. **So "keeping many small-layer snapshots" has almost no additional memory cost.**

One more easily overlooked detail: that 400 ms is not ordinary debouncing — it's **leading-edge debounce**, serving as `zundo`'s `handleSet` :

firing only on the leading edge records the pre-burst state once

The distinction is critical — **what needs to be recorded is "the state before this burst of consecutive operations,"** so it must be recorded on the first trigger, not after things settle. **The value before you started dragging the slider is the value you want to undo back to.** Using ordinary trailing debounce would capture some intermediate value during the drag.

3.4 Virtualization Only Governs Rendering

The attribute table uses `@tanstack/react-virtual` for virtualization, but **sorting, filtering, and selection operate on the full data model.**

This sounds modest, but it's a high-frequency bug: many virtual list implementations "sort only the rendered rows," leading to sort results that change with scroll position, or "select all" that only selects visible rows. **And these bugs are impossible to reproduce in small-dataset testing.**

3.5 Bundle Size: How Far Does Lazy Loading Go?

The code-splitting configuration in `apps/geolibre-desktop/vite.config.ts` (`manualChunks` plus the Rolldown `CODE_SPLITTING_GROUPS`) is the core of this approach. Take the most representative example — Cesium. The source comments record that when Cesium's chunk once swallowed shared helpers, "the whole ~4.7 MB Cesium chunk was modulepreloaded on every boot"; it now sits in its own group because **"Cesium only loads when a pane switches to the 3D globe."**

Keeping it off the boot path is the key point. For web GIS, this is the easiest performance win: the users who actually need 3D may only be 20%; there is no reason for 100% of users to pay the first-screen cost for it.

A detailed size breakdown from the source comments, organized here:

Heavy Resource	Size	Strategy
CesiumJS	~4.7 MB	Separate chunk, <code>import()</code> only when switching to globe view
PGlite + PostGIS	~25 MB (bundling into desktop would add ~22 MB of nearly incompressible size)	Default via jsDelivr CDN, not included in the build
gdal3.js	WASM ~28 MB + data ~12 MB	Never bundled , always from CDN; disabling CDN disables the feature
Pyodide / CereusDB	Tens of MB each	CDN-loaded; PGlite and CereusDB can be bundled-in via build switches

So that "only 30 MB installer" figure is achieved by "almost no heavy engine is bundled." The philosophy worth borrowing: the default location for heavy engines should be "downloaded the first time the user clicks on it," not "included in the package just in case."

A companion practice: **all external resource URLs are runtime-configurable.**

`VITE_PYODIDE_INDEX_URL`, `VITE_DUCKDB_SPATIAL_EXTENSION_PATH`, `VITE_CESIUM_TOKEN` can all be pointed to internal mirrors at runtime — **no rebuild required.**

For intranet/offline delivery scenarios, this is a lifesaving design. Hard-coded CDN URLs in the bundle mean an emergency rebuild when the network is down on site.

One more easily forgotten detail: `packages/processing/src/ort.ts` notes that onnxruntime-web's WASM artifacts are loaded from CDN, and **the version on CDN must exactly match the version pinned in npm** — otherwise it's a runtime crash. **For any library following the "JS glue in the package, WASM artifacts on CDN" pattern, this is the trap. The fix is to encode the version as a single constant consumed by both sides.**

Bottom line: CDN URLs, WASM versions, mirror paths — hard-code any of these three, and what awaits is an emergency packaging session on some Friday evening.

3.6 One Honest Negative Result

The final section of `docs/architecture.md` is literally titled "Performance: map rendering on Linux (WebKitGTK)." The details are worth reading in full:

- Empty map at any zoom level: steady 60 FPS
- Once there is a tile layer (vector **or** raster XYZ): **FPS drops to single digits during tile loading**, immediately returning to 60 once loading stops
- Root cause: WebKitGTK performs GPU uploads for each new tile on the main thread (textures for raster, vertex buffers for vectors), via synchronous WebGL calls, plus subsequent fade-in repaints
- **A single tile-integration render cycle measures ~125 ms; Chromium takes only a few ms**
- **Explicitly ruled out:** vector tile parsing and bucket construction run in MapLibre's Worker — they are not the bottleneck

The exclusion list is exhaustively detailed: software rendering (confirmed using Intel i915 GPU), GPU saturation (rendering engine still ~20% idle), Tauri IPC file reads (22 MB GeoJSON ~126 ms), `JSON.parse` (~36 ms), KWin compositor latency, `renderWorldCopies`, globe vs. Mercator projection, `preserveDrawingBuffer`.

The document even provides a one-line FPS counter for reproduction, plus three **not-yet-implemented** mitigations (increase `maxTileCacheSize`, 512px raster tiles, `fadeDuration: 0` — with a note to gate them on WebKitGTK only so they don't penalize Chromium builds).

Two lessons:

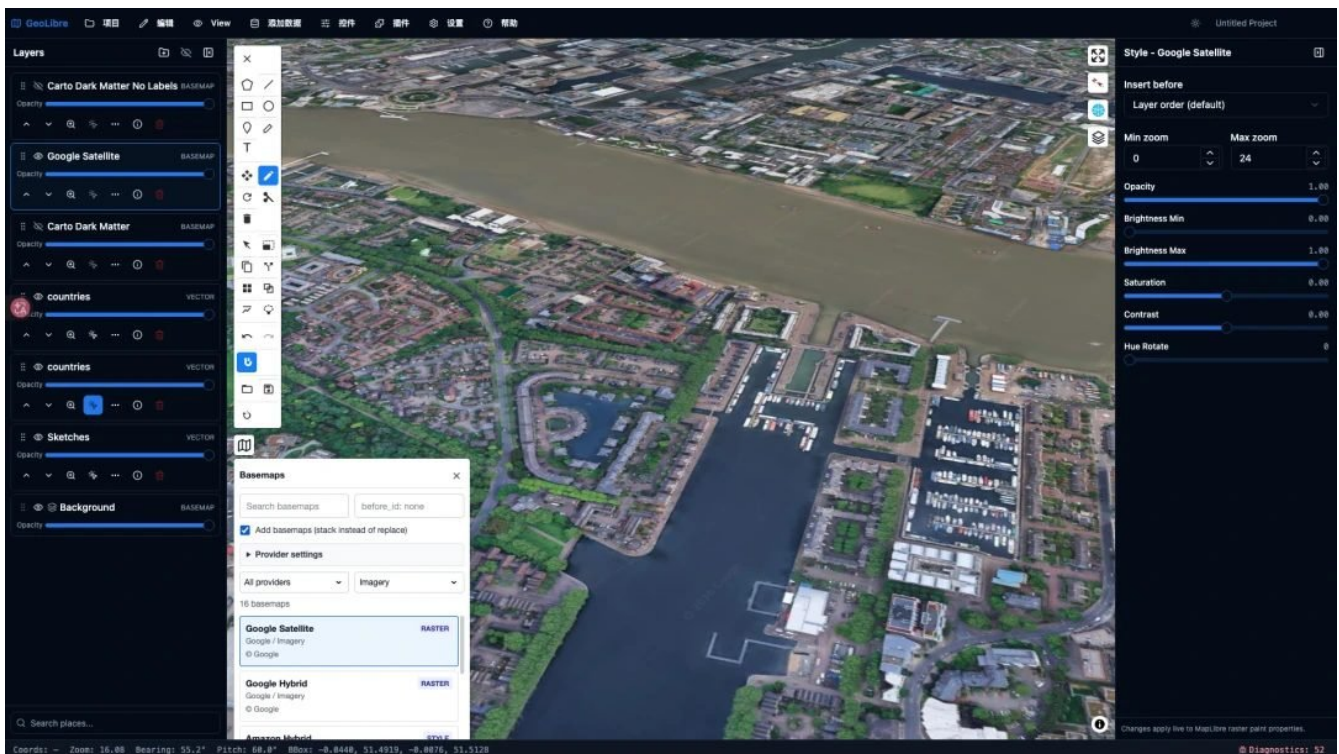
- **Cross-WebView performance does not extrapolate.** 60 FPS measured in Chrome does not guarantee the same in Electron's older Chromium, domestic browser kernels, or WKWebView. **Any project with client-side delivery requirements must test on the target WebView.**
- **Documenting the investigation process along with "what was ruled out" is more valuable than only stating the conclusion.** That 8-item exclusion list can save the next person two days.

Editor's note: Being willing to write this kind of negative result into the architecture documentation, complete with reproduction steps and unimplemented TODOs, speaks well of the project's engineering maturity.

4. State Management: Three Directly Reusable Patterns

This section is pure web engineering, less GIS-specific, but has the lowest migration cost.

@geolibre/core has only four dependencies: zustand, zundo, uuid, @maplibre/maplibre-gl-style-spec. **Note that it does not depend on maplibre-gl itself** (the style-spec package is only used to evaluate style expressions), so the state layer and the rendering layer are genuinely separated.



Pattern 1: Constant array as single source of truth. The 20 layer types at packages/core/src/types.ts:63:

```
export const LAYER_TYPES = [
  "geojson", "raster", "wms", "wmts", "xyz", "vector-tiles", "arcgis",
  "pmtiles", "mbtiles", "zarr", "lidar", "gaussian-splat", "3d-tiles",
  "cog", "flatgeobuf", "geoparquet", "duckdb-query", "deckgl-viz",
  "video", "image",
] as const;
export type LayerType = (typeof LAYER_TYPES)[number];
```

The comments explain the rationale: **"as a runtime list so untrusted input (an imported Layer Library bundle, a hand-edited project) can be validated against it,"** and LayerType is derived from the array, **"so the two cannot drift."**

Takeaway: A single definition simultaneously provides runtime validation data and compile-time types, while preventing the two from drifting apart. Every TypeScript project with a "finite enum + need to validate external input" scenario should follow this pattern.

Pattern 2: State is a flat record + minimal view state. MapViewState has only five fields:

```
export interface MapViewState {
  center: [number, number];
  zoom: number; bearing: number; pitch: number;
```

```
bbox?: [number, number, number, number];
}
```

This minimalism is deliberate. The smaller the state, the greater the confidence in serializing it into a project file, and the lower the cost of connecting a second consumer (a second map panel, a second renderer, an exporter).

Pattern 3: Cross-language boundaries marked with SYNC: . This was the most surprising discovery from reading through the source.

The 17 vector file extensions exist as `VECTOR_FILE_DIALOG_EXTENSIONS` in TypeScript (`lib/file-io/paths.ts:39`) and as `RESTORABLE_VECTOR_EXTENSIONS: [&str; 17]` in Rust (`src-tauri/src/lib.rs:800`) — **two copies, because constants can't be shared across languages**. Their solution is to annotate both sides. Quoted verbatim from `lib/file-io/paths.ts:35-38`:

SYNC: RESTORABLE_VECTOR_EXTENSIONS in src-tauri/src/lib.rs must list the same extensions, or a format added here would be rejected by the Rust restore guard on every project reopen (**the bug this PR fixes**). Grep "SYNC:" to find the partner list.

Note the parenthetical "the bug this PR fixes" — it refers to the pull request that added the comment, so it will not mean much to a future reader, but the failure it records is real: a new format was added in TypeScript, the Rust-side guard was not updated, and every project reopen was rejected.

Takeaway: When you can consolidate into one source, do so. When you can't (cross-language, cross-process, cross-repository), use a unified, grep-able marker to pin them together, and document in the comment exactly what happens if they drift. This is far more effective than "everyone please remember to keep these in sync."

5. Offline Capability: Workbox Three-Tier Caching Strategy

This section stands alone because it's especially relevant for intranet, offline, and government deployment scenarios, and the "Offline support (PWA)" section of `docs/architecture.md` documents it comprehensively.

The web build is an installable PWA using `vite-plugin-pwa` + Workbox. Caching is **deliberately split into three tiers**:

Tier	Strategy	Content	Why This Split
Precache	Precache	HTML + JS/CSS chunks essential for map startup	After first visit, the shell works without network ; heavy chunks are excluded to avoid massive first-screen downloads

Tier	Strategy	Content	Why This Split
Same-origin runtime cache	CacheFirst	Content-hashed artifacts under <code>/assets/</code> : MapLibre, DuckDB-WASM and its spatial extension , plugin chunks	Content-hashed filenames make CacheFirst safe — redeployment generates new URLs, old entries won't be served as new
CDN engine cache	CacheFirst (separate rule <code>geolibre-cdn-engines</code>)	Pyodide, PGLite/PostGIS, CereusDB, gdal3.js on jsDelivr	URLs embed exact version numbers, similarly preventing stale serving; jsDelivr's CORS headers make these properly verifiable and evictable 200 responses, not opaque

So the accurate description for these CDN engines is: in the web PWA, the network is required for the first *successful* fetch — CacheFirst only serves from cache once a matching response has actually been stored — and they are available offline from then on. Desktop builds install no Service Worker at all, so this does not apply to them; see the bullets below.

Why not just bundle everything? The following details make it clear.

Several particularly pragmatic details:

- To eliminate the "network required on first use," you can set `GEOLIBRE_PGLITE_CDN=0` and `GEOLIBRE_CEREUS_CDN=0` to bundle them into `/assets/` — **at the cost of PGLite alone adding back ~22 MB to the Tauri binary**
- **Pyodide and gdal3.js have no such switch.** Pyodide always loads from CDN (`VITE_PYODIDE_INDEX_URL` can change the mirror, but **that mirror is not within the scope of the two CacheFirst rules**; unless you place it under `/assets/`, it falls back to ordinary HTTP caching). `gdal3.js` is never vendored; `GEOLIBRE_GDAL_CDN=0` disables **that export feature**, it does not bundle it
- **To strip all GeoLibre-controlled external CDN references** (for deployments that cannot reference any untrusted CDN), set `GEOLIBRE_NO_EXTERNAL_CDN=1`. This implies all `*_CDN=0` flags and additionally disables features that embed CDN URLs in their source code (storymap HTML export, built-in detection models, ONNX WASM, 3D Tiles decoder paths, and the Pyodide default URL). Third-party npm packages may still contain internal CDN URL strings that are not under GeoLibre's control
- **Desktop builds do not install a Service Worker** (Tauri's own resources are already offline), so desktop installations re-fetch CDN engines on every install
- Basemaps only cache CORS-friendly default sources (OpenFreeMap, CARTO); other remote tiles and WMS/WFS are **by design unavailable offline**
- New deployments use `registerType: "autoUpdate" + skipWaiting`, but **deliberately suppress Workbox's default "force reload on activation"** — because under relative-base sub-paths like `/demo/`, it incorrectly triggers and wipes out the user's in-progress map state. Page recovery is handled by `installStaleChunkReload`, which **only reloads when an orphaned lazy chunk 404s**, with cooldown protection

The Most Valuable Detail

This last detail is the most valuable in the entire section — the kind of code "only written after experiencing real production issues." Auto-update causing users to lose unsaved work is the most common and most damaging PWA mistake.

Key Takeaways

- **1. Split caching into three tiers** — "shell / same-origin heavy resources / CDN engines" — don't use one rule for everything.
- **2. Content hashes or version numbers are the prerequisite for CacheFirst to be safe.** Without them, CacheFirst is "permanently serve the old version."
- **3. Explicitly document "what is definitely unavailable offline"** (remote services, non-allowlisted basemaps) — don't make users guess.
- **4. Auto-update should not force-reload the page.** Only reload on orphaned chunk detection, with cooldown.

6. Cloud-Native Formats: What Gets Replaced Is Not the Frontend Library, but the Entire Distribution Architecture

The previous five sections dealt with code. This section deals with something that affects **architecture and cost**.

Traditional pipeline: **Data → ingest into database → publish via GeoServer / ArcGIS Server → generate tiles → frontend requests services**. Requires maintaining a server, a publishing workflow, a tile cache, plus monitoring, backup, and scaling for all of it.

Cloud-native pipeline: **Data → convert to COG / GeoParquet / PMTiles / FlatGeobuf → drop onto any static storage that responds to Range requests → frontend fetches byte ranges on demand**. The server tier disappears entirely; what remains is just object storage traffic costs.

The key phrase is HTTP Range. What these formats share is not "better compression," but that **each file carries its own layout metadata — a header or footer, tile directory, R-tree index, overviews, row-group statistics** — the client reads a few KB of that metadata, calculates which byte ranges it needs, and issues a `Range` request to fetch them. Static storage is sufficient; no "service" is required.

6.1 What Makes This Good: Five Concrete Benefits

Benefit 1: Download volume converges on "what you need to see," not "how large the file is."

This is the most fundamental shift. Traditional formats (GeoJSON, Shapefile, striped GeoTIFF) **offer no useful partial-range access — even when an index exists it sits in a sidecar file, so the client still has to fetch all of the data**. Cloud-native formats keep that index inside the data file itself, enabling pruning across three dimensions:

Pruning Dimension	Mechanism	Effect
Spatial	FlatGeobuf's R-tree index, PMTiles tile layout	Only fetch features/tiles within the viewport
Resolution	COG pyramids (overviews), PMTiles zoom levels	View the entire country at the coarsest level; don't decode 100 million pixels
Attribute columns	GeoParquet's columnar storage	If you need only 3 columns, read only those 3 columns' bytes; other columns are never touched

The third dimension is GeoParquet's most underappreciated strength. Columnar storage + row-group statistics (min/max) means `WHERE` predicates can be pushed down — DuckDB can determine from statistics alone that an entire row group doesn't satisfy the condition and skip it, without ever downloading those bytes.

Benefit 2: Parse cost drops from "tens of milliseconds blocking the main thread" to near zero.

The hidden cost of text formats is `JSON.parse`. Section 3.6's investigation checklist provides two measured numbers: **a 22 MB GeoJSON takes ~126 ms to read from disk, then ~36 ms more for `JSON.parse`** — all on the main thread.

Binary columnar formats eliminate this step: the byte layout is the memory layout. DuckDB reads vectorized columns directly — no per-character parsing, no constructing hundreds of thousands of JS objects. **When feature counts are high, what actually freezes the page is often not rendering but parsing and GC.**

Benefit 3: Caching becomes nearly free.

Cloud-native formats are **static files**, so they naturally consume the entire existing HTTP caching pipeline: **CDN edge caching, browser cache, Service Worker, ETag / immutable URLs** — none of which you need to implement yourself.

Compare with dynamic tile services: tile caches you must build yourself, invalidation strategies you must write yourself, cache hit rates you must monitor yourself. **A Range request hitting the same byte range of the same immutable file is just an ordinary cache hit at the CDN layer. The reason Section 5's Workbox three-tier caching strategy works at all is precisely because the resources are static and content-hashed.**

Benefit 4: The concurrency bottleneck moves off your servers.

The capacity bottleneck for tile services is server-side CPU: 100 concurrent users means 100 copies of rendering cost. **Static storage + CDN follows a completely different scaling curve** — there is no per-user rendering to scale, so the additional load shows up mostly as traffic cost. And edge caching

intercepts the vast majority of requests at the node closest to the user. What does not disappear is capacity planning: cache misses still reach origin, and object stores have their own per-prefix request-rate limits and egress bills.

This is especially critical for government dashboards and public service portals — the "idle most of the time, but everyone watches simultaneously during meetings" traffic pattern. Handling that shape with a tile service means either overpaying for peak capacity long-term or crashing during the peak.

Benefit 5: A whole link disappears from the chain, and operational costs collapse with it.

No publishing workflow, no service processes, no database connection pools, no middleware to patch. A data update is just overwriting a file. GeoLibre's entire "directly read remote GeoParquet / COG / PMTiles in the browser" capability is proof this path works. Every online catalog it connects to (STAC, Source Cooperative, Overture Maps, Planetary Computer) follows this model.

But the trade-offs are real — these four points are genuine:

- **Small data is actually slower.** Reading the header, reading the index, then fetching the data — that's several network round-trips. A few hundred KB of GeoJSON is one request and done; wrapping it in a cloud-native format is counterproductive. **The threshold is roughly "too large for a single request."**
- **Range request count increases significantly.** HTTP/1.1 handles `Range` fine, but HTTP/2 or HTTP/3 is strongly recommended at this request volume; otherwise connection overhead will eat into the benefits.
- **The server must support `Range` and CORS** (plus `Access-Control-Expose-Headers`) — this trips people up more often than expected; the next sub-section covers it in detail.
- **Computation moves from server to client.** The decoding, filtering, and stitching that servers used to do now runs in the user's browser, subject to the ~4 GiB memory and single-thread constraints (see Section 2.5). **It's not "got faster" — it's "moved where the computation happens" — and it's only a win when the new location computes more cost-effectively.**

So cloud-native formats have a well-defined sweet spot: large, static, multi-user reads, viewport-constrained access — all four satisfied yields maximum benefit. Conversely, high-frequency-write dynamic operational data does not suit this model, which is why Section 6.7's checklist explicitly says "do not touch the PostGIS path."

6.2 How GeoLibre Reads These Formats

Format	Read Path	Key Point
PMTiles	MapLibre custom protocol (<code>pmtiles</code> package)	One file replaces an entire tile service; frontend only registers one extra protocol
COG	<code>cog-tiler-wasm</code> (default) / <code>deck.gl</code> GPU / <code>TiTiler</code>	Three switchable engines; the first two are fully client-side

Format	Read Path	Key Point
GeoParquet	DuckDB-WASM <code>read_parquet</code>	Optional "in-place streaming query" — no need to copy the entire dataset into memory
FlatGeobuf	DuckDB Spatial <code>ST_Read</code>	Built-in spatial index, naturally suited for range reads
MosaicJSON / STAC catalog	Raster control reads catalog, stitches scenes at read time	The catalog itself contains no data — only a list of asset URLs
Cloud-optimized NetCDF/HDF5	kerchunk reference manifest → Zarr rendering pipeline	See 6.5

6.3 The Real Barrier Is Not Format Conversion — It's CORS

Format conversion is a one-time task. CORS is a wall you hit every single day. The source code's description of Source Cooperative is precise: the data endpoint `data.source.coop` sends `Access-Control-Allow-Origin: *` **and** supports byte-range requests, so **PMTiles protocol, DuckDB-WASM, and COG readers can all connect directly without any proxy.** But its **metadata API sends no CORS headers at all.** So how does the browser handle it? The desktop version goes through Tauri's native HTTP (server-to-server, no such thing as CORS). The web version goes through a self-built Cloudflare Worker that re-serves the JSON with CORS headers added.

The judgment "data can connect directly, metadata needs a proxy" is the first call to make when integrating any cloud-native data source.

The same Worker contains an even more instructive comment about why planetary basemaps must be proxied:

MapLibre uses `fetch()` to retrieve raster tiles, which goes through CORS checks — so the map renders as a black screen. Meanwhile, openplanetarymap.org itself works fine because Leaflet loads tiles via `<img>` tags, **and `<img>` does not perform CORS checks.**

Critical Trap

The same tile URL displays in Leaflet but not in MapLibre. The root cause may be neither the code nor the tile itself — it's that the two libraries fetch images differently.

Three practices worth borrowing from this pipeline: **fetch data server-side and add CORS headers** (avoid having the browser hit the wall directly), **cache results at the edge** (repeat requests don't go back to origin), **strict allowlist, never an open proxy** (the source code's words: "keyed to a tight allowlist so it is never an open proxy"). The third is a security baseline: a public proxy that can forward arbitrary URLs will eventually be abused, and the consequences are on you.

One more detail worth remembering: Source Cooperative's unknown API paths don't return 404 — they fall through to a catch-all page route, **returning HTML with status code 200**. So the source code validates the parsed structure on every read. `response.ok` **only means the network layer didn't error; it does not mean you got what you asked for**.

6.4 "Convert to COG" Does Not Mean Renaming the Extension to `.tif`

COG's ability to be read via Range requests depends on **internal tiling and pyramid overviews within the file**. A striped GeoTIFF can technically be range-read too — its strip offsets and byte counts are in the header — but a strip spans the full image width, so fetching a small map extent drags in far more bytes than it needs, and with no overviews there is no coarse level to zoom out against. Naming it `.tif` changes none of that, and GeoLibre's client-side readers require internal tiles outright.

GeoLibre's handling is instructive: the panel **first reads the file header with a Range request** to determine whether internal tiles exist; if not, it offers to convert the file to a COG in the browser (geolibre-wasm's `CogBuilder`, in `packages/processing/src/cog-convert.ts`) before loading. **A few KB of header for a definitive answer, rather than making the user wait through an inevitably slow load.**

The trade-offs among the three raster engines are also documented in the source: `cog-tiler-wasm` (default, browser WASM, **at the cost of only using built-in color ramps** — custom classification is lost), `maplibre-gl-raster` (GPU, full symbology), `titiler` (server-side). **The default is chosen for stability, not feature completeness.**

6.5 What If the Data Isn't Cloud-Native? Externalize the Index

NetCDF / HDF5 are de facto standards for scientific data, but they weren't designed for HTTP. GeoLibre's approach is **kerchunk reference manifests**: the manifest maps Zarr keys to `[url, offset, length]` (byte ranges within the original file), and implements a **minimal zarrita** `Readable` (**only needs a `get(key)`**) — then feeds it directly into the existing Zarr rendering pipeline. The source comment's phrase "with no rewrite" is the key point: **not a single line of rendering code changed.**

This idea generalizes: don't change the data, don't change the renderer — just insert a "key → byte range" mapping layer in between. Historical formats unsuitable for streaming reads become demand-readable as long as you can compute the offsets.

6.6 Catalog Browsers Should Do Exactly One Thing: Produce URLs

GeoLibre integrates STAC, Source Cooperative, Overture Maps, Planetary Computer, Hugging Face — a large number of online data sources — and stays manageable through one principle: **when adding a layer, deliberately delegate to the controls that already understand each format** — PMTiles to

`addPMTilesLayerFromUrl`, GeoParquet to `addVectorLayerFromUrl`, COG to `app.addCogLayer`. As a result, data clicked in from a catalog and data added manually via Add Data are **fully equivalent**: the same layer panel, the same styling capabilities, the same persistence into projects.

One data source integration = one "how to find the URL" adapter. Reading and rendering are always a single implementation. This is the only way to integrate N data sources without losing control.

6.7 Adoption Order & Key Takeaways

Adoption order (from lowest to highest investment):

1. Convert **externally-facing static data** to PMTiles — eliminate an entire tile service
2. Convert raster outputs to **genuine** COG (verify internal tiles and pyramids) — direct frontend read or pair with titiler
3. Convert large vector tables to GeoParquet — query directly with DuckDB
4. For historical formats that can't be converted, consider external indexes (the kerchunk approach)
5. **Dynamic operational data stays on PostGIS — do not touch this path**
6. **1. The essence is "index in the file + HTTP Range" — so what gets replaced is the server side, not the frontend library.**
7. **2. First distinguish "data can connect directly" from "metadata needs a proxy." The proxy must be allowlisted + edge-cached, and must never be an open proxy.**
8. **3. When programming against external APIs, validate the parsed structure — don't trust `response.ok`.**
9. **4. Multi-source integration must converge on a single set of format-reading implementations; catalog panels are only responsible for producing URLs.**

Item 1 has the highest ROI: one PMTiles file + a static server replaces an entire tile publishing workflow. Many projects can complete this transition in a short time.

In Closing: An Adoption Checklist Ordered by Investment

Validatable Within a Day

1. Try **DuckDB-WASM Spatial** once — read a remote GeoParquet directly and experience "no backend required"
2. Convert heavy engines in your project to **on-demand lazy loading + separate chunks**, especially 3D engines — get them off the first-screen path

3. Make all CDN / resource URLs runtime-configurable

Achievable This Iteration

1. Set a clear threshold for large-data layers; switch to client-side tiling above the line (`@maplibre/geojson-vt` + `@maplibre/vt-pbf`)
2. Wire `AbortController` into all async pipelines, including non-network long loops
3. Check whether your undo stack has a memory budget and whether the newest snapshot is always retained; check whether virtual list sorting operates on the full data model
4. Rewrite enums using `as const` arrays so runtime validation and type definitions share a single source
5. Pin cross-language / cross-process duplicate constants with a unified `SYNC:` marker

Worth Evaluating as a Project

1. Upgrade DuckDB-WASM Spatial **from "format reader" to "query layer"**: let layers simultaneously be SQL tables, so users can write `JOIN` and `ST_Intersects` across two layers
2. De-framework the computation layer so the browser and server run the same code — never reconcile results again
3. Move static data pipelines to cloud-native formats, starting with PMTiles; confirm storage supports `Range` and CORS, and clarify which metadata must go through a proxy
4. If building a web application, rebuild your Service Worker caching using the three-tier strategy

None of the items on this checklist require importing a single line of GeoLibre's code. Its greatest contribution is not yet another mapping application — it's proving, with a complete project you can run, install, and read the source of, that **a spatial database running in the browser is sufficient to replace the half of the work that used to require a server.**

<https://geolibre.app/tutorials/source-code-deep-dive-zh/>

GeoLibre 源码深度拆解：值得 WebGIS 开发者复用的核心技术

摘要：本文深度剖析 GeoLibre 源码架构，包含依赖选型、DuckDB-WASM Spatial 核心引擎、Web 端性能优化、状态管理、离线 PWA 缓存及云原生地理空间数据方案。文中全部标注源码文件路径，方便开发者查阅复用。

原文来源：微信公众号"GIS开发手记"，原文链接：<https://mp.weixin.qq.com/s/482yiH-VsKP7OmBw9UWG6g>

本文使用简体中文撰写。

前言

本文对照 GeoLibre 仓库源码逐层拆解项目架构，梳理项目依赖选型、DuckDB-WASM 空间计算内核、前端性能优化策略、状态管理设计、离线缓存三层策略与云原生地理数据整套方案，文中标注全部对应源码文件路径，所有技术方案均可直接复用至 WebGIS 项目。

一、它的工具箱：值得单独拿出来看的开源库

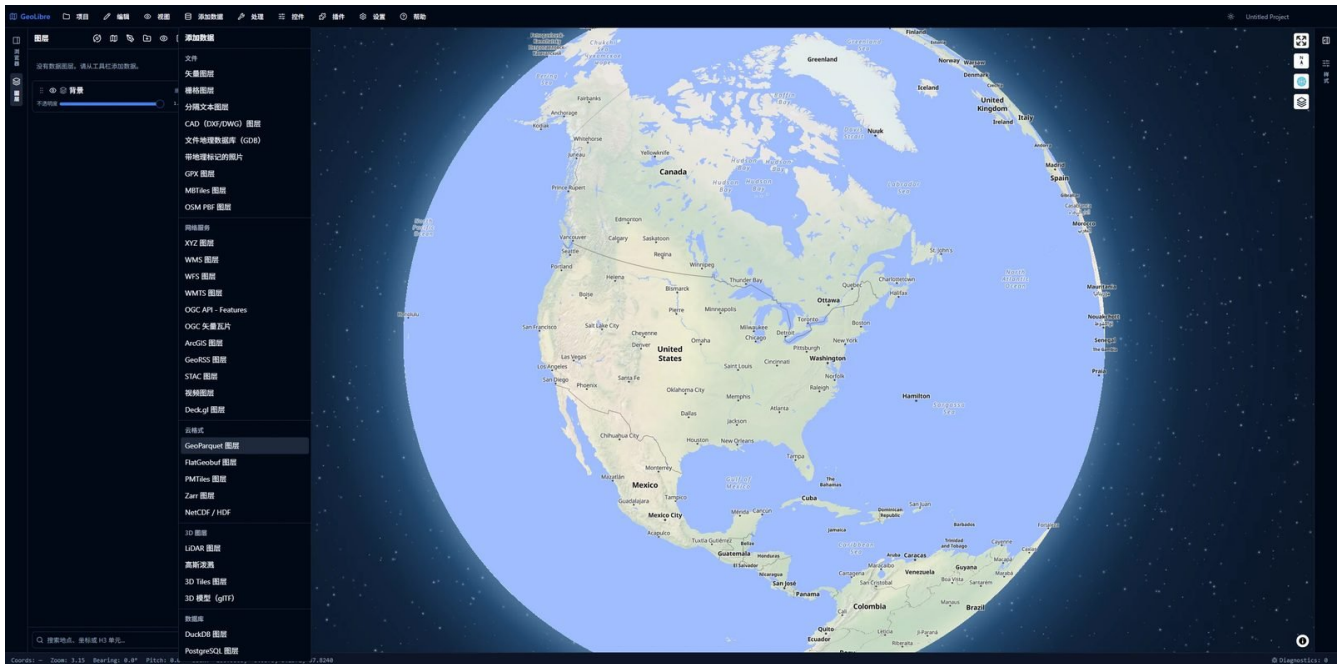
仓库是 npm workspaces 单体仓库，7 个包 + 桌面应用。依赖按用途归类如下，以下标注的为值得重点关注的库。

1.1 格式解析：轻量优先，重武器备用

库	版本	负责	为什么是它
@duckdb/duckdb-wasm	1.33.1	GeoParquet、FlatGeobuf、GML、DXF、TAB，以及空间 SQL	一个库同时当 格式驱动和计算引擎
shpjs	6.2	Shapefile	纯 JS 几十 KB；.prj 定投影、.cpg 定编码（中文属性乱码可以得到正确处理）
fflate	0.8	zip / kmz / aprx 解压	极小极快
sql.js	1.14	GeoPackage 读和写	GPKG 本质就是 SQLite ，轻量高效
exifr	7.1	照片 EXIF GPS → 点图层	无人机场景实用
gdal3.js	2.8	只用于地理配准的 GeoTIFF/COG 导出	wasm ~28MB + data ~12MB，只从 CDN 拉，从不打包
geotiff	3.0	GeoTIFF 解码	—

库	版本	负责	为什么是它
h5wasm / netcdfjs	—	HDF5 / NetCDF-3	分工明确，各管一种
@osmix/pbf @osmix/core	—	OSM PBF	跑在 Web Worker 里
pmtiles proj4 fast-xml-parser	—	瓦片包 / 投影 / XML	基础件

这里最值得借鉴的是「按格式选最轻的路径」。源码里 `packages/plugins/package.json` 有 60 多个依赖，但没有一个「大一统读取层」——每种格式各走各的最短路。



KML 那条尤其能说明取舍。`docs/architecture.md` 的 DuckDB-WASM 一节写得很直白：KML 由自研解析器读取，为的是保留内嵌符号化，输出 `simplestyle-spec` 属性（`fill`、`stroke`、`stroke-width`），这样带样式的 KML 在 GeoLibre 里和在 Google Earth 里长得一样；解析器读不了的才退回 DuckDB Spatial，代价是丢掉样式。

关键启发：通用库为了统一数据模型，一定会丢掉格式特有的信息。而那些信息往往正是用户最在意的部分。

同样的模式在 Shapefile 上再现一次：先用 `shpjs`，读不了才试 DuckDB Spatial。快路径 + 慢而全的兜底，是格式解析的通用套路。

1.2 空间计算：四层引擎，默认最轻

这块的分工很容易看混，按源码捋一遍（依据 `docs/architecture.md` 的 Python sidebar 一节）：

引擎	在哪跑	定位
Turf.js (@turf/* 二十五个左右子包)	浏览器纯 JS	矢量工具的默认引擎，零依赖零后端

引擎	在哪跑	定位
GeoPandas / Shapely	Python sidecar	需要投影感知结果时的升级项
GeoPandas / Shapely	浏览器 Pyodide	同一份代码，Web 版也能用
DuckDB / PGlite+PostGIS / SedonaDB	浏览器 或 sidecar	SQL Workspace 的三个引擎

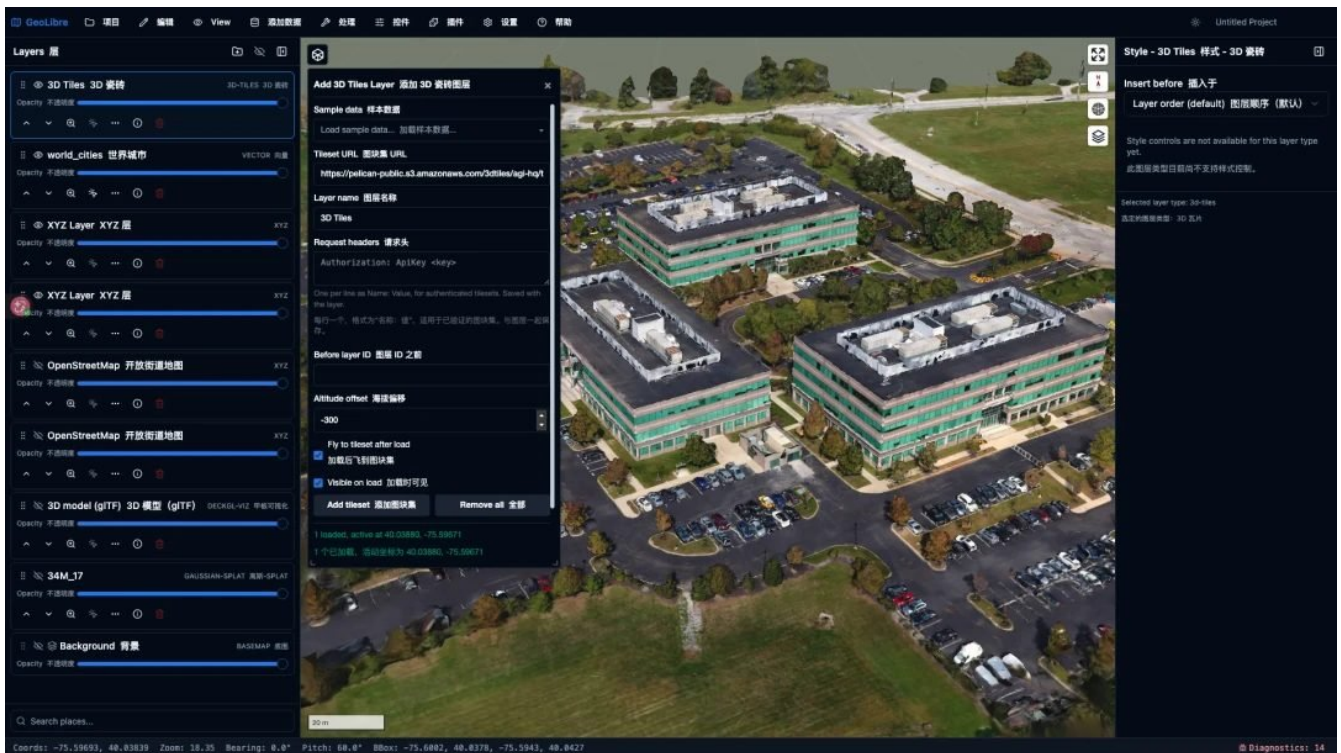
注意 Turf 是**按需引入子包**的（`@turf/buffer`、`@turf/intersect`），不是整包 `import`。这个习惯很重要——Turf 全量引入是很大一坨，按子包引才有意义。

那条「一份代码两处运行」的设计也写在同一节：几何逻辑是一个**无框架模块** `backend/geolibre_server/geolibre_server/vector_ops.py`，一个 Vite 插件（`vite-plugins/copy-vector-ops.ts`）把它复制进前端包，浏览器侧用一个经典 Web Worker 加载 Pyodide、装 `geopandas`、通过 JSON 字符串边界调 `run_vector_tool`。

实际教训：做过 GIS 的都知道这个坑——前端 turf 算缓冲区，后端 PostGIS 算缓冲区，面积差 0.3%，追两天发现是分段数默认值不同。

1.3 渲染与图层：MapLibre 插件生态

库	负责
<code>maplibre-gl 6.11</code>	主地图
<code>deck.gl 9.4 (core/layers/geo-layers/mesh-layers/aggregation-layers/mapbox)</code>	COG、3D Tiles、I3S、可视化图层，交织进 MapLibre 画布
<code>maplibre-gl-3d-tiles / -lidar / -splat / -raster / -vector</code>	不换引擎，MapLibre 上直接加 3D Tiles、点云、高斯泼溅
<code>@developmentseed/deck.gl-geotiff / -raster</code>	COG 渲染
<code>@carbonplan/zarr-layer</code>	Zarr 科学数据
<code>@loaders.gl/i3s / @esri/maplibre-arcgis</code>	Esri 生态接入
<code>@geoman-io/maplibre-geoman-free</code>	绘制与编辑
<code>maplibre-gl-time-slider / -swipe / -layer-control / -basemap-control</code>	交互控件
<code>@tanstack/react-virtual</code>	属性表虚拟化
<code>cesium 1.146</code>	可选的三维球分屏， 懒加载 ~4.7MB 独立 chunk



`maplibre-gl-*` 这一串是很多人不知道的：3D Tiles、COPC 点云、高斯泼溅，全都能在 MapLibre 画布里加载，不需要引入 Cesium。如果项目只是「想看看 3D Tiles」，这条路比上一整套三维引擎轻得多。

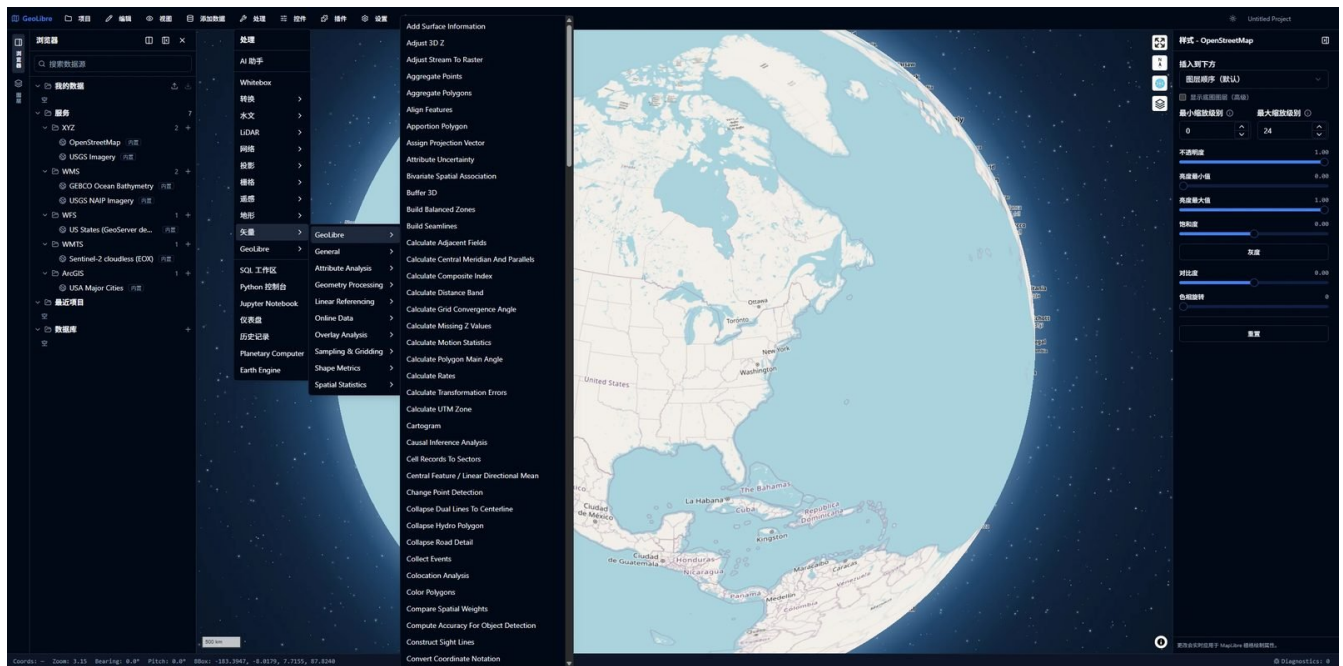
借鉴要点

- 一、按格式选最轻的路径。前端库体积差异是数量级的（shpjs 几十 KB vs gdal3.js 40MB），选型体积的代价是用户在承担首屏加载时间。
- 二、快路径 + 兜底链：90% 的正常数据走轻量解析，10% 的怪数据退回重武器。
- 三、保真度优先于通用性。为「保留 KML 符号化」自研一个解析器是值得的。
- 四、Turf 按子包引入，不要整包 import。

二、DuckDB-WASM Spatial：一个库同时当格式驱动和计算引擎

这一节是这篇的重点。如果只看一件事，看这个。

先把 WebAssembly 这件事一句话说完：`wasm` 这个标识符在 GeoLibre 源码里出现了一千多次，数据库、语言运行时、原生工具链、编解码、机器学习五类能力全部由 WASM 引擎承担——DuckDB、sql.js（SQLite）、PGLite + PostGIS、CereusDB（SedonaDB）、Pyodide、`geolibre-wasm`（编译成 WASI 的 Whitebox 引擎，外加 GeoLibre 自己的 Rust 工具）、`gdal3.js`、`cog-tiler-wasm`、`h5wasm`、`onnxruntime-web`。原生语言连起来看就明白了：C、C++、Rust。WASM 在这里不是给 JS 加速，是把 GIS 领域几十年积累的原生生态整体搬进浏览器。



这十来个引擎里，有一个值得当下就该去试的：[@duckdb/duckdb-wasm](#) 加上它的 `spatial` 扩展。原因很简单——其他引擎解决的是「某一类活儿」，而它同时是**格式驱动和计算引擎**，一个库就能把「读数据」和「算数据」两件事一起接管。

2.1 先把三层关系讲清楚

这三个名字经常被混着说，其实是套娃：

层	是什么	关系
WebAssembly	浏览器里的二进制指令格式	只是一种「运行原生代码的能力」，本身不做任何 GIS 的事
DuckDB-WASM	DuckDB（C++ 写的分析型数据库）编译成 WASM 的产物	它是「用 WASM 实现的 DuckDB」，不是 WASM 本身
Spatial 扩展	DuckDB 的空间扩展，同样是一份独立 <code>.wasm</code>	需要单独 <code>INSTALL / LOAD</code> ，不加载就没有任何空间函数

最容易踩的认知误区：以为装了 `duckdb-wasm` 就有空间能力。没有。`ST_Read`、`ST_Transform`、`ST_AsWKB` 这些全在 `spatial` 扩展里，它是运行时从 CDN 拉下来再加载的第二份 WASM。

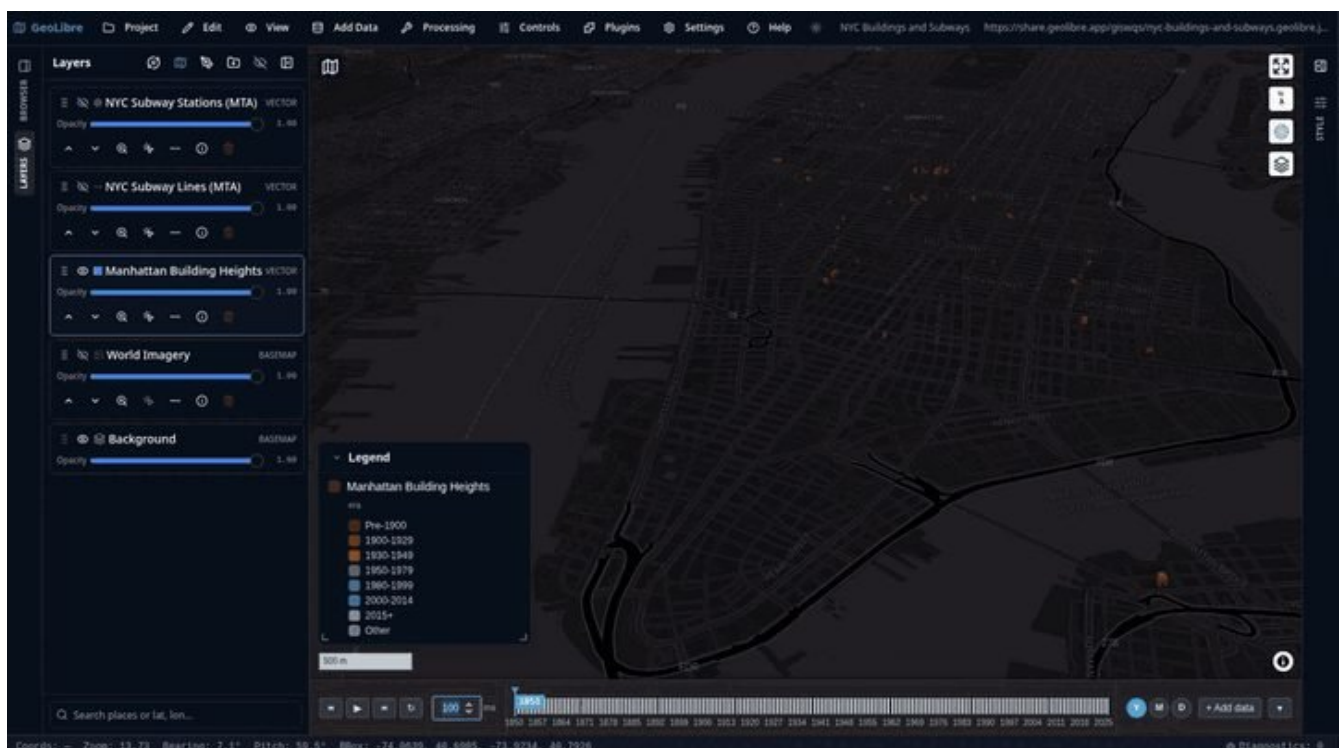
而 `spatial` 扩展的 `ST_Read` 背后，是 **GDAL 的一个子集**。这句话的含义是：在浏览器里发一条 SQL，就能读这个子集覆盖的那些矢量格式。注意它用的是扩展自带的 GDAL，不是机器上装的那份，所以具体能读哪些取决于当前加载的构建——用 `SELECT * FROM ST_Drivers()` 可以列出来。

2.2 GeoLibre 拿它干了什么

源码中 DuckDB 的调用点归类如下：

用途	走的接口	备注
GeoParquet	<code>read_parquet</code>	本地和远程都走它，远程按 HTTP Range 拉
FlatGeobuf / GML / DXF / TAB 等	<code>ST_Read</code>	GDAL 后端，能读多少看扩展装载情况
Shapefile (zip)	先 <code>shpjs</code> ，读不了再交给 Spatial	轻量优先，重武器兜底
KML	先自研解析器（保符号化），失败再交给 Spatial	Spatial 只能拿到几何，样式会丢
CSV 里的 WKT 几何列	DuckDB SQL	直接把文本几何转成图层
坐标系转换	<code>ST_Transform</code>	老式 <code>crs</code> 字段的 GeoJSON 靠它重投影
SQL 工作区	完整 DuckDB SQL	已加载的图层被注册成表，可以直接 JOIN

关键洞察： 注意最后一行。这是一个很容易被低估的设计：地图上的图层同时是 SQL 里的表，用户可以对着两个图层写 `JOIN`、写 `ST_Intersects`，结果再直接变成新图层。「地图」和「数据库」在这里是同一份东西的两个视图。



SQL 工作区里还有三处「帮用户把话说全」的改写，做 SQL 控制台的可以直接照这个思路来：

- `FROM` / `JOIN` 后面裸写一个 URL 或本地路径就行，源码会按扩展名自动套上对应 reader（`read_parquet`、`read_csv_auto`、`ST_Read`），用户不用记哪种格式配哪个函数
- `s3://`、`gs://`、`az://` 会被翻译成对应的公开 HTTPS 地址，走同一条 HTTP Range 通道
- reader 参数里的 HTTP URL 会被改注册成 DuckDB 文件句柄，由 JS 侧发 Range 请求，而不是走 WASM 内置的 https——因为后者在很多服务器上直接报错

实现细节：这三处改写都只作用于真正的 **reader 调用参数**，字符串字面量和注释里长得像 URL 的东西会被跳过（源码里专门做了一遍 SQL 字面量遮罩）。做 SQL 改写千万别拿正则直接怼原文。

2.3 接进项目时，源码里这四个细节值得借鉴

第一，bundle 要按浏览器能力选。 duckdb-wasm 发了多个 WASM 构建（mvp / eh 等），能力不同、体积不同。duckdb-wasm-bundles.ts 里只做了一件事：把 mvp 和 eh 两份产物用 Vite 的 ?url 拿到地址，交给 duckdb.selectBundle() 在运行时挑：

```
const MANUAL_BUNDLES: duckdb.DuckDBBundles = {
  mvp: { mainModule: duckdbWasmMvp, mainWorker: mvpWorker },
  eh: { mainModule: duckdbWasmEh, mainWorker: ehWorker },
};
export async function selectDuckDbBundle() {
  // 在这里才 import，让 DuckDB-WASM 的 JS 不进启动路径
  const { selectBundle } = await import("@duckdb/duckdb-wasm");
  return selectBundle(MANUAL_BUNDLES);
}
```

手动列 bundle 而不是用默认 CDN 解析，是为了让 Vite 把 wasm 和 worker 打成带哈希的本地产物——这样才能被 Service Worker 的 CacheFirst 安全缓存（见第五节）。（npm run lite:build 那个变体会换成 duckdb-wasm-bundles.cdn.ts，从 jsDelivr 加载同一个锁定版本，给有单文件体积上限的托管平台用。）

第二，扩展加载必须做成「按实例只跑一次」。 INSTALL spatial 会走网络，LOAD 有状态，并发调用会互相打架。GeoLibre 的做法很值得借鉴：

```
// 按实例记忆化：共享库和 SQL 工作区各有自己的 DB，各记各的
const spatialExtensionByDb = new WeakMap<duckdb.AsyncDuckDB, Promise<void>>>();

export async function ensureSpatialExtension(db, connection, beforeLoad?) {
  let promise = spatialExtensionByDb.get(db);
  if (!promise) {
    promise = (async () => { /* ...INSTALL / LOAD... */ })();
    spatialExtensionByDb.set(db, promise);
  }
  try {
    await promise;
  } catch (error) {
    // 只在记忆项仍指向这次失败时才清除，否则下次永远重试不了
    if (spatialExtensionByDb.get(db) === promise) spatialExtensionByDb.delete(db);
    throw error;
  }
}
```

这段小代码里有三个决定。缓存的是 Promise 不是布尔值——并发调用共享同一次加载。用 WeakMap 按实例存——不是模块级单例，实例被重建后自动失效。失败要清除记忆，但清除前先校验身份——避免清除别人刚写进去的新加载。三条缺一，就会在生产环境上遇到「偶发的空间函数不存在」。

第三，留一个「不联网也能加载扩展」的口子。 INSTALL spatial 默认从 DuckDB 的扩展仓库下载。GeoLibre 加了一个环境变量 VITE_DUCKDB_SPATIAL_EXTENSION_PATH：配了就直接 LOAD '<本地路径>'，跳过 INSTALL。

做内网、政务、离线桌面端的，这一行配置就是能不能用的分界线。同样的模式还复用给了 h3 社区扩展（`INSTALL h3 FROM community`）——DuckDB 的扩展生态在浏览器里是能用的，这点很多人不知道。

第四，Worker 和生命周期要自己管。 DuckDB 跑在独立 Worker 里，**WASM 堆是跟着 Worker 走的，只有 `terminate()` 才真正还给系统。** GeoLibre 给 SQL 工作区的实例做了引用计数：还有查询在跑就先标记「空闲时销毁」，等最后一个查询释放再关。**这在需要重建实例的场景里是刚需——直接 `terminate` 会把正在跑的查询打断。**

2.4 一个真实的坑，比任何教程都值钱

源码注释中有一条值得关注的记录，它比上面所有内容都更能说明「在浏览器里跑数据库」是什么体验：

duckdb-wasm 1.33.1-dev45 会**永久破坏**一个实例的远程 `read_parquet` ——只要这个实例在**完成第一次远程读取之前**执行了 `LOAD spatial`。之后所有远程 Parquet 读取都报 `stoi: no conversion`，而且无法在原地恢复。

GeoLibre 的应对是两手：

一是预热。 `ensureSpatialExtension` 那个 `beforeLoad` 钩子就是干这个的——在 `LOAD spatial` 之前，先拿这条查询自己的远程 reader 跑一次 `SELECT 1 FROM read_parquet(...) LIMIT 0`。`LIMIT 0` **只会取 Parquet 的 footer，几乎不产生额外流量。**如果这条查询本身没有远程 Parquet，就退回读一个公开的小样本文件兜底。

二是重建。万一还是中招了（比如上一次预热失败），`runSqlQuery` 会捕获 `stoi: no conversion`、调 `resetSqlDatabase(db)` 把那个实例整个换掉、然后**重试一次**。重建时会重新走一遍预热，所以第二次是干净的。

这里还有两个防御细节：重试只在语句里**确实有远程 reader 调用**时才触发（字符串字面量里出现的 URL 不算），并且 `resetSqlDatabase` 会先确认「要换掉的实例仍是当前实例」再动手。

三条能带走的经验

- **WASM 扩展的加载顺序是有副作用的。**不要假设「什么时候 LOAD 都一样」，尤其是涉及网络子系统的扩展。
- 把「预热」和「重建」当一等公民，而不是补丁。遇到这类不可逆的坏状态，能优雅恢复的前提是实例本来就是可替换的。
- 给上游 bug 留退路，但把范围收窄到症状。匹配具体错误消息 + 具体条件 + 只重试一次，而不是无脑 catch 全部重试。

2.5 它的边界在哪

关于 DuckDB-WASM 的赞誉文章很多，这一节专门整理源码中揭示的实际限制。了解这些限制之后，反而更清楚其适用边界——知道约束所在，比在不知边界的情况下使用更稳妥。

限制	数值	原因
远程文件大小	2 GiB	DuckDB-WASM 的 HTTP 文件系统用 32 位存远程文件大小
单标签页内存	~4 GiB	WASM 的地址空间上限
要素数确认阈值	500,000	超过要用户确认才物化
线程	单线程	超大数据该回服务端就回服务端

2 GiB 那条有个细节很有参考价值：这个门禁是在开始摄取之前就判掉的（`_registerSource` 跑在流式分支前面，所以「流式读取」也绕不过去）。GeoLibre 的做法是在文件浏览面板上直接把话说清楚，而不是让用户点了 Add 之后等一个必然失败的结果。能提前判定的失败，就不要留到运行时。

还有一条关于拷贝 vs 流式的取舍：只有 GeoParquet 真正支持「原地查询不拷贝」，其他格式即使给了 stream 选项也会悄悄回落成拷贝——所以 GeoLibre 干脆只在 GeoParquet 上显示这个按钮。一个点了没区别的按钮，比没有这个按钮更糟。而拷贝模式的内存占用大致跟解压后的数据量走，是 Parquet 磁盘体积的好几倍，超过 100 MB 就会提示改用流式。

最后一个有意思的对照：GeoLibre 桌面端留了个 `native-duckdb` 编译开关，可以换成原生 DuckDB。但 Rust 侧写着一条硬约束——

```
compile_error!("the `mas` (Mac App Store) build must not enable `native-duckdb`: \
DuckDB loads its spatial extension as unsigned native code at runtime, \
which App Sandbox and App Store guideline 2.5.2 forbid.");
```

原生版更快，但它运行时加载未签名的原生扩展，直接违反 Mac App Store 的规则。反过来说，WASM 版本因为跑在沙箱里，反而是那个「哪都能上架」的选择。这是选型时很少有人算到的一笔账。

2.6 借鉴要点

- 一、先试 DuckDB-WASM Spatial，再考虑别的。一个库同时解决「读格式」和「算空间」，是目前 Web 端投入产出比最高的一步。远程 GeoParquet + `read_parquet`，十几行代码，不需要任何后端。
- 二、扩展加载做成「按实例记忆化的 Promise + 失败带身份校验地清除」。这个模式对所有「异步一次性初始化」都成立，不止 DuckDB。
- 三、必须留本地扩展路径的口子。内网 / 离线 / 桌面端，全靠这一行配置。
- 四、WASM 引擎一律：懒加载 + 独立 chunk + Worker 执行 + 版本锁死 + 生命周期可重建。别让它上主线程，别让它进首屏。
- 五、把硬限制写成常量并前置校验。2 GiB、50 万要素、~4 GiB 堆——能提前告诉用户的失败，不要留到运行时。

三、Web 端性能：五条优化，加一个负面结果

这一节所有数字均定位到了具体文件行号。

3.1 阈值化：定好数字，过线换实现

核心思路一句话：**不追求一套方案通吃，而是在明确的阈值上切换实现。**

源码里的阈值常量（都是可查的）：

常量	值	位置	保护什么
<code>LARGE_VECTOR_FEATURE_THRESHOLD</code>	50,000	<code>core/src/types.ts:783</code>	主线程 GeoJSON 解析
<code>maxHistoryFeatureCount</code>	500,000	<code>core/src/history.ts:29</code>	撤销栈内存
<code>DUCKDB_VECTOR_FEATURE_WARN_COUNT</code>	100,000	<code>core/src/types.ts:2540</code>	结果物化内存
<code>MAX_CEREUS_FEATURES</code>	50,000	<code>lib/sedona-workspace.ts:25</code>	WASM 堆
<code>MAX_DERIVED_FEATURES</code>	50,000	<code>map/src/derived-geometry.ts:42</code>	派生几何计算
<code>historyCoalesceMs</code>	400 ms	<code>core/src/history.ts:6</code>	撤销记录爆炸
远程文件	2 GiB	<code>plugins/remote-file-formats.ts</code>	DuckDB-WASM 32 位

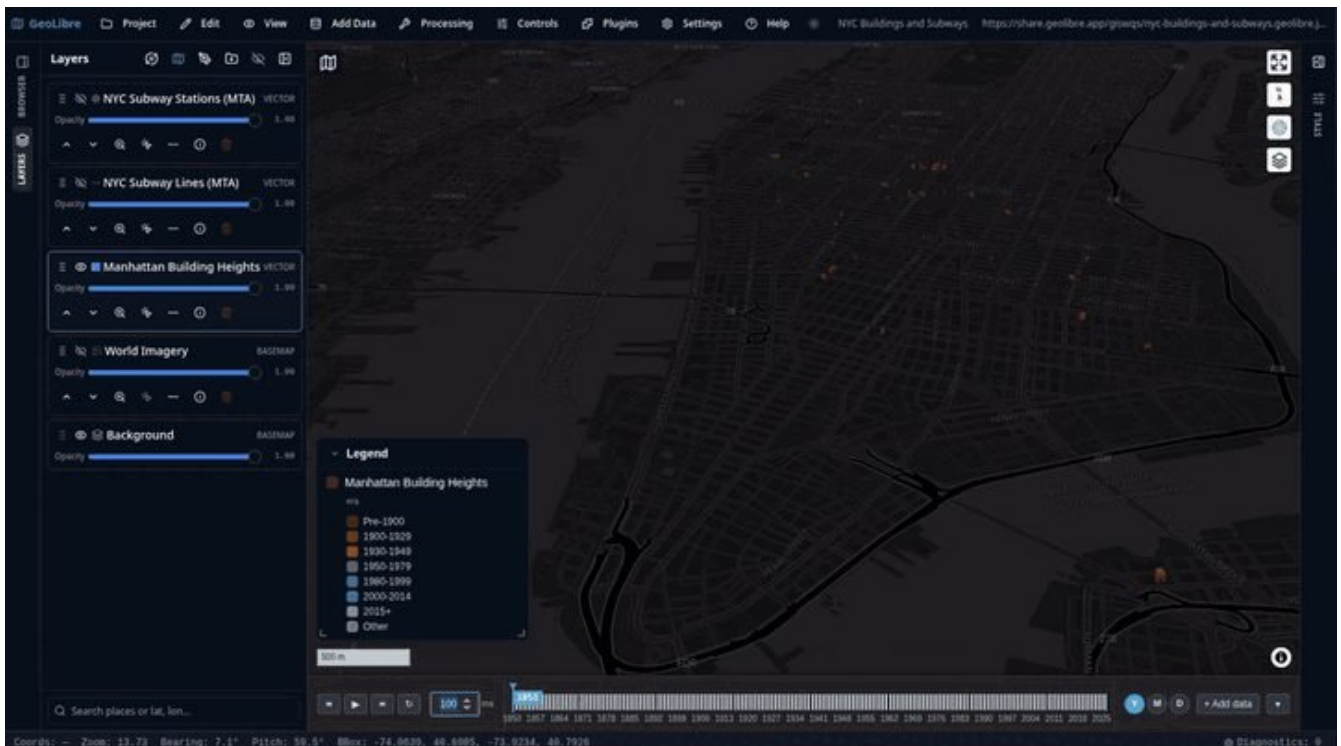
注意这些阈值全是命名常量、全带文档注释、而且 `historyCoalesceMs` 和 `maxHistoryFeatureCount` 都有 **setter** 可运行时改（测试里设成 0）。这比散落在代码里的 magic number 高一个段位。

启发：每个阈值都要说得出口「它保护什么」。说不清保护目标的阈值缺乏可靠依据，难以长期维护。

3.2 客户端切片：超过 5 万要素就现场切瓦片

流水线在 `packages/map/src/geojson-vt-protocol.ts`（协议和注册表）和 `geojson-vt-index.ts`（建索引和编码）里，一步步是这样的：

- 索引用 `@maplibre/geojson-vt`（注意是 MapLibre 的 fork，**Supercluster 也在这个包里**，注释说它就是「the same engine MapLibre uses internally」）
- 点图层用 `Supercluster` 索引，其它用 `GeoJSONVT`
- 编码用 `@maplibre/vt-pbf` 的 `fromGeojsonVt`
- 通过自定义协议 `geolibre-gjvt` 喂给 MapLibre
- `TILE_EXTENT = 4096`，`TILE_MAX_ZOOM = 16`（超过就让 MapLibre over-zoom）
- 建索引和瓦片编码跑在 Web Worker（`geojson-vt.worker.ts`）里，大图加载时不再卡主线程；没有 Worker 的环境下，同一套代码直接在主线程跑



两个细节值得单独说。

第一，按图层存的瓦片注册表放模块级 Map，不进 store。 源码注释是这么写的：

Keyed by layer id. Module-level rather than on the Zustand record because **tile indexes are large, non-serializable objects that must not enter app state or be written to .geolibre.json.**

这给了一个特别好用的判据：**什么该进全局状态？——能写进工程文件的才进。** 派生产物（瓦片索引、空间索引、WebGL buffer、解码位图）一律放模块级缓存或引擎侧。

第二，编码前先看中止信号。 就一行：

```
// packages/map/src/geojson-vt-protocol.ts
if (signal?.aborted) return empty;
```

因为 MapLibre 会取消滚出屏幕的瓦片请求，结果反正会被丢掉，那就别算。

一般化：凡是「按需生产」的异步管线，都必须支持取消。 对应到实际项目里：快速拖动地图时还在给离屏瓦片做投影转换、属性表快速翻页时前几页查询还在跑、图层快速切换时上一个的解析结果晚到覆盖了新图层（经典竞态）。

`AbortController` 不只是给 `fetch` 用的

任何超过一帧的循环，都该在循环体里检查 `signal.aborted` 并早退。

3.3 撤销栈：三个精细到值得借鉴的处理

`packages/core/src/history.ts` 这个文件值得直接打开看，不到两百行，密度很高。用的是 `zundo`（Zustand 的时间旅行中间件）。

问题：**每个快照都持有图层的完整 GeoJSON**，反复编辑会把好几份副本钉在内存里（注释里点了 issue #341）。

处理一，软预算按要素数算。 注释解释了为什么不算字节：「Feature count is a cheap proxy for payload size; it avoids serializing geometry on every edit」——**为了度量内存而序列化几何，本身就是性能问题。**

处理二，永远保留最新一个快照：

```
// trimHistoryBySize: 从新往旧走，累计装不下就停
// The newest snapshot is always retained, even if it alone exceeds the budget.
let total = distinctFeatureCount(pastStates[lastIndex], seen);
```

「内存不够就清空撤销栈」是很多项目的做法，但那意味着用户在最需要撤销的那一刻——刚做完一次大编辑——恰好没有撤销可用。

处理三，按对象引用去重。 `distinctFeatureCount` 用一个 `Set` 记住见过的 payload：没改动的图层在多个快照间共享同一个引用，只算一次。所以**「保留很多个小图层的快照」几乎没有额外内存开销。**

还有一个容易忽略的细节：那个 400ms 不是普通防抖，是**前沿防抖（leading-edge debounce）**，作为 `zundo` 的 `handleSet`：

firing only on the leading edge records the pre-burst state once

差别很关键——要记录的是**「这串连续操作之前」**的状态，所以必须在第一次触发时就记，而不是等安静下来再记。**拖滑块前的值才是想撤销回去的值。**如果用普通的尾部防抖，存下来的是拖拽过程中的某个中间值。

3.4 虚拟化只管渲染

属性表用 `@tanstack/react-virtual` 虚拟化，但**排序、筛选、选择跑在完整数据模型上。**

这句话平淡，却是个高频错误：很多虚拟列表实现是「只对已渲染的行排序」，结果排序结果随滚动位置变化、全选只选中可见行。**而且这类 bug 在小数据量测试时根本复现不出来。**

3.5 体积：懒加载做到什么程度

`apps/geolibre-desktop/vite.config.ts` 里的分包配置（`manualChunks` 加上 `Rolldown` 的 `CODE_SPLITTING_GROUPS`）是这套东西的核心。举个最有代表性的例子——Cesium。源码注释记录过一次教训：Cesium 的 chunk 曾经吞进了共享的辅助模块，结果「the whole ~4.7 MB Cesium chunk was modulepreloaded on every boot」；现在它单独成组，因为**「Cesium only loads when a pane switches to the 3D globe.」**

让它不进启动路径，是关键。 对 Web 端 GIS 来说，最容易获得的性能提升就是这个：真正需要三维的用户可能只有 20%，没理由让 100% 的人为它承担首屏加载开销。

源码注释中有一份详细的体积分析，整理如下：

重资源	体积	处理方式
CesiumJS	~4.7 MB	独立 chunk，切到球面视图才 <code>import()</code>
PGLite + PostGIS	~25 MB（打进桌面端会多 ~22 MB 几乎不可压缩的体积）	默认走 jsDelivr CDN，不进构建
gdal3.js	wasm ~28 MB + data ~12 MB	从不打包 ，只从 CDN 取；关掉 CDN 就是关掉这个功能
Pyodide / CereusDB	各数十 MB	CDN 加载，PGLite 和 CereusDB 可用构建开关改成内置

所以那个「安装包只有 30MB」的数字，是用「几乎所有重引擎都不打包」换来的。这个思路值得借鉴：重引擎的默认位置应该是「用户第一次点它的时候才下载」，不是「装在包里以防万一」。

配套还有一条：**所有外部资源地址都做成运行时可配置。** `VITE_PYODIDE_INDEX_URL`、`VITE_DUCKDB_SPATIAL_EXTENSION_PATH`、`VITE_CESIUM_TOKEN` 都能在运行时指到内部镜像，**不用重新构建。**

这条对国内内网/离线交付是保命设计。CDN 地址硬编码进 bundle，等着的就是现场断网时重新打包。

还有一条容易忘的：`packages/processing/src/ort.ts` 里注明，onnxruntime-web 的 WASM 产物从 CDN 拉，**CDN 上的版本必须和 npm 里 pin 的版本严格一致**，否则是运行时崩。**凡是「JS 胶水在包里、WASM 产物在 CDN」的库都有这个坑**，做法是把版本号写成一个常量，两边都从它取。

说到底：CDN 地址、WASM 版本、镜像路径——三样东西任何一样写死在代码里，等着的就是某个周五晚上的紧急打包。

3.6 一个诚实的负面结果

`docs/architecture.md` 最后一节标题就叫「Performance: map rendering on Linux (WebKitGTK)」，里面的细节值得完整看一遍：

- 空白地图任意缩放级别稳定 60 FPS
- 一旦有瓦片图层（矢量或栅格 XYZ），**加载瓦片期间 FPS 掉到个位数**，加载停止立刻回到 60
- 根因：WebKitGTK 在主线程做每张新瓦片的 GPU 上传（栅格是纹理，矢量是顶点缓冲），走同步 WebGL 调用，加上随后的淡入重绘
- **单次瓦片集成渲染周期实测 ~125 ms，Chromium 只要几 ms**
- **明确排除**：矢量瓦片解析和 bucket 构建跑在 MapLibre 的 Worker 里，不是瓶颈

排除清单写得极细：软件渲染（确认在用 Intel i915 GPU）、GPU 饱和（渲染引擎还有 ~20% 空闲）、Tauri IPC 文件读取（22MB GeoJSON 约 126ms）、`JSON.parse`（约 36ms）、KWin 合成器延迟、`renderWorldCopies`、球面 vs 墨卡托投影、`preserveDrawingBuffer`。

文档里甚至给了复现用的一行 FPS 计数器，和三条**尚未实现**的缓解措施（加大 `maxTileCacheSize`、512px 栅格瓦片、`fadeDuration: 0`，并且注明要按 WebKitGTK 门控，别拖累 Chromium 构建）。

两个教训：

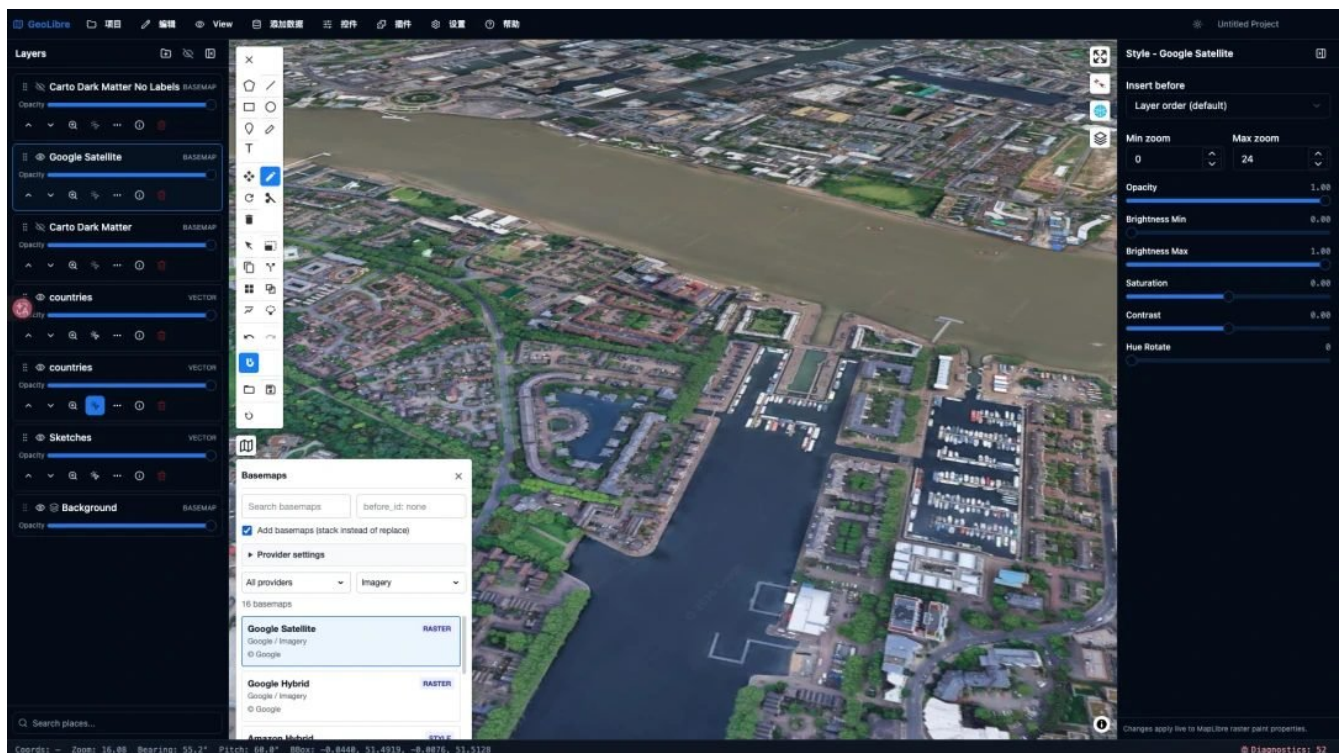
- **跨 WebView 的性能不可外推。** Chrome 上测的 60 FPS，在 Electron 的旧 Chromium、国产浏览器内核、WKWebView 上都可能不成立。**有客户端交付需求的必须在目标 WebView 上实测。**
- **把排查过程连同「已排除项」写进文档，比只写结论有价值。** 那份 8 项排除清单能给下一个人省两天。

编者注：能把这种负面结果写进架构文档，还附上复现方法和未实现的待办，这个项目的工程素养是够的。

四、状态管理：三个可直接复用的小模式

这节讲的是纯 Web 工程，跟 GIS 关系不大，但迁移成本最低。

`@geolibre/core` 的依赖只有四个：`zustand`、`zundo`、`uuid`、`@maplibre/maplibre-gl-style-spec`。注意它不依赖 `maplibre-gl` 本体（那个 `style-spec` 包只是用来求值样式表达式的），所以状态层和渲染层是真的分开的。



模式一，常量数组当单一来源。 `packages/core/src/types.ts:63` 那 20 个图层类型：

```
export const LAYER_TYPES = [
  "geojson", "raster", "wms", "wmts", "xyz", "vector-tiles", "arcgis",
```

```
"pmtiles", "mbtiles", "zarr", "lidar", "gaussian-splat", "3d-tiles",
"cog", "flatgeobuf", "geoparquet", "duckdb-query", "deckgl-viz",
"video", "image",
] as const;
export type LayerType = (typeof LAYER_TYPES)[number];
```

注释说明了为什么要这么写：「**as a runtime list so untrusted input (an imported Layer Library bundle, a hand-edited project) can be validated against it**」，而且 `LayerType` 由数组派生，「**so the two cannot drift**」。

启发：一份定义同时得到运行时校验数据和编译期类型，还杜绝了两者的漂移。TS 项目里凡是「有限枚举 + 需要校验外部输入」的场景都该这么写。

模式二，状态是扁平记录 + 极小的视图状态。 `MapViewState` 就五个字段：

```
export interface MapViewState {
  center: [number, number];
  zoom: number; bearing: number; pitch: number;
  bbox?: [number, number, number, number];
}
```

这个「小」是刻意的。状态越小，能序列化进工程文件的把握越大，能接第二个消费者（第二个地图面板、第二个渲染器、一个导出器）的成本越低。

模式三，跨语言边界用 SYNC 标记。这是翻阅源码过程中最意外的收获。

那 17 个矢量扩展名，在 TS 里是 `VECTOR_FILE_DIALOG_EXTENSIONS` (`lib/file-io/paths.ts:39`)，在 Rust 里是 `RESTORABLE_VECTOR_EXTENSIONS: [&str; 17]` (`src-tauri/src/lib.rs:800`)——**两份，因为跨语言没法共享常量**。他们的处理是在两边都写注释，下面这段原样引自 `lib/file-io/paths.ts:35-38`：

SYNC: `RESTORABLE_VECTOR_EXTENSIONS` in `src-tauri/src/lib.rs` must list the same extensions, or a format added here would be rejected by the Rust restore guard on every project reopen (**the bug this PR fixes**). Grep "SYNC:" to find the partner list.

注意括号里那句「the bug this PR fixes」——它指的是当初加上这条注释的那个 PR，对后来的读者已经没什么指向性了，但它记录的故障是真的：在 TS 里加了新格式，Rust 侧的守卫没加，于是每次重开工程都被拒。

启发：能合并成一份就合并；合并不了（跨语言、跨进程、跨仓库）就用一个统一的可 grep 标记把它们钉在一起，并在注释里写清楚「不同步会出什么事」。这比「大家注意保持一致」有用得多。

五、离线能力：Workbox 三层缓存策略

这节单独拎出来，因为国内做内网、离线、政务项目的人特别需要，而 docs/architecture.md 的「Offline support (PWA)」一节把这套写得非常完整。

Web 构建是一个可安装的 PWA，用 vite-plugin-pwa + Workbox。缓存刻意分成三层：

层	策略	内容	为什么这么分
预缓存	Precache	HTML + 启动地图必需的 JS/CSS chunk	首访之后无网也能开壳；重型 chunk 排除在外，避免首屏巨量下载
同源运行时缓存	CacheFirst	/assets/ 下的内容哈希产物： MapLibre、DuckDB-WASM 及其 spatial 扩展、各插件 chunk	哈希文件名让 CacheFirst 安全——重新部署会生成新 URL，旧条目不会被当成新的
CDN 引擎缓存	CacheFirst（独立规则 geolibre-cdn-engines）	jsDelivr 上的 Pyodide、PGLite/PostGIS、CereusDB、gdal3.js	URL 里嵌了精确版本号，同样不会供旧版；jsDelivr 的 CORS 头让它们是可正常校验和淘汰的 200，不是 opaque 响应

所以这些 CDN 引擎的准确表述是：在 Web PWA 里，首次「成功」取回需要网络——CacheFirst 只有在真的存下了一份响应之后才会走缓存——此后才离线可用。桌面端根本不注册 Service Worker，这条对它不成立，见下面的要点。

那为什么不直接全打包进来？往下看这几个细节就明白了。

几个特别实在的细节：

- 想去掉「首次也要联网」，可以用 `GEOLIBRE_PGLITE_CDN=0`、`GEOLIBRE_CEREUS_CDN=0` 把它们塞回 `/assets/` ——代价是 **PGLite 一个就给 Tauri 二进制加回 ~22 MB**
- **Pyodide 和 gdal3.js 没有这个开关**。Pyodide 永远走 CDN（`VITE_PYODIDE_INDEX_URL` 能改镜像，但那个镜像不在两条 **CacheFirst** 规则的匹配范围内，除非把它放到 `/assets/` 下，否则只能退回普通 HTTP 缓存）；gdal3.js 从不 vendor，`GEOLIBRE_GDAL_CDN=0` 的效果是**关掉那个导出功能**，不是打包它
- **要去掉所有由 GeoLibre 控制的外部 CDN 引用**（适用于不允许引用任何不受信外部 CDN 的部署），设置 `GEOLIBRE_NO_EXTERNAL_CDN=1`。它隐含所有 `*_CDN=0` 标志，并额外禁用那些源码中嵌入了 CDN URL 的功能（故事地图 HTML 导出、内置检测模型、ONNX WASM、3D Tiles 解码器路径、Pyodide 默认 URL）。第三方 npm 包内部的 CDN URL 字符串不受 GeoLibre 控制，无法移除
- **桌面构建不装 Service Worker**（Tauri 自带资源已离线），所以桌面版每次安装都会重新拉一次 CDN 引擎
- 底图只缓存 CORS 友好的默认源（OpenFreeMap、CARTO），其它远程瓦片和 WMS/WFS **按设计就是离线不可用**
- 新部署用 `registerType: "autoUpdate" + skipWaiting`，但故意压掉了 **Workbox 默认的「激活时强制刷新」**——因为在 `/demo/` 这种相对 base 的子路径下会误触发，把用户正在编辑的地图状态冲掉。页面恢复交给 `installStaleChunkReload`，**只在孤儿 lazy chunk 404 时才重载**，还带冷却保护

最值钱的一条

最后这条是整节最有价值的——「只有经历过实际生产问题才会写出来」的代码。自动更新导致用户丢失未保存工作，是 PWA 最容易犯又最伤人的错误。

借鉴要点

- 一、缓存按「壳 / 同源重资源 / CDN 引擎」分三层，别用一条规则打天下。
- 二、内容哈希或版本号是 CacheFirst 能安全使用的前提。没有它，CacheFirst 就是「永久供应旧版本」。
- 三、明确写下「哪些东西离线一定不可用」（远程服务、非白名单底图），别让用户去猜。
- 四、自动更新不要强制刷新页面。只在检测到孤儿 chunk 时重载，并加冷却。

六、云原生格式：它换掉的不是前端库，是整个分发架构

前面五节讲的都是代码，这一节讲的东西影响的是**架构和成本**。

传统链路：数据入库 → GeoServer / ArcGIS Server 发布 → 切瓦片 → 前端请求服务。需要维护一台服务器、一套发布流程、一份切片缓存，还要为它做监控、备份和扩容。

云原生链路：数据转成 COG / GeoParquet / PMTiles / FlatGeobuf → 放到任何能响应 Range 请求的静态存储 → 前端按需取字节区间。服务器这一环整个消失了，剩下的只是对象存储的流量费。

关键词是 HTTP Range。这些格式的共同点不是「压缩得更好」，而是**每个文件都把自己的布局元数据带在身上——头部或尾部、瓦片目录、R 树索引、金字塔概视、行组统计——**客户端先读几 KB 这样的元数据，算出自己要的是哪几段字节，再发一个 Range 请求取回来。所以静态存储就够了，不需要任何「服务」。

6.1 它到底好在哪：五条实打实的收益

收益一，下载量按「要看什么」收敛，而不是按「文件有多大」。

这是最根本的一条。传统格式（GeoJSON、Shapefile、条带式 GeoTIFF）**没法按字节区间局部读取——即便有索引也在旁挂的独立文件里，客户端仍然只能把数据整份拿下来。**云原生格式把索引放进数据文件自身，于是有了三个维度的裁剪：

裁剪维度	靠什么实现	效果
空间	FlatGeobuf 的 R 树索引、PMTiles 的瓦片编排	只取视口内的要素/瓦片
分辨率	COG 的金字塔（overviews）、PMTiles 的缩放层级	看全国就取最粗那一层，不解一亿像素

裁剪维度	靠什么实现	效果
属性列	GeoParquet 的列式存储	只要 3 个字段就只读 3 列的字节，其余列一个字节都不碰

第三条是 **GeoParquet 最容易被低估的地方。列式存储 + 行组统计信息 (min/max) 意味着 WHERE 条件能被下推**——DuckDB 先看统计信息就知道整个行组都不满足条件，直接跳过，连那段字节都不用下载。

收益二，解析成本从「几十毫秒的主线程阻塞」降到接近零。

文本格式的隐性代价是 `JSON.parse`。这篇第三节 3.6 那份排查清单里有两个实测数字：**一份 22MB 的 GeoJSON，读文件约 126ms，`JSON.parse` 再花约 36ms**——而且都在主线程上。

二进制列式格式不存在这个环节：字节布局本身就是内存布局，DuckDB 读进来是向量化的列，不需要逐字符解析、不需要构造几十万个 JS 对象。**要素多的时候，真正卡住页面的往往不是渲染，是解析和 GC。**

收益三，缓存变得几乎免费。

云原生格式是**静态文件**，所以它天然吃得下整条现成的 HTTP 缓存链路：**CDN 边缘缓存、浏览器缓存、Service Worker、ETag / 不可变 URL**，一条都不用自己实现。

对比一下动态瓦片服务：切片缓存要自己建、失效策略要自己写、缓存命中率要自己盯。而 **Range 请求命中的是同一个不可变文件的同一段字节，CDN 层面就是一次普通的缓存命中**。这篇第五节那套 **Workbox 三层缓存策略**，之所以能生效，前提就是资源是静态且带内容哈希的。

收益四，并发瓶颈从你的服务器上挪走了。

瓦片服务的容量瓶颈在服务端 CPU：同时来 100 个用户就是 100 份渲染开销。**静态存储 + CDN 的扩容曲线是另一回事**——没有按用户计的渲染要扩，多来的量主要体现在流量费上，而且边缘缓存会把绝大部分请求截在离用户最近的节点。但容量规划并不会因此消失：缓存未命中仍会打到源站，对象存储也有自己的按前缀请求速率上限和出网流量账单。

这条对做政务大屏、公共服务门户这类「平时没人、开会时全省一起看」的场景特别关键：这种流量形状用瓦片服务扛，要么长期为峰值付费，要么峰值时挂掉。

收益五，链路短了一整段，运维成本跟着塌下来。

没有发布流程，没有服务进程，没有数据库连接池，没有要打补丁的中间件。数据更新就是覆盖一个文件。GeoLibre 整套「浏览器里直读远程 GeoParquet / COG / PMTiles」的能力就是这条路走通的证明，它接的那些在线目录（STAC、Source Cooperative、Overture Maps、Planetary Computer）全是这个模式。

但也得说清代价，这四条是真的：

- **小数据反而更慢**。读头部、读索引、再取数据，是好几个网络往返；几百 KB 的 GeoJSON 一次请求就完事了，套云原生格式并不划算。**阈值大概在「一次请求下不完」的量级。**
- **Range 请求数量会明显变多**。HTTP/1.1 同样支持 Range，但在这个请求量级下强烈建议用 HTTP/2 或 HTTP/3，否则连接开销会把优势吃掉。

- **服务端必须支持 Range 和 CORS**（还要 `Access-Control-Expose-Headers`），这一点比预想中更容易出问题，下一小节专门讲。
- **算力从服务端挪到了客户端**。原来服务器干的解码、过滤、拼接，现在在用户的浏览器里跑，受 ~4 GiB 内存和单线程约束（见第二节 2.5）。不是「变快了」，是「换了个地方算」——只有当那个地方算得更划算时才成立。

所以云原生格式的适用面是有边界的：**大体量、静态、多用户读、按视口取用——四条都满足时收益最大**。反过来，高频写入的动态业务数据不适合它，这也是为什么下面第 6.7 节那份清单里明确写着「PostGIS 那条不要动」。

6.2 GeoLibre 直读这些格式时走的路

格式	读取路径	关键点
PMTiles	MapLibre 自定义协议（ <code>pmtiles</code> 包）	一个文件顶替整个瓦片服务，前端只多注册一个 protocol
COG	<code>cog-tiler-wasm</code> （默认） / <code>deck.gl GPU</code> / <code>TiTiler</code>	三种引擎可切，前两种完全在客户端
GeoParquet	DuckDB-WASM <code>read_parquet</code>	可选「原地流式查询」，不必整份拷进内存
FlatGeobuf	DuckDB Spatial <code>ST_Read</code>	自带空间索引，天然适合 range 读取
MosaicJSON / STAC 清单	栅格控件读清单， 读时拼接场景	清单本身不含数据，只有一串资产地址
云优化 NetCDF/HDF5	<code>kerchunk</code> 引用清单 → <code>Zarr</code> 渲染管线	见 6.5

6.3 真正的门槛不是格式转换，是 CORS

格式转换是一次性的活儿，**CORS 是每天都会撞的墙**。源码里对 Source Cooperative 的描述很精确：数据端 `data.source.coop` 发 `Access-Control-Allow-Origin: *` 并且支持字节范围请求，所以 **PMTiles 协议、DuckDB-WASM、COG 读取器都能直连，不需要任何代理**；但它的元数据 API 一个 **CORS 头都不发**。那浏览器怎么办？桌面端走 Tauri 原生 HTTP（服务端到服务端，没有 CORS 这回事），Web 端过一个自建 Cloudflare Worker 把 JSON 重发一遍、补上 CORS 头。

「数据能直连、元数据要代理」这个判断，是接入任何云原生数据源时最先要下的。

同一个 Worker 里还有一段更有意思的注释，讲行星底图为什么必须代理：

MapLibre 用 `fetch()` 取栅格瓦片，会走 CORS 检查，所以地图渲染成一片黑。而 `openplanetarymap.org` 自己没事，是因为 Leaflet 用 `<img>` 标签加载瓦片，而 `<img>` 不做 CORS 检查。

关键陷阱

同一个瓦片地址，Leaflet 能显示、MapLibre 显示不出来，根因可能既不是代码也不是瓦片本身，而是两个库取图的方式不同。

这条链路上有三个做法值得借鉴：**服务端取数据再补 CORS 头**（避免浏览器直接发起跨域请求）、**结果放边缘缓存**（重复请求不回头）、**严格白名单绝不做开放代理**（源码原话是「keyed to a tight allowlist so it is never an open proxy」）。第三条是安全底线：能转发任意 URL 的公开代理，容易被恶意利用，后果需要自行承担。

还有个细节值得记：Source Cooperative 的未知 API 路径不返回 404，而是落到页面兜底路由，**返回 HTML、状态码 200**。所以源码每次读取都校验解析后的结构。`response.ok` 只说明网络层没出错，不代表拿到的是想要的东西。

6.4 「转成 COG」不是把后缀改成 `.tif`

COG 能被 range 直读，靠的是**文件内部有瓦片切分和金字塔（overviews）**。条带式 GeoTIFF 严格说也能 range 读——一条带的偏移和字节数就写在头里——但一个条带横跨整幅影像的宽度，取一小块地图范围要连带拉回远超所需的字节，而且没有 overviews 就没有可供缩小时使用的粗级别。后缀叫不叫 `.tif` 都改变不了这一点，况且 GeoLibre 的客户端读取器直接要求文件内部有瓦片。

GeoLibre 的处理值得借鉴：面板**先用一个 Range 请求把文件头读回来**，判断有没有内部瓦片；没有就提示在浏览器里先转成 COG（geolibre-wasm 的 `CogBuilder`，见 `packages/processing/src/cog-convert.ts`）再加载。**用几 KB 的头部换一个明确判断，而不是让用户等一次必然很慢的加载。**

三个栅格引擎的取舍源码里也写清楚了：`cog-tiler-wasm`（默认，浏览器 WASM，代价是只能用内置色带，自定义分级失效）、`maplibre-gl-raster`（GPU，符号化完整）、`titiler`（服务端）。默认值选的是最稳的那个，不是功能最全的。

6.5 数据不是云原生格式怎么办：把索引外置

NetCDF / HDF5 是科学数据的事实标准，却不是为 HTTP 设计的。GeoLibre 的做法是 **kerchunk 引用清单**：清单把 Zarr 的 key 映射到 `[url, offset, length]`（原始文件里的一段字节区间），再实现成一个最小的 `zarrита Readable`（**只要一个 `get(key)`**），直接交给项目里已有的 Zarr 渲染管线——源码注释那句「with no rewrite」是重点，**渲染代码一行没改。**

思路可以推广：**不改数据、不改渲染，只在中间加一层「key → 字节区间」的映射**。手里那些不适合流式读取的历史格式，只要能算出偏移量，就能变成按需读取的。

6.6 目录浏览器只该做一件事：产出 URL

GeoLibre 接了 STAC、Source Cooperative、Overture Maps、Planetary Computer、Hugging Face 一大堆在线数据源，靠一条原则不失控：**添加图层时特意委托给已经懂每种格式的那些控件**——PMTiles 给 `addPMTilesLayerFromUrl`，GeoParquet 给 `addVectorLayerFromUrl`，COG 给 `app.addCogLayer`。于是目录里点进来的数据和手动 Add Data 的**完全同权**：一样进图层面板、一样能改样式、一样能存进工程。

一个数据源接入 = 一个「怎么找到 URL」的适配器，读取和渲染永远只有一份实现。这是接入 N 个数据源还不失控的唯一办法。

6.7 切入顺序与借鉴要点

切入顺序（投入从低到高）：

1. 把**对外展示的静态数据**转成 PMTiles，省掉一整个瓦片服务
2. 栅格成果转成**真正的** COG（记得验内部瓦片和金字塔），前端直读或配 titiler
3. 大矢量表转 GeoParquet，用 DuckDB 直接查
4. 动不了的历史格式，考虑外置索引（kerchunk 那个思路）
5. **动态业务数据**仍然走 PostGIS，这条不要动
6. 一、本质是「索引在文件里 + HTTP Range」，所以它换掉的是服务端，不是前端库。
7. 二、先分清「数据能直连」和「元数据要代理」。代理必须白名单 + 边缘缓存，永远不做开放代理。
8. 三、对着外部 API 编程，校验解析后的结构，不要信 `response.ok`。
9. 四、多数据源接入要收敛到同一套格式读取实现，目录面板只负责产出 URL。

第 1 条投入产出比最高：一个 PMTiles 文件 + 一个静态服务器，替掉一整套瓦片发布流程，许多项目在短时间内即可完成改造。

写在最后：按投入排序的采纳清单

一天内能验证的

1. 试一次 **DuckDB-WASM Spatial**，直接读一个远程 GeoParquet，感受一下「不用后端」是什么体验
2. 把项目里的重引擎改成**按需懒加载 + 独立 chunk**，尤其是三维引擎，让它离开首屏路径
3. 把 CDN / 资源地址全部改成运行时可配置

这个迭代能做的

1. 给大数据量图层定一个明确阈值，过线换客户端切片（@maplibre/geojson-vt + @maplibre/vt-pbf）
2. 给所有异步管线接 AbortController，包括非网络的长循环
3. 检查撤销栈有没有内存预算、有没有永远保留最新快照；检查虚拟列表的排序是不是走完整数据模型
4. 用 as const 数组重写枚举，让运行时校验和类型定义同源
5. 把跨语言/跨进程的重复常量用统一的 SYNC 标记钉起来

值得立项评估的

1. 把 DuckDB-WASM Spatial 从「读格式的工具」升级成「查询层」：让图层同时是 SQL 表，用户能对两个图层写 JOIN 和 ST_Intersects
2. 把算子层去框架化，让浏览器和服务端跑同一份代码，从此不用再对账
3. 把静态数据线换成云原生格式，先从 PMTiles 开始；顺手确认存储支持 Range 和 CORS，并想清楚哪些元数据必须走代理
4. 如果做 Web 应用，按三层策略把 Service Worker 缓存重做一遍

这份清单里没有一条需要引入 GeoLibre 的代码。它最大的贡献不是又一个地图应用，而是把「一个跑在浏览器里的空间数据库，足以替代过去必须放在服务端的那些工作」这件事，用一个能跑、能装、能查源码的完整项目证明了一遍。